AI Engineering and Claude Architect · Part V, Certification

Certification Self-Evaluation

500 Questions, With Answers at the End

Practice for Claude Certified Architect – Foundations (CCAR-F), written from the official exam guide's 30 task statements and weighted the way the guide weights its domains.

Exam	CCAR-F, guide v1.0, effective July 2026
Questions	500, and all original
Answers	Answer key and explanations at the end
Time	About 2 minutes per question, as on the exam

How to Use This Bank

Key idea

These are original practice questions, written from the public exam guide. None is a real or leaked exam question, and no score here predicts your exam result.

What the real exam looks like. According to the guide: 60 multiple-choice and multiple-response items in 120 minutes, across 4 scenarios drawn from 6, with a pass mark of a scaled 720 out of 1,000. The guide doesn't publish a raw-score equivalent, so a scaled 720 doesn't mean 72% correct.

How this bank differs. Items are grouped by domain and aren't presented as scenario sets. Each names its scenario, so you can practise reading in context. Exam items may be longer.

Where the guide and current products differ, the answer follows the guide, because that's what the exam is written against. An answer marked Build today tells you what current documentation says. Check for a newer guide before you sit the exam.

Suggested method:

1. Do one domain at a time, without notes, marking your answers on paper.
2. Check them against the answer key, then read the explanation for each one you missed or guessed.
3. Record your percentage per domain below. Note the task statement of each miss, as that shows what to restudy.
4. Retake your misses a few days later.

Score sheet

These targets are my own study suggestions, and not official figures.

Domain	Questions	Exam weight	Your score	Target
1. Agentic Architecture and Orchestration (Q1–135)	135	27%		80%
2. Tool Design and MCP Integration (Q136–225)	90	18%		80%
3. Claude Code Configuration and Workflows (Q226–325)	100	20%		80%
4. Prompt Engineering and Structured Output (Q326–425)	100	20%		80%
5. Context Management and Reliability (Q426–500)	75	15%		80%

Aim for 80% overall, with no domain below 70%.

Your notes

Domain 1: Agentic Architecture and Orchestration

Q1. Code Generation · Hard. Two developers use a named session. Which limitation is most relevant when one of them refactors code?

A. The session is locked, so the second developer can't commit to the repository

B. The refactor is automatically reverted when the session is next resumed

C. The session's memory of files can diverge from the repo unless it's told what changed

D. Session names are global, so renames will propagate to all of the team's repos

Q2. Multi-Agent Research · Hard. After synthesis, the coordinator finds that the draft has no coverage of regulation. What's the best next step?

A. Re-delegate targeted queries on regulation to search and analysis, then re-invoke synthesis

B. Accept the draft, since all the subagents have reported that they completed successfully

C. Ask the synthesis agent to write the regulation section from its general knowledge

D. Restart the entire pipeline from scratch, with the same queries and same subagents

Q3. Code Generation · Medium. After resuming, the agent re-reads your entire repo to catch up, using most of its context. What would've been better?

A. Skipping any mention of changes, to save a few tokens

B. Naming the specific changed files for targeted re-analysis

C. Forking first, so that the re-read can be done twice

D. Raising `max_tokens`, so that the re-read completes faster

Q4. Multi-Agent Research · Easy. What do subagents automatically inherit from the coordinator's conversation?

A. All of the tool results, but none of the reasoning
B. Nothing: their context must be explicitly provided
C. The full message history, up to the point of spawning
D. The system prompt and the last ten messages

Q5. Multi-Agent Research · Easy. Which is a good way of partitioning a research topic across subagents?

A. Give each one the same full topic and hope they differ
B. Give each one a different temperature setting
C. Give each one a distinct subtopic or source type
D. Give each one the same sources in a different order

Q6. Developer Productivity · Hard. A pipeline step generates a migration script, and another applies it to production. How should "human approval before apply" be implemented?

A. A line in CLAUDE.md asks all developers to review each migration
B. The apply tool is blocked in code until a recorded approval exists
C. The apply tool's description says that human approval is expected
D. The generating agent is told to wait until a person has said yes

Q7. Customer Support · Medium. Policy: refunds above $500 must go to a human. What's the right mechanism?

A. A tool description that says "please use only for amounts up to $500"
B. A hook that intercepts large `process_refund` calls and escalates them
C. A few-shot example of a $600 refund being escalated, added to the system prompt
D. A daily audit that detects any refunds above $500 and sends an alert to managers

Q8. Multi-Agent Research · Easy. Which risk comes from overly narrow task decomposition?

A. Faster but more expensive subagents
B. Loss of the `custom_id` correlation field
C. Incomplete coverage of broad topics
D. Too many duplicate tool definitions

Q9. CI/CD · Medium. A 20-file review is done in a single pass, with uneven depth. What's the best restructuring?

A. Reviewing only the five largest files, as they have the most code
B. One pass, asking the model to spend equal attention on each file
C. Per-file local passes, followed by a separate cross-file integration pass
D. One pass using a larger model, with a higher limit on output tokens

Q10. Customer Support · Medium. Which is an anti-pattern in a handoff to a human?

A. Forcing the customer to repeat everything already established
B. Including the customer ID and the relevant order number
C. Stating the root cause of the problem that's been identified
D. Recommending an action for the human agent to consider

Q11. Customer Support · Medium. Your gate blocks `process_refund` because verification hasn't happened. What should the agent receive?

A. A fake success, so that the customer isn't worried by the delay
B. No response, so that the model will time out and then try again
C. A clear error result stating that verification is required first
D. A generic "Operation failed" with no other explanation at all

Q12. Developer Productivity · Hard. Select the two signals that a task needs dynamic decomposition. **Select 2.**

A. The output must follow a fixed JSON schema in all cases
B. The same steps have run unchanged for many past examples
C. The scope isn't known until some exploration has been done
D. The task has a strict deadline measured in milliseconds
E. The next step depends upon what the current step reveals

Q13. Multi-Agent Research · Easy. A coordinator can't spawn subagents, even though they're defined. What's the first configuration check?

A. Whether `allowedTools` includes "Task"
B. Whether the model supports forked sessions
C. Whether the subagents share its history
D. Whether `stop_reason` is set to "Task"

Q14. CI/CD · Hard. A team proposes dynamic decomposition for its standard, unchanging PR checklist. What's the best response?

A. Neither pattern can apply to a checklist, which should only be handled by one long prompt
B. A prompt chain is simpler and more predictable here, as the steps don't depend on findings
C. Dynamic decomposition is required whenever a workflow has at least three separate steps
D. Dynamic decomposition is always better, as adaptivity improves every workflow it's used on

Q15. Customer Support · Medium. How should a prerequisite gate for `process_refund` decide that it can proceed?

A. It asks the model whether it believes that the customer has now been verified
B. It checks that the word "verified" appears in the customer's latest message
C. It waits ten seconds, to give the verification tool enough time to complete
D. It checks that `get_customer` returned a verified customer ID in this session

Q16. Developer Productivity · Hard. Select the two practices for a resumed session after code changes. **Select 2.**

A. Re-explore the entire codebase from the beginning
B. Fork the session and delete the original branch
C. Assume that it has noticed the changes itself
D. List the files that changed since it last ran
E. Ask for re-analysis of those specific areas

Q17. Developer Productivity · Hard. An agent's investigation spans several days. What approach best keeps reliability high?

A. One session kept open permanently, which never needs anything else at all
B. A new blank session every morning, with nothing carried over from the last
C. Forking at each message, in order to keep every possible branch available
D. Named sessions, change notes, and a fresh summary if much goes stale

Q18. Customer Support · Hard. An agent's loop runs correctly, but it sometimes acts on an outdated order status after a tool result was silently dropped by a filter. What does this show most directly?

A. The `stop_reason` was wrong, and it should have returned `tool_error` for the dropped result
B. The issue can only be solved by raising the iteration cap, so the model has time to recheck
C. The model reasons only from what's in the conversation, so a missing result leaves it working on old information
D. The model automatically retrieves current data from the server, so the filter must have broken that sync

Q19. Customer Support · Hard. Select the two controls that provide deterministic enforcement. **Select 2.**

A. A note in the tool's description about when it should be used
B. A prerequisite gate that blocks a tool until an earlier step succeeds
C. A few-shot example showing the correct order of tool calls
D. A system prompt rule, written in bold capital letters
E. A hook that intercepts tool calls and rejects policy violations

Q20. Developer Productivity · Medium. A developer's agent must never run `rm -rf` through Bash. What's the strongest option listed?

A. A few-shot example of the agent choosing a safer command
B. A line in CLAUDE.md asking it not to run destructive commands
C. A friendlier, more detailed description for the Bash tool
D. A pre-execution hook that inspects and rejects those commands

Q21. Multi-Agent Research · Medium. When should the coordinator stop its iterative refinement loop?

A. When a subagent reports that it would like to stop
B. After exactly five rounds, regardless of what's been found
C. As soon as the first synthesis draft has been returned
D. When coverage meets its stated quality criteria

Q22. Multi-Agent Research · Hard. A review of subagent output shows statements like "the report says" with no way to find which report. What should the prompt require?

A. Structured output with each claim's source document, page and URL as separate fields
B. Longer prose, so that each paragraph can describe sources in greater detail
C. A confidence score for each claim, as reported by the subagent itself
D. A closing bibliography, listing each source that was consulted during the task

Q23. Customer Support · Medium. A tool returns status codes such as 3 and 7, and the agent misinterprets them. What should you do?

A. Ask the model to guess the meaning of the codes from nearby context
B. Escalate every case to a human when a numeric code has appeared
C. Add every code to a long table in the system prompt, and hope it's used
D. Translate the codes into meaningful values with a `PostToolUse` hook

Q24. Customer Support · Easy. "Prompt instructions alone have a non-zero failure rate." When does this matter most?

A. When compliance must be guaranteed
B. When formatting a closing message
C. When choosing a friendly greeting
D. When suggesting a helpful FAQ link

Q25. Multi-Agent Research · Hard. A synthesis agent's definition allows Task, WebSearch and all document tools "for flexibility." It starts researching rather than synthesizing. What's the fix?

A. Restrict its tools to those its role needs, with goals in its prompt
B. Give the search agent an instruction to run before synthesis does
C. Increase its iteration cap, so that it can finish searching and then write
D. Add "do not search" to its prompt, and keep its tools as they are

Q26. Customer Support · Hard. Select the two appropriate uses for tool-call interception hooks. **Select 2.**

A. Redirecting prohibited actions to a human workflow
B. Choosing a warmer tone for customer messages
C. Deciding which questions are ambiguous to the customer
D. Normalizing tool results after they have returned
E. Blocking refunds beyond a policy limit

Q27. Developer Productivity · Medium. A developer's loop stops as soon as a response contains any text block. Users report that the agent quits halfway through tasks. What's the cause?

A. Claude can emit explanatory text alongside tool calls, so text doesn't indicate completion
B. Claude returns text only when it has failed, so the loop is detecting real errors correctly
C. The SDK removes tool calls from any response that contains a text block
D. Text blocks consume the token budget, so the model is ending its own turn early

Q28. Customer Support · Hard. A workflow must run credit check, then approval, then disbursement. Occasionally, a model reorders the steps. What's the proportionate design?

A. Code enforces order, with each step's tool gated on the previous step's real result
B. A longer system prompt, with a numbered list and bold warnings on each of the steps
C. A second agent that reads the logs each night and reverses wrong disbursements
D. A larger model, as more capable models will always follow the order correctly

Q29. Developer Productivity · Medium. A subagent needs the project's coding conventions, which the coordinator knows. How does it get them?

A. It asks the coordinator in mid-run through a shared channel
B. They're included in the subagent's prompt by the coordinator
C. It inherits them from the coordinator's history automatically
D. It can't receive them, because subagents only use their tools

Q30. Customer Support · Medium. A tool fails with a timeout. How should the loop report this so the model can decide what to do?

A. Stop the loop immediately and return a generic apology to the customer
B. Retry forever inside the tool, and never return to the model until it works
C. Omit the failed tool's result, since the model only needs successful results
D. Return a `tool_result` describing the failure, then keep looping

Q31. Developer Productivity · Hard. Select the two statements that explain why text checks are unreliable as completion signals. **Select 2.**

A. Text blocks are returned only when streaming, and not otherwise
B. Text blocks are charged at a higher rate, which distorts the check
C. A completed answer's wording varies, so no phrase can be relied on
D. Claude may write text and also request tools in the same response
E. The API encrypts assistant text until the final turn has been sent

Q32. Multi-Agent Research · Medium. A team sets `max_iterations = 3` as its loop's main stopping rule, and research tasks now end with half-finished reports. What should change?

A. End on `stop_reason`, and keep any cap only as a safety limit

B. Raise the cap to 30 and continue to treat reaching the cap as the normal way to finish

C. Remove the tool results from history so that each of the three iterations can do more

D. Ask the model to complete all of its research within three turns, using the system prompt

Q33. Customer Support · Hard. A refund agent must handle unpredictable customer requests, though identity verification must always happen before refunds. Which design fits best?

A. A model-driven loop, with a code gate enforcing verification

B. A fixed decision tree covering every request type that you can think of in advance

C. Separate agents for each request type, each one with a fixed sequence of three tools

D. A model-driven loop, with a firmly worded prompt requiring verification before any refund

Q34. Customer Support · Easy. Which hook type intercepts tool results for transformation before the model processes them?

A. `OnStopReason`

B. `PostToolUse`

C. `ToolPrompt`

D. `PreRequest`

Q35. Developer Productivity · Medium. An MCP tool returns paths relative to various roots, confusing the agent. What's the most reliable fix?

A. Ask the agent in the prompt to figure out each tool's root

B. Restart the agent whenever a relative path gets returned

C. Convert paths to a common form with a `PostToolUse` hook

D. Add more tools, with one tool for each root directory

Q36. Code Generation · Medium. What's the main risk of resuming without saying that files have changed?

A. A failed resume, with an error about the changed files
B. Automatic rollback of any changes that you've made
C. Immediate loss of the session's whole conversation history
D. Confident recommendations based on code that no longer exists

Q37. Multi-Agent Research · Hard. The synthesis subagent's output contradicts one source, and you want to investigate. In a hub-and-spoke design, what makes this easiest?

A. The final report's text includes a transcript of every model call
B. Subagents can be asked afterward what they remember about the earlier run
C. Each subagent stores its private notes in a hidden directory for later reading
D. All exchanges pass through the coordinator, so one place shows them

Q38. Customer Support · Hard. A `PostToolUse` hook trims order records from 40 fields to 5. What does this help with?

A. Schema validity: tool inputs now pass strict validation on every call
B. Context use: irrelevant fields no longer accumulate across turns
C. Rate limiting: fewer fields mean that each tool can be called less often
D. Security alone: fields are hidden from logs, but the model still sees them

Q39. Code Generation · Hard. A resumed session references a function that was renamed last night, and proposes edits based on the old code. What's happening?

A. It's reasoning from stale tool results held in its history
B. Its model has been downgraded since the last session ran
C. Function renames always invalidate a session's stored memory
D. `--resume` loads the wrong session whenever a file has changed

Q40. Developer Productivity · Medium. "Add comprehensive tests to this legacy codebase." What's the best way to begin?

A. Map the structure, find high-impact areas, then build a prioritized plan that adapts
B. Start with the smallest files, because small files will be the easiest to cover
C. Ask for a single prompt to generate all of the tests in one very large response
D. Write tests for each file in alphabetical order until the budget has been spent

Q41. Customer Support · Hard. A hook redirects over-limit refunds to `escalate_to_human`. What should the agent's escalation include?

A. An apology to the customer and no further details, in order to protect their privacy
B. The customer's details, the amount, and the reason for the block, in a structured summary
C. Only the amount, as the human can find the rest in the hook's own server logs
D. Nothing further, as the hook has handled the entire case on behalf of the agent

Q42. Multi-Agent Research · Medium. A bug investigation changes direction after each log file is read. A fixed pipeline keeps doing irrelevant steps. Why?

A. The model needs a bigger context window for its plan to work
B. The work is exploratory, so plans need to adapt as findings emerge
C. Fixed pipelines can't read logs, so the tool must be changed
D. Investigation tasks must always use a batch API, which adapts

Q43. Data Extraction · Hard. Most invoices follow one flow, yet some have attachments needing extra investigation. What's a good design?

A. A longer chain, with attachment handling for all invoices whether they need it or not
B. Rejecting all invoices that have attachments, sending them to a separate manual queue
C. A chain for the common path, with an adaptive step or agent only for the exceptions
D. A fully dynamic agent for every invoice, so that any exception is automatically handled

Q44. Customer Support · Medium. A customer writes: "My order is late, I was double-charged, and my address is wrong." How should the agent handle it?

A. Escalate at once, since requests with three concerns are too complex for an agent
B. Treat it as one issue, and pick whichever tool seems to be most relevant overall
C. Separate the concerns, investigate each with shared context, then give a unified resolution
D. Handle only the first concern, then ask the customer to open tickets for the others

Q45. Developer Productivity · Medium. An agent should explore two migration approaches from the same analysis, without one affecting the other. What fits?

A. A new blank session for each approach, with no prior context
B. Fork-based sessions from the shared analysis baseline
C. One session in which both approaches are tried alternately
D. Two subagents sharing one memory file that both can overwrite

Q46. Code Generation · Medium. Your session has a valid analysis, and just one changed file. Which approach fits?

A. Start fresh, as resuming is always less reliable
B. Export the transcript into a new system prompt
C. Fork twice, and compare the two forks' answers
D. Resume, and point it at the one file that changed

Q47. Developer Productivity · Hard. A loop has a hard limit of 50 iterations that's almost never reached. When it is reached, it logs an alert and stops. How should an architect judge this?

A. It's reasonable, since it's a safety limit and `stop_reason` still ends normal runs
B. It's an anti-pattern, as every iteration cap in an agent is forbidden
C. It's the primary stopping mechanism, so it must be lowered to about 5 to be safe
D. It's unnecessary, because the API automatically stops agents at 50 turns on its own

Q48. Multi-Agent Research · Hard. A coordinator runs three research subagents with identical instructions, hoping for broader coverage, and receives near-duplicate results. What's the root cause?

A. The subagents' context is too isolated, so they need to share a history
B. Three is too few, so more subagents would naturally diversify the results
C. The synthesis agent isn't deduplicating, so it needs a stricter prompt
D. Scope wasn't partitioned, so each agent explored the same space

Q49. CI/CD · Medium. Why add a cross-file pass after per-file analysis?

A. It gives a second chance to repeat each of the per-file findings word for word
B. The API requires a final pass that references every previous tool result
C. Per-file passes only ever detect style issues and never any real bugs
D. Per-file passes can't see data flow or contracts spanning several files

Q50. Customer Support · Medium. An engineer's loop checks whether the assistant's text includes "I have completed your request." What's the main problem with this design?

A. It prevents tool results from being appended to the history between iterations
B. It uses too many tokens, because the string comparison adds to each request's input
C. It only works with streaming responses, and not with standard requests
D. It relies on natural language, which can vary or be missing, to control the program's flow

Q51. Multi-Agent Research · Hard. Select the two signs that a report's coverage gap comes from the coordinator's decomposition rather than from the subagents. **Select 2.**

A. The synthesis agent used a shorter prompt than the others did
B. The delegation prompts never mentioned the missing subtopic
C. The final report's formatting differs from the agreed template
D. The subagents completed what they were assigned, and did it correctly
E. The search tool occasionally returned transient timeout errors

Q52. Developer Productivity · Hard. After a crash, you could resume or start fresh. Half of the earlier results concern files that have since changed. Which factor should decide?

A. Whether the model has changed its version
B. How many tokens the earlier session has used
C. Whether the session has a memorable name
D. How much of the prior context is still valid

Q53. CI/CD · Hard. Select the two tasks best suited to prompt chaining. **Select 2.**

A. A review that always checks security, then performance, then style
B. Exploring a legacy system whose structure nobody understands
C. Finding the root cause of an intermittent production outage
D. Researching an unfamiliar market with no fixed line of inquiry
E. Extract fields, validate, then format for a downstream system

Q54. Developer Productivity · Easy. Which is a correct description of a single iteration of the agentic loop?

A. Send the request, inspect `stop_reason`, execute any tools, and append the results
B. Send the request, wait for a webhook, and then read the results from a queue
C. Execute all tools, send the request, parse the text, and delete the older results
D. Inspect the text, choose a tool in your own code, and then send the request

Q55. Multi-Agent Research · Hard. A subagent's prompt says only: "Now write the summary based on what we found." What's wrong?

A. It should be phrased as a list of steps, as only step lists are valid in prompts
B. "What we found" means nothing to it, since it has no access to prior context
C. The sentence is too short, as subagent prompts have a minimum required length
D. It needs a forced `tool_choice` before it can write any free-form summary

Q56. Developer Productivity · Hard. Your hook normalizes results, and now an upstream tool changes its format. Results pass through un-normalized, and the agent is confused. What practice would catch this soonest?

A. Removing the hook, because the model is able to handle most formats reasonably well
B. A monthly review of the agent's reasoning, sampled from a few of its conversations
C. Tests on the hook, plus flagging of unknown formats
D. Asking the model to report any time that it feels confused by the data it's reading

Q57. Multi-Agent Research · Hard. Select the two things a coordinator needs in order to spawn a synthesis subagent that uses earlier research. **Select 2.**

A. `tool_choice` set to "any" on the coordinator
B. "Task" in the coordinator's `allowedTools`
C. The research findings placed in the subagent's prompt
D. The coordinator's history copied to shared memory
E. A `--resume` of the search agent's session

Q58. Developer Productivity · Hard. An agent is mapping a legacy system, and discovers that a "minor" module is used everywhere. What should a good plan do?

A. Stick to the original plan to stay deterministic
B. Re-prioritize, given the newly discovered dependency
C. Restart the entire exploration from the beginning
D. Ask the user to rewrite the module before going on

Q59. Customer Support · Easy. A response arrives with `stop_reason: "tool_use"`. What should your loop do next?

A. Ask the model in a follow-up message whether it really wants the tool to run
B. Run the requested tools, append the results to the history, and send another request
C. Start a new conversation containing only the tool output, to keep the context small
D. Return the response text to the customer, because a tool call means the turn has ended

Q60. CI/CD · Hard. Per-file reviews flag a pattern as bad in one file and accept it in another. What helps most?

A. Merging all the files back into a single pass, so that the model can see them together
B. Shared explicit criteria per file, plus an integration pass to reconcile findings
C. Removing the integration pass, because that pass is likely causing the contradictions
D. Asking the model to be more consistent, with a line at the top of the prompt

Q61. Code Generation · Hard. Select the two conditions that favor starting a new session with a summary over resuming. **Select 2.**

A. The session has an easily remembered name
B. Many earlier tool results now describe outdated code
C. The session's analysis is valid, with one small change
D. The prior analysis was based on a different branch
E. You want to keep all of the session's detailed history

Q62. Code Generation · Medium. Most tool results in a saved session are stale after a large rebase. What's the most reliable way to continue?

A. Resume it and run `/compact`, as that will refresh all of the tool results
B. Start a new session with a structured summary of what's still valid
C. Resume it as it is, and rely on the model to notice the stale results
D. Fork the session, since forks automatically re-read files that have changed

Q63. Code Generation · Medium. What's a good structured summary for starting a fresh session?

A. A raw paste of all the earlier tool results from the previous session
B. Key decisions, current state, file locations, and what remains to be done
C. A single sentence which says to continue where we had left off
D. The previous session's name, so that it can load the context itself

Q64. Multi-Agent Research · Hard. A coordinator summarizes findings to "keep prompts small" before passing them to synthesis. Numbers and sources go missing from reports. What's the better trade-off?

A. Pass the shortest summary possible, and ask synthesis to re-derive the numbers
B. Pass structured findings containing the facts and sources, and drop verbose reasoning
C. Pass all the raw transcripts, because more text always gives more accurate reports
D. Pass nothing, and let the synthesis agent perform the entire search again itself

Q65. CI/CD · Easy. When is prompt chaining the better decomposition pattern?

A. Only for tasks that can be completed in a single short model response
B. Whenever a task is expected to need more than one tool during its run
C. For open-ended investigation, where the steps can't be known in advance
D. For predictable, multi-aspect work, such as a fixed review checklist

Q66. Customer Support · Medium. Which task is a poor fit for a hook?

A. Converting all timestamps into a single standard format
B. Removing unused fields from a large order record
C. Blocking any refund above a set limit in dollars
D. Judging whether a customer's explanation sounds reasonable

Q67. Customer Support · Easy. Why would you choose a hook over a prompt for a refund limit?

A. Hooks use more tokens than prompts do
B. Prompts can't mention amounts of money
C. The rule requires guaranteed compliance
D. The API forbids limits in system prompts

Q68. Customer Support · Medium. A support coordinator delegates to billing and shipping subagents. The billing agent hits an unrecoverable error. Which design gives the most consistent handling?

A. The billing agent directly tells the shipping agent to stop its work
B. The billing agent reports the failure, and the coordinator decides
C. The coordinator is bypassed so that the customer gets the raw error faster
D. Each subagent handles every failure independently and never reports back

Q69. Customer Support · Hard. A customer has a damaged-item claim and a billing question. Investigating them in parallel found the same order ID relevant to both. What does "shared context" add?

A. It forces the two issues to be merged into one tool call for lower API costs
B. It means that only one of the two issues needs a resolution in the final reply
C. Each investigation can reuse known facts, and the final response stays consistent
D. It lets the agent skip verifying the customer, as the order is already known

Q70. Multi-Agent Research · Medium. Your loop treats an empty text response as the signal that work is finished. What's likely to go wrong?

A. A turn can hold only `tool_use` blocks, so work may remain
B. Empty text means the model refused, so this would hide safety issues in production
C. Claude always includes text with its final answer, so the check would never fire at all
D. Empty responses always raise exceptions, so that code path could never actually execute

Q71. Customer Support · Hard. You've normalized dates with a hook. A new tool is added that returns a different date format. What's a good design consideration?

A. Keep normalization rules central, so a new format is added in one place and tested
B. Require the new tool's team to stop returning dates in any of their responses at all
C. Move the normalization into the prompt, since prompts adapt to new formats by themselves
D. Add a subagent for each new tool, with a prompt that explains how it formats dates

Q72. Multi-Agent Research · Medium. You want three search subagents to run in parallel. How should the coordinator do this?

A. Emit one Task call per turn, for three turns

B. Start three separate coordinators with one task each

C. Emit multiple Task calls in one response

D. Set a `parallel: true` flag on the system prompt

Q73. Multi-Agent Research · Medium. How should the coordinator pass web results and document analysis to the synthesis agent?

A. Rely on the shared memory that's created between invocations in a session

B. Tell it which agents ran, so that it can query their conversation histories

C. Provide only a one-line summary, and let it search again for the details

D. Include the complete findings directly in the synthesis subagent's prompt

Q74. Multi-Agent Research · Hard. You're adding a fact-checking subagent. Which change respects the coordinator pattern?

A. It replaces the coordinator as the hub whenever facts are being checked

B. It subscribes to all subagent conversations so it can check facts as they happen

C. The synthesis agent spawns and manages it directly, without the coordinator knowing

D. The coordinator invokes it when needed and passes its findings to other agents

Q75. Multi-Agent Research · Medium. Why use structured data when passing findings between agents?

A. It automatically verifies that each of the cited claims is accurate

B. It separates content from metadata such as URLs, so attribution survives

C. It lets the model skip reading the text, which reduces its token use

D. It's the only input format that subagent prompts are able to accept

Q76. Multi-Agent Research · Medium. Which brings the most value when a subagent's job is to synthesize across twenty findings?

A. The coordinator's interpretation of the findings, and no originals
B. Each finding delivered with its source, date and key data as fields
C. A pointer to the search agent, so that it can ask about each finding
D. Each finding delivered as a long narrative with reasoning included

Q77. Developer Productivity · Medium. Your loop sends only the latest tool result in each request, to save tokens. The agent keeps repeating earlier steps. Why?

A. The API is stateless, so the model sees only what's sent, and it has forgotten earlier steps
B. Sending one tool result at a time turns on a mode that restarts the task
C. Repeating steps is expected behavior whenever a model uses more than three tools
D. The model caches earlier turns on the server, and the partial history is corrupting that cache

Q78. CI/CD · Medium. What's "attention dilution" in the context of large reviews?

A. Reviewers stop reading comments after a long CI run
B. The model's temperature rises as more files are added to it
C. Too many files in one pass gives shallow or uneven analysis
D. Tool descriptions lose effect as more tools get added

Q79. Customer Support · Hard. Select the two appropriate uses for `PostToolUse` hooks. **Select 2.**

A. Choosing which tool the model will call next
B. Rewriting the system prompt for each new turn
C. Removing irrelevant fields from verbose tool results
D. Blocking a refund before it can be executed
E. Converting heterogeneous timestamps to one format

Q80. Multi-Agent Research · Medium. A coordinator always invokes four subagents in the same order. What does this suggest?

A. It's correctly implementing dynamic selection, as the order never changes
B. It's acting as a fixed pipeline and isn't selecting agents by query requirements
C. The subagents should each decide independently whether they ought to run
D. It should be replaced by agents that communicate with each other directly

Q81. Customer Support · Medium. Which is the key difference between hooks and prompt instructions?

A. Hooks replace the system prompt entirely once at least one has been registered
B. Hooks are faster than prompts, while prompts are more reliable than hooks
C. Hooks give deterministic guarantees, while prompts give probabilistic compliance
D. Hooks only apply to built-in tools, while prompts only apply to MCP tools

Q82. Customer Support · Easy. What's the main purpose of a structured handoff summary?

A. Humans can act with no need for the transcript
B. It lets the agent restart itself in another session
C. It lowers the model's token use on the next turn
D. It's needed by the API before calling a human tool

Q83. Developer Productivity · Medium. An SDK agent must run tests before it can commit. What's the most reliable way to enforce this?

A. A reminder message inserted into the conversation after every tenth model turn
B. A tool description for commit that politely suggests that tests may be useful
C. A CLAUDE.md instruction, saying that tests are always to be run before a commit
D. A hook or gate that blocks the commit tool unless a passing test result exists

Q84. Customer Support · Hard. An agent decomposed a three-part request, resolved two, and one needs a human. What's the best outcome for the customer?

A. Resolve two, and escalate the third with a structured summary
B. Escalate all three together, so that one human can handle the whole request at once
C. Keep trying the third autonomously, because partial resolution counts as a failure
D. Close the case after telling the customer that two issues out of three have been solved

Q85. Multi-Agent Research · Medium. A search subagent keeps finding sources that another subagent has already covered, which wastes budget. What's the best first fix?

A. Have the coordinator assign distinct subtopics or source types
B. Reduce the number of subagents to one, so that duplication can't happen at all
C. Let the subagents message each other directly to compare their results as they go
D. Add a deduplication service in front of the search tool, and cache every query

Q86. Developer Productivity · Medium. You want to compare a mock-based testing strategy with an integration-based one, from one codebase analysis. What's the best method?

A. Finish the analysis, then fork a branch for each strategy
B. Run the analysis twice, in two separate new sessions
C. Try both strategies in one session, alternating between them
D. Ask a subagent to write both of them into one shared file

Q87. Customer Support · Medium. Which is the best recommended action field in a handoff?

A. "Customer seems upset, and would probably appreciate some kind of goodwill gesture."
B. "The agent could not finish this one, so please take a look when you are able to."
C. "See the conversation for full details about what has happened with this customer."
D. "Approve $212 refund for order 7781; damage photo verified, above agent's limit."

Q88. Developer Productivity · Medium. Which is the best first step for "understand how payments work in this monolith"?

A. Ask the model to describe how payment systems typically work in monoliths
B. Search for entry points, then trace the flows outward, adjusting as you learn
C. Read every file in the repository from top to bottom, before forming a view
D. Write a fixed list of ten files to read, based upon their names alone

Q89. Customer Support · Medium. Three MCP tools return dates as Unix timestamps, ISO 8601 and "Jan 3." The agent misorders events. What's the best fix?

A. A longer system prompt, explaining how to convert each of the three formats
B. A separate subagent, whose only job is to convert dates when they appear
C. A `PostToolUse` hook that normalizes dates before the agent sees them
D. A change to the model's temperature, so that it reads dates more consistently

Q90. Multi-Agent Research · Easy. Which of these does the guide list as part of `AgentDefinition`?

A. Tool restrictions
B. Tokenizer choice
C. Network routes
D. Billing limits

Q91. Customer Support · Hard. Compliance asks whether "the agent's prompt requires verification" is an adequate control. What's the most accurate answer?

A. Yes, as long as the requirement is repeated in the user turn as well as the system prompt
B. Yes: Claude always follows system prompts, so a rule in the prompt is itself a guarantee
C. No: it's guidance with some failure rate, so a programmatic gate is required for a guarantee
D. No: the only adequate control is to remove all automated refunds, permanently, from the agent

Q92. Customer Support · Medium. A refund limit is enforced in a hook, and also stated in the system prompt. Is this redundant?

A. Yes: having both causes the model to fail, so the prompt line must be removed
B. No: both give guarantees, so one acts as a backup if the other fails
C. Yes: if there's a hook, the model will be told automatically by the API
D. No: the prompt helps the agent plan and explain, while the hook guarantees the limit

Q93. Customer Support · Medium. An agent's system prompt requires `get_customer` before `process_refund`. It complies 98% of the time. Why isn't that enough for this rule?

A. It's good enough, since all software has bugs and 2% is within normal tolerance
B. The other 2% could be fixed by repeating the instruction three times in the prompt
C. 98% is below the threshold at which the API enforces tool order on its own
D. Prompt guidance has a non-zero failure rate, and each failure here moves real money

Q94. Developer Productivity · Hard. Select the two uses for fork-based session management. **Select 2.**

A. Trying two testing strategies without cross-contamination
B. Running the same task for a guaranteed identical result
C. Making a session survive after its context window is full
D. Comparing two refactoring approaches from one analysis
E. Passing context to subagents more cheaply than prompts do

Q95. Customer Support · Medium. Select the two practices that make an agentic loop correct. **Select 2.**

A. Discard prior turns once a tool has returned, to keep requests small
B. Append tool results to the history before the next request
C. Treat a fixed iteration count as the normal end of a run
D. Continue while `stop_reason` is `tool_use`
E. End the loop as soon as any assistant text appears

Q96. Customer Support · Easy. Your support agent's loop must decide whether to run another iteration. Which signal should control this?

A. Whether the reply contains the word "done" or "finished"

B. A fixed count of five iterations for every request

C. The `stop_reason` field on each response

D. Whether the response contains any assistant text at all

Q97. Customer Support · Medium. A refund step is gated on verification. Later, a new tool `issue_credit` is added, also moving money. What should an architect check?

A. That `stop_reason` is changed for financial tool calls

B. That the same prerequisite applies to the new tool

C. That customers are told about the new tool in advance

D. That the new tool has a longer description than others

Q98. Multi-Agent Research · Medium. A coordinator's response has two `tool_use` blocks. How should your loop return results?

A. Send a separate user message for each result, with a model call between them

B. Run only the first tool, since the model can use just one result in each turn

C. Merge both outputs into one text block in the system prompt of the next request

D. Run both, and return a `tool_result` for each, matched by ID, in the next user message

Q99. Customer Support · Hard. Which pair is best matched?

A. Tone of the reply: prompt guidance. Refund limits: programmatic enforcement

B. Both of them: strongly worded system prompt rules, with capitalized text

C. Tone of the reply: a gate. Refund limits: a polite line in the system prompt

D. Tone of the reply: a hook. Refund limits: a few-shot example in the prompt

Q100. Developer Productivity · Medium. What happens to one fork when you make changes in the other?

A. Both forks receive the changes on their next model turn
B. Nothing: their conversations are independent
C. The two forks are merged automatically once both finish
D. The older fork is closed and can no longer be resumed

Q101. Developer Productivity · Medium. Which prompt gives a code-exploration subagent the best chance to adapt?

A. "Open a.java, then b.java, then c.java, then stop and print the contents of each."
B. "Use Grep five times exactly, then Read twice, then return one sentence at the end."
C. "Find how refunds flow; report entry points and dependencies, with paths."
D. "Do whatever seems useful for a while, and report back whenever it seems appropriate."

Q102. Code Generation · Easy. Why use named sessions?

A. To turn off context limits for that conversation
B. To share CLAUDE.md files across team repositories
C. To speed up model inference for that conversation
D. To return to a specific investigation later

Q103. Multi-Agent Research · Hard. A coordinator's prompt lists twelve exact steps for each search subagent. Results are poor when sources differ from what was expected. What's better?

A. Specify research goals and quality criteria, so that subagents can adapt
B. Use forced tool choice for every step, so that the order can't change
C. Add a thirteenth step, describing what to do when a source is unexpected
D. Replace the subagents with one agent, following the same twelve steps

Q104. Data Extraction · Medium. A pipeline classifies documents, extracts fields and then validates them, always in that order. What pattern fits?

A. Dynamic decomposition
B. Coordinator re-planning at every step
C. Prompt chaining
D. An open-ended investigation agent

Q105. Multi-Agent Research · Easy. In a hub-and-spoke multi-agent system, how do the subagents communicate with each other?

A. Through the coordinator, which routes all information between them
B. Directly, with each subagent calling the others as it needs them
C. Through a shared conversation history inherited from the coordinator
D. Through a shared memory that every subagent can read automatically

Q106. Customer Support · Hard. Logs show that when a customer states "I already verified with your colleague," the agent sometimes skips `get_customer`. What's the right fix?

A. Log the event and rely on human review for unusual conversations
B. Tell the model that customers sometimes lie, and to be more careful
C. Add sentiment analysis, to detect customers who may be manipulating it
D. Gate downstream tools in code on a real verification result

Q107. Multi-Agent Research · Medium. A research agent always performs five steps, even if step 2 shows the question's already answered. What's the issue?

A. It's missing a forced tool on step 5
B. Fixed steps ignore intermediate findings
C. Five steps exceed the context window
D. Its iteration cap has been set too low

Q108. Multi-Agent Research · Hard. Every subagent reports success, yet a report on "renewable energy" covers only solar. Logs show the coordinator asked for "solar panel efficiency" and "solar costs." Where's the root cause?

A. The synthesis agent should have added wind and hydro from its own knowledge
B. The coordinator's decomposition was too narrow, so the fix belongs there
C. The report agent should reject any report that covers fewer than three sources
D. The search agent's queries were too short, so its tool needs a longer description

Q109. Customer Support · Easy. Which of these is an anti-pattern for ending an agentic loop?

A. Stopping when `stop_reason` is `end_turn`
B. Continuing when `stop_reason` is `tool_use`
C. Appending tool results before the next request
D. Parsing the reply for "all done"

Q110. Customer Support · Easy. A compliance rule says that identity must be verified before any financial operation. Which control gives a deterministic guarantee?

A. A programmatic gate that blocks the refund tool until verification has succeeded
B. A post-hoc weekly report listing refunds that had no verification step
C. Few-shot examples which show the agent verifying customers before it refunds
D. A system prompt that says, in strong words, that verification must come first

Q111. Multi-Agent Research · Hard. A team replaces its agent loop with a script that always calls search, then fetch, then summarize. Simple questions now get worse answers and waste tool calls. Why?

A. A fixed sequence can't skip or adapt its steps to the question
B. Scripts can't call tools at all, since only the model is able to execute them
C. The summarizer depends on `end_turn`, which is never emitted when a script runs tools
D. Fixed sequences always exceed token limits, so results are truncated by the API

Q112. Multi-Agent Research · Medium. What belongs in an `AgentDefinition`?

A. A description, a system prompt, and tool restrictions

B. The coordinator's complete conversation history to date

C. The API key and the billing account for each subagent

D. A list of all the other subagents' memory locations

Q113. Multi-Agent Research · Medium. A team wants subagents to ask each other questions directly because "the coordinator is a bottleneck." What should the architect weigh first?

A. Peer communication is required by the API once there are more than three subagents

B. Direct links have no real downside, so they should be used for all agent communication

C. Direct links lose central observability, so measure first and try scoped tools

D. The coordinator can't be a bottleneck, as coordinators don't use any model calls

Q114. Customer Support · Medium. Select the two items that belong in a good escalation summary. **Select 2.**

A. The list of all the tools that were available to the agent

B. The model's full internal reasoning from every turn

C. Root cause of the issue, as far as has been established

D. The recommended action, with the amount involved

E. A count of how many tokens the conversation has used

Q115. Multi-Agent Research · Medium. A team wants attribution through a chain of search, analysis and synthesis. What should each step's output preserve?

A. A link to the coordinator's logs for use in later auditing

B. The name of the model that generated each of the paragraphs

C. A single overall summary of what sources were consulted

D. A claim-to-source mapping that each later step carries forward

Q116. Multi-Agent Research · Easy. Which tool does the guide describe as the mechanism for spawning subagents?

A. Fork
B. Delegate
C. Task
D. Spawn

Q117. Customer Support · Hard. Which statement best contrasts model-driven decisions with a pre-configured decision tree in an agent?

A. Both are equivalent as long as the tools have good descriptions and strict schemas
B. Model-driven: Claude picks the next tool from context. Tree: branches are fixed
C. A decision tree always uses fewer tokens and is therefore more accurate for ambiguous requests
D. In model-driven loops, Claude follows a fixed order; a tree allows the order to change at runtime

Q118. Developer Productivity · Easy. Which is the best fit for `fork_session`?

A. Running one task in a CI pipeline
B. Reading one small configuration file
C. Fixing a single-line typo in a README
D. Comparing two refactoring approaches

Q119. Developer Productivity · Easy. Which command continues a named prior Claude Code conversation?

A. `--session load <name>`
B. `--restore <session-name>`
C. `--resume <session-name>`
D. `--history <session-name>`

Q120. Customer Support · Medium. After a tool runs, where do its results go so the model can use them?

A. In the assistant's turn, appended after the original `tool_use` block
B. In a `tool_result` block in the next user message
C. In the system prompt of the next request, replacing whatever was there before
D. In a separate API call that registers results before the next message is sent

Q121. Developer Productivity · Hard. Select the two reasons why subagents having isolated context is useful. **Select 2.**

A. Verbose exploration stays out of the coordinator's context
B. It guarantees each subagent will produce identical output
C. Each subagent can focus on its task without unrelated history
D. It removes any need to pass findings in their prompts
E. It lets subagents read each other's memory more quickly

Q122. Developer Productivity · Easy. When is dynamic adaptive decomposition the better pattern?

A. When the same four steps are always performed in order
B. When subtasks depend upon what's discovered along the way
C. When latency needs to be strictly bounded for each request
D. When each subtask can be fully specified ahead of time

Q123. Multi-Agent Research · Easy. Do subagents share memory between invocations by default?

A. Yes, as long as they have the same `AgentDefinition`
B. Only when they're run in the same parallel batch
C. No: each invocation needs its context in the prompt
D. Yes: the last run's context is restored automatically

Q124. Developer Productivity · Medium. A coding assistant's documentation subagent needs facts found by its analysis subagent. An engineer proposes letting the documentation agent read the analysis agent's history. What's better?

A. The coordinator passes the findings in its prompt
B. Set a configuration flag so that the two subagents share a memory space
C. Have the analysis agent write its entire transcript into the system prompt
D. Merge the two subagents' conversation histories into one on every turn

Q125. Multi-Agent Research · Medium. A synthesis subagent writes reports with no citations, though the search agent had found good sources. What's the most likely cause?

A. Subagents can only cite sources they've found themselves, using their own tools
B. The synthesis agent's system prompt is too long for it to process citations well
C. The findings and their sources weren't included in the synthesis agent's prompt
D. The search agent's memory expired before the synthesis agent was able to read it

Q126. Multi-Agent Research · Medium. Why route all subagent communication through the coordinator?

A. It removes the need for subagents to have their own tools or system prompts
B. It reduces total token use, since subagents no longer need to receive any prompts
C. It gives observability, consistent error handling, and controlled information flow
D. It's the only way that the API allows two agents to exchange data with each other

Q127. Multi-Agent Research · Easy. What does `stop_reason: "end_turn"` tell the coordinator's loop?

A. The conversation history has grown too large to continue with
B. The model has hit its token limit and needs a follow-up request
C. A tool failed, and it should be retried before continuing
D. The model has finished its turn, so the loop can terminate

Q128. Multi-Agent Research · Medium. For the query "What's the capital of Australia?", your research system still runs search, analysis, synthesis and report agents. What should change?

A. The search subagent should be given a shorter timeout for simple factual queries
B. Every query should pass through the full pipeline, to guarantee consistent quality
C. The coordinator should analyze the query and invoke only the subagents it needs
D. The report agent should detect simple questions, and shorten its output format

Q129. Customer Support · Hard. A hook blocks a $700 refund. How should it best communicate this?

A. Reduce the refund to $500 automatically, and then process it anyway
B. Silently drop the call, and return a success message to the agent
C. Return the reason, and direct the agent to the escalation path
D. Crash the session, so that a person can notice and investigate it

Q130. Multi-Agent Research · Medium. Which responsibility belongs to the coordinator?

A. Rendering the final PDF output and managing its storage
B. Decomposing the task, delegating, and aggregating the results
C. Performing every web search to avoid spawning any subagents
D. Sharing its memory automatically with each spawned subagent

Q131. Multi-Agent Research · Medium. You spawned parallel subagents across three separate coordinator turns. What's the effect?

A. They shared one context across the turns
B. They failed, as turns can't contain Task calls
C. They ran sequentially, adding latency
D. They ran in parallel, as it's the same

Q132. Customer Support · Medium. Which statement about hooks and the model is correct?

A. The model can disable hooks when it explains why a particular case is an exception

B. Hooks run inside the model, so their effect depends upon the wording of the prompt

C. Hooks are only suggestions, which the model is free to weigh against its instructions

D. The model can't talk its way past a blocking hook, as it runs outside the model

Q133. Customer Support · Medium. A case is escalated mid-process to a human agent who can't see the transcript. What should the handoff contain?

A. A short note saying the customer needs help, with their name only

B. The model's confidence score, plus the number of turns so far

C. The full raw transcript, so the human can find whatever they need

D. Customer ID, root cause, refund amount, and the recommended action

Q134. Code Generation · Medium. You resume yesterday's refactoring session, and you've since changed three files. What should you do?

A. Say nothing, as Claude automatically detects all changes on resume

B. Delete the session, since any change makes prior context unusable

C. Tell the session which files changed, so it can re-analyze them

D. Ask it to re-read the entire repository before doing anything else

Q135. Developer Productivity · Easy. What's `fork_session` for?

A. Independent branches from a shared analysis baseline

B. Merging two old sessions into a single new session

C. Copying a session onto another developer's machine

D. Reducing the context size of a long-running session

Domain 2: Tool Design and MCP Integration

Q136. Customer Support · Easy. Which of these is a retryable error?

A. A refund above the policy limit
B. A user lacking access to a record
C. A brief service timeout
D. A customer ID in the wrong format

Q137. Data Extraction · Medium. Which name and description pair is best for a tool that only reads PDF invoices?

A. `extract`: "Extracts everything from any file. Works with whatever the user has provided."
B. `doc_tool`: "A document tool. See the internal wiki for details on how it should be used."
C. `process`: "Processes the input data and returns the appropriate result as needed by the task."
D. `extract_invoice_fields`: "Vendor, date and totals from one invoice PDF. Not for receipts."

Q138. Multi-Agent Research · Easy. Which change directly reduces functional overlap between two tools?

A. Increasing the model's iteration limit for ambiguous tasks
B. Renaming and rescoping them so that each has one clear job
C. Adding a longer system prompt about how good choices matter
D. Listing the two tools further apart in the tool array

Q139. Developer Productivity · Medium. How do you give a shared `.mcp.json` access to a GitHub token without committing it?

A. Reference `${GITHUB_TOKEN}`, and set it in each developer's environment
B. Commit the token in a private repository, since access there is limited
C. Base64-encode the token in the file, so that it isn't readable directly
D. Put the token in CLAUDE.md, with a note that it shouldn't be shared

Q140. Developer Productivity · Hard. Select the two things to review when good tool descriptions are being ignored. **Select 2.**

A. System prompt wording that names a tool in response to keywords
B. The time of day at which the tool's backend was deployed
C. The API version's support for system prompts in general
D. The number of characters in the customer's latest message
E. Other tool descriptions overlapping with this one's stated purpose

Q141. Multi-Agent Research · Medium. Agents keep confusing `analyze_content` and `analyze_document`, whose descriptions are nearly identical. What's the best first fix?

A. Remove one of the tools, and make the other one handle every type of analysis
B. Rename them for their purpose and rewrite the descriptions to state clear boundaries
C. Tell the model in the system prompt to think harder before it chooses a tool
D. Add a routing classifier that picks the right tool before the model ever sees them

Q142. Developer Productivity · Medium. An agent has three MCP servers configured. When are their tools available?

A. Tools load one by one, when the model first asks about each server
B. Only one server's tools are available at a time, chosen by the user
C. All are discovered at connection time, and available simultaneously
D. Each server needs its own separate agent session to expose its tools

Q143. Multi-Agent Research · Hard. A search agent sometimes needs to load a known document, which is the analysis agent's tool. Calls for this are frequent. What's best?

A. Give it all the analysis tools, because the need occasionally turns up
B. Never allow it, and always round-trip through the coordinator, however often
C. Give it a narrow `load_document` tool, and no analysis tools
D. Merge the search and analysis agents into one agent with every tool

Q144. Customer Support · Easy. Which is an example of a business error?

A. A refund exceeding the permitted policy limit
B. A database connection that has timed out
C. A malformed email address sent as input
D. An expired token presented by a service

Q145. Developer Productivity · Hard. An agent runs `grep` through Bash for everything, and it floods the context with long outputs. What's a reasonable improvement?

A. Use the Grep tool with narrow patterns or paths, so that output is focused
B. Use Write to save the output, then read back the entire saved file
C. Switch to Glob for content searches, as it returns fewer results
D. Keep using Bash, but ask the model to read the output more quickly

Q146. Developer Productivity · Medium. When is a custom MCP server the better choice?

A. Whenever a community server has a slightly different tool name
B. Whenever a developer would like to learn how MCP servers work
C. For a team-specific workflow that no existing server covers
D. For any standard integration, for the sake of full control

Q147. Multi-Agent Research · Medium. A synthesis agent with access to web search keeps searching rather than synthesizing. What's the best fix?

A. Keep the tool, and add "avoid searching" to its system prompt in capitals
B. Give it more tools, so that it no longer focuses only on using search
C. Force the synthesis tool with `tool_choice` on all of its turns in a run
D. Remove search from its toolset, and restrict it to what its role needs

Q148. Multi-Agent Research · Hard. Select the two anti-patterns in tool error handling. **Select 2.**

A. Retrying a transient failure locally, within a limit

B. Returning empty results as success when a backend failed

C. Using one generic failure message for all failures

D. Marking policy violations as not being retryable

E. Returning `isError` with a clear, specific description

Q149. Developer Productivity · Hard. You add a fourth MCP server with 30 tools, and tool selection gets worse across the board. Why?

A. MCP has a hard limit of 75 tools across all connected servers

B. Each server reduces the other servers' descriptions to a single line

C. Only the most recently added server's tools are usable by the agent

D. All tools are available at once, so choosing gets harder

Q150. Customer Support · Hard. A system prompt says "use the billing tool for any money-related question." The agent calls it for refund status, which belongs to `lookup_order`. Which fix addresses the root cause?

A. Add a post-processing step that reroutes calls after the agent has made them

B. Add "refund" to the billing tool's description, so that it can answer those as well

C. Hide `lookup_order` from the agent whenever a message mentions any money

D. Rewrite the prompt instruction, so that it no longer ties all money words to one tool

Q151. Data Extraction · Hard. You need to extract from invoices, receipts or contracts, with one schema per type, and the type isn't known. Which approach fits?

A. Define one extraction tool per type, and set `tool_choice: "any"`

B. Define one tool and `tool_choice: "auto"`, then parse any text that comes back

C. Force the invoice tool on all documents, as invoices are the most common type

D. Ask the model first in text which type it is, then parse the word it replied with

Q152. Customer Support · Medium. A user tries to access another customer's invoice. What's the right structured error?

A. Permission error, not retryable, with a safe explanation
B. Validation error, asking them to correct their user ID
C. Transient error, retryable, in case access is restored
D. Empty success, so as to avoid revealing that it exists

Q153. Developer Productivity · Easy. Where do you configure an experimental MCP server that's just for you?

A. `~/.claude.json`
B. `.claude/rules/mcp.md` in the repository
C. A note in the team's CLAUDE.md
D. `.mcp.json` in the shared repository

Q154. Customer Support · Hard. Select the two changes that address unreliable selection between `get_customer` and `lookup_order`. **Select 2.**

A. Reduce both of their descriptions to one short phrase each
B. State in each description when to use the other tool instead
C. Add input formats and example queries to each description
D. Rename both of them with a shared prefix like `get_data_`
E. Add a classifier in front of the model as the first step

Q155. Multi-Agent Research · Hard. A generic `analyze_document` tool produces mixed results across tasks. What's the best redesign?

A. Add a `mode` string parameter that accepts any instruction, and keep the single tool
B. Wrap the tool in a subagent, which then decides how the document should be analyzed
C. Keep the tool as it is, and add several hundred words of general usage notes to it
D. Split it into `extract_data_points`, `summarize_content` and `verify_claim_against_source`

Q156. Developer Productivity · Medium. Two MCP servers both expose a `search` capability, and the agent picks the wrong one. Both are correctly configured. What's the best first fix?

A. Add a CLAUDE.md note saying that the second of the two servers is the better one
B. Move one of the servers to user scope, which gives it a lower selection priority
C. Disconnect one of the servers, since an agent can use only one server at a time
D. Make each tool's description state what it searches, and when to prefer the other

Q157. Customer Support · Hard. Select the two fields that help an agent choose a recovery action. **Select 2.**

A. A timestamp of the most recent deployment
B. The name of the server's hosting region
C. An error category, such as transient or business
D. The size of the tool's source code in lines
E. A flag showing whether a retry could succeed

Q158. Code Generation · Medium. You're trying an unproven MCP server. Why not put it in `.mcp.json` straight away?

A. It would remove all other servers from the team's config
B. It would then affect everyone who pulls the repository
C. `.mcp.json` only supports servers published by Anthropic
D. User-level files are the only ones that can run new servers

Q159. Data Extraction · Medium. `extract_metadata` must run before any enrichment tool. How do you ensure that?

A. Set `tool_choice: "any"`, and list it first in the array of available tools
B. Force it on the first turn with `{"type": "tool", ...}`, then continue
C. Add to its description that it should preferably be called before anything else
D. Run all of the enrichment tools first, and call the metadata tool only if needed

Q160. Developer Productivity · Medium. Which option is a form of scoped tool access?

A. A single agent that has all the tools from all connected servers
B. A coordinator that gives each subagent all the tools it has itself
C. A subagent that has every tool, plus a prompt telling it to be careful
D. A read-only exploration agent with only Read, Grep and Glob

Q161. Multi-Agent Research · Hard. A search subagent got results from two of three backends before the third failed. What's the best thing to return?

A. A request for the coordinator to rerun the whole search from the very beginning
B. A failure with no results, as partial data could mislead the synthesis agent
C. The partial results, plus structured details of the failed backend and what was tried
D. The partial results as a complete success, with no mention of the third backend

Q162. Developer Productivity · Easy. Which is a good example of an MCP resource?

A. A function that issues a refund to a customer
B. A credential that's used to log in to a service
C. A shell command that deletes a temporary directory
D. A hierarchy of documentation pages, or database schemas

Q163. Data Extraction · Medium. What does `tool_choice: "auto"` allow?

A. The model may call a tool, or respond with text
B. The API chooses the tool that matches the schema
C. The model must call one specific, named tool
D. The model must call a tool, but it chooses which

Q164. Developer Productivity · Hard. Select the two correct scope assignments. **Select 2.**

A. Shared token value: committed in `.mcp.json`

B. Personal experiment: `~/.claude.json`

C. Personal experiment: `.mcp.json`

D. Shared GitHub server: `~/.claude.json`

E. Shared GitHub server: `.mcp.json`

Q165. Multi-Agent Research · Medium. Why are generic tools riskier than constrained alternatives in multi-agent systems?

A. They let agents act outside their role, as nothing limits what they can do

B. They can't be described in the JSON schemas that the API requires

C. They prevent parallel Task calls from running in a single response

D. They always cost more per call than tools with narrow scope do

Q166. Multi-Agent Research · Hard. Select the two changes that reduce cross-specialization misuse. **Select 2.**

A. Restricting each subagent's tools to its role

B. Letting subagents borrow tools from each other when needed

C. Replacing broad tools with validated, narrow ones

D. Giving each subagent every tool, with more instructions

E. Raising each subagent's maximum number of iterations

Q167. Developer Productivity · Easy. Where do you configure an MCP server that the whole team should share?

A. `CLAUDE.md` at the root of the repository

B. `.mcp.json` in the project

C. A personal `.env` file in each checkout

D. `~/.claude.json` in each home directory

Q168. Developer Productivity · Hard. An agent must find where an error message "quota exceeded" comes from. Which sequence is most efficient?

A. Glob all the files, Read each of them in turn, and then Write a summary
B. Read the main file, then guess which modules are likely to be relevant
C. Bash a loop that prints every file, then ask the model to review the output
D. Grep for the message, Read the matching file, then Grep for its callers

Q169. Developer Productivity · Easy. Which built-in tool finds every call to `processRefund` across a codebase?

A. Glob
B. Grep
C. Write
D. Edit

Q170. Customer Support · Medium. Two tools both accept "an identifier," and the model passes order numbers to `get_customer`. What helps most?

A. Adding a retry, so that a wrong ID eventually finds the right record
B. Removing the parameter descriptions, in order to shorten what the model reads
C. Stating each tool's exact identifier format, with examples, in each description
D. Giving both tools the same parameter name, for reasons of consistency

Q171. Customer Support · Hard. Select the two settings or practices that guarantee some tool gets called (per the guide). **Select 2.**

A. `tool_choice: "any"`
B. A very clear description on the most important tool
C. `tool_choice: {"type": "tool", "name": "get_customer"}`
D. A prompt saying "you must always call a tool"
E. `tool_choice: "auto"`

Q172. Customer Support · Medium. A refund is refused because it's outside the 30-day window. What should the tool return?

A. A business error, not retryable, with a shareable explanation

B. A transient error that's retryable, in case the policy happens to change soon

C. Success with a zero refund amount, so that the customer doesn't see a failure

D. A validation error asking the agent to change the order date and try it again

Q173. Code Generation · Easy. What does Write do to an existing file?

A. Searches it for text

B. Appends to the end

C. Edits a unique match

D. Replaces its contents

Q174. Code Generation · Medium. Edit fails because the text to replace appears in several places. What's the guide's fallback?

A. Retry the same Edit until one of them happens to match

B. Use Grep to replace it across the repository in one step

C. Use Glob to pick which of the occurrences is to be changed

D. Read the full file, then Write the modified content

Q175. Customer Support · Easy. How does an MCP tool signal a failure to the agent?

A. With the `isError` flag on its result

B. By omitting its result from the response

C. By returning an HTTP 200 with empty text

D. By raising `stop_reason: "tool_error"`

Q176. Multi-Agent Research · Hard. After you improve descriptions, selection is correct in most cases. An ambiguous class of queries still routes wrongly. What's a good next step?

A. Rename every tool again, so that they all start with their domain
B. Add a few examples of those cases, with the reasoning
C. Remove all the descriptions, and rely on a set of examples only
D. Add a separate model call to double-check every tool selection

Q177. Developer Productivity · Medium. A function is re-exported under different names by wrapper modules. How do you find all of its usage?

A. Use Glob with the original name, since aliases are visible in the file paths
B. Grep only for the original name, as aliases work the same way at runtime
C. Identify all exported names first, then Grep for each across the codebase
D. Read the wrapper modules alone, as those files must contain all the callers

Q178. Developer Productivity · Easy. Which built-in tool finds all files named `**/*.test.tsx`?

A. Edit
B. Grep
C. Read
D. Glob

Q179. Customer Support · Medium. What should a good tool description include?

A. The tool's internal database schema, plus the service's deployment history
B. A reminder that the model should be careful and accurate when using it
C. Input formats, examples, edge cases, and when to use it versus similar tools
D. A list of the engineers who maintain it, along with their contact details

Q180. Developer Productivity · Medium. Your MCP tool got a valid request for a file that doesn't exist. How do you distinguish this from the file server being down?

A. Report both as permission errors, since neither of them returned any file content
B. Report "not found" as a clear result, and report the outage as a transient error
C. Return an empty string for both of them, and let the agent work out which it is
D. Treat both as the same, as the agent can't do anything different in either case

Q181. Multi-Agent Research · Hard. Which subagent behavior fits the guide's error recovery design?

A. Propagate each failure to the coordinator at once, so that it can make all decisions
B. Terminate the whole workflow on any failure, to protect the quality of the report
C. Swallow all failures, so that the coordinator never has to deal with any of them
D. Retry transient failures locally, and propagate only what can't be resolved

Q182. Multi-Agent Research · Medium. A search returns zero results for a valid query. What should the tool report?

A. A successful result with no matches, and not an error
B. A generic failure, as nothing useful came back from it
C. `isError: true`, with a suggestion to escalate the query
D. `isError: true`, category transient, so that it gets retried

Q183. Developer Productivity · Hard. A tool throws a raw exception containing "password rejected for orders-db at 10.2.3.4", and the framework passes that text to the model. What's the concern?

A. The model will try to fix the database password by itself on the server
B. Internal details may leak into model context and customer-facing replies
C. Exceptions make `stop_reason` become `end_turn`, which ends the run early
D. Long exception text will cause the model to stop calling any more tools

Q184. Developer Productivity · Hard. A `.mcp.json` uses `${API_TOKEN}`, and one developer's server fails to authenticate. What should they check first?

A. Whether `.mcp.json` should be moved to `~/.claude/commands`
B. Whether the variable should be added to CLAUDE.md instead
C. Whether `API_TOKEN` is set in their own environment
D. Whether the token has been committed into the repository

Q185. Customer Support · Medium. A subagent has tools for reading orders, and it needs to issue refunds only through the coordinator. How do you enforce this?

A. List the refund tool last, so it's less likely to be chosen
B. Leave the refund tool out of that subagent's definition
C. Give the refund tool a longer name, so that it's harder to use
D. Tell it in the prompt that it isn't allowed to refund

Q186. Multi-Agent Research · Medium. An agent keeps retrying a tool with the same invalid date format. What would stop this?

A. A generic failure message, which encourages the agent to try something different
B. A forced tool choice, which makes the agent call the same tool one more time
C. A validation error stating the format expected and that it isn't retryable unchanged
D. A bigger iteration cap, so that the agent eventually finds the correct format

Q187. Multi-Agent Research · Easy. What's the guide's rule of thumb for an agent's tools?

A. About 4-5 tools relevant to its role
B. All of the tools in the organization's catalogue
C. Exactly one tool, in order to avoid confusion
D. At least 15 tools for full flexibility

Q188. Customer Support · Hard. Why include a customer-friendly explanation in a business error?

A. It lets the agent retry with a friendlier request, which might then succeed

B. The agent can communicate the reason accurately, rather than inventing one

C. The API displays that field to the customer with no further agent involvement

D. It's needed for the model to accept the tool result as being valid

Q189. Customer Support · Medium. What does `isRetryable: false` prevent?

A. The tool from being shown in the next request's tool list

B. Wasted retry attempts on errors that can't succeed

C. The customer from ever contacting the support team again

D. The agent from calling any other tools during this turn

Q190. Developer Productivity · Hard. Which MCP description best stops an agent preferring Grep for ticket lookups?

A. "Searches tickets. Uses our internal indexing cluster, which was upgraded during last June."

B. "A ticket tool. It's really good, and it should generally be preferred over the others."

C. "Full-text search of tickets and comments; use instead of searching exported files."

D. "This tool is MCP-based, so it's considered more advanced than built-in file tools."

Q191. Code Generation · Medium. Which tool suits changing one unique line in a file?

A. Grep

B. Edit

C. Glob

D. Write

Q192. Customer Support · Easy. A tool's description should tell the model...

A. which engineer wrote it, and how it was deployed

B. that it's the best tool available for most tasks

C. how many times it was called during the last month

D. when to use it, and when to use a similar tool instead

Q193. Developer Productivity · Hard. Which is the strongest reason to split a generic tool, rather than expand its description?

A. Tools with narrow names are executed by the API at higher priority
B. It has several distinct purposes, each needing its own inputs and output contract
C. Its description has reached the maximum number of words that a tool can have
D. More tools always improve the reliability of selection, no matter how many

Q194. Multi-Agent Research · Easy. An agent has 18 tools, and picks wrongly more often than an agent with 5. Why?

A. Large tool lists disable the `tool_use` stop reason for the whole session
B. Agents can only read the first five tool descriptions in any request
C. More tools increase decision complexity, which degrades selection reliability
D. The API randomly samples tools when more than ten have been registered

Q195. Code Generation · Hard. Select the two steps in the fallback when Edit can't find a unique anchor. **Select 2.**

A. Glob the directory, to refresh the file's anchor
B. Grep the file repeatedly until the match is unique
C. Read the file's full contents
D. Write back the modified contents
E. Delete the file and start it again from scratch

Q196. Customer Support · Medium. A tool returns "Operation failed" for all failures. Why is this a problem?

A. It forces the agent to escalate every failure to a human representative
B. The agent can't tell whether to retry, fix its input, or tell the customer
C. It causes the loop to terminate, because the agent reads it as `end_turn`
D. The string is too short for the API to accept it as a valid tool result

Q197. Developer Productivity · Medium. An agent wastes calls exploring what documentation exists. What MCP feature reduces this?

A. Lengthening the descriptions of all unrelated tools
B. Setting `tool_choice: "none"` during exploration
C. Exposing a content catalog as an MCP resource
D. Adding a prompt that says to explore less often

Q198. Multi-Agent Research · Medium. Which description helps most with edge cases for a `web_search` tool?

A. "Searches the web quickly, and returns the very best possible results for all queries."
B. "Returns up to 10 results; an empty list means no matches. For a known URL, use load_document."
C. "Web search. If it doesn't work, then just try it again until it finally does."
D. "Searches. Always use this first before you try anything else that's available to you."

Q199. Customer Support · Hard. Your system prompt says: "Always check the customer's account first when they mention an order." The agent now calls `get_customer` for every order question, despite good descriptions. What's the likely cause?

A. The agent has too few tools, so it falls back to using the most general one
B. The tool descriptions are too long, and the model has started ignoring them
C. Keyword-sensitive prompt wording is overriding the tool descriptions
D. `get_customer` is listed first, and the model prefers whatever comes first

Q200. Customer Support · Medium. A support agent's billing subagent also has shipping and HR tools "just in case." Which risk is greatest?

A. It may misuse those tools, and selection among all of them gets less reliable
B. Other subagents will lose access to those tools while this one holds them
C. The model will refuse to run because of the conflicting tool schemas it has
D. The API charges an extra fee for each tool that's defined and not called

Q201. Customer Support · Medium. `lookup_order` receives an order ID containing letters, and the format must be digits. Which error category fits?

A. Business
B. Permission
C. Validation
D. Transient

Q202. Code Generation · Hard. A new developer's agent can't see the team's Linear server, while others can. What's the most likely cause?

A. The server was configured in someone's `~/.claude.json` and not in `.mcp.json`
B. `.mcp.json` only applies to developers who created it on their machine
C. The new developer's model doesn't support more than one MCP server
D. MCP servers only become available after a CLAUDE.md `@import` has run

Q203. Multi-Agent Research · Medium. A search tool's backend is unreachable, and the tool returns an empty list as success. What happens?

A. The API converts the empty result into an error before the agent sees it
B. The agent concludes that nothing exists, when in truth the search never ran
C. The agent automatically detects the outage, as the list has no metadata
D. The agent retries anyway, as empty lists always trigger a further call

Q204. Multi-Agent Research · Hard. A drafting agent often needs to confirm one statistic, and most checks are simple lookups. Sending each through the coordinator is slowing it down. What's the best design?

A. Make it batch the checks, and send them all to the coordinator at the end of a run
B. Give it the full web search tool, so that it can answer all of its own questions
C. Keep all of the checks routed through the coordinator, since hub-and-spoke is absolute
D. Give it a scoped `verify_fact` tool, with complex cases via the coordinator

Q205. Developer Productivity · Hard. Select the two correct tool choices. **Select 2.**

A. Finding where a config key is used: Glob

B. Finding all `Dockerfile` paths: Glob

C. Finding where a config key is used: Grep

D. Replacing a unique line: Grep

E. Finding all `Dockerfile` paths: Edit

Q206. Code Generation · Medium. Edit keeps failing with "no unique match" on a config value that repeats. Besides Read + Write, what would help the Edit succeed?

A. Run Glob first, so that Edit knows which file it's looking at

B. Include more surrounding text, so that the target string is unique

C. Use a shorter target, so that it matches more easily somewhere

D. Run the Edit twice in quick succession on the same string

Q207. Data Extraction · Medium. You must guarantee that the model calls a tool instead of replying in text, though it may choose among several. Which setting fits?

A. `tool_choice: "any"`

B. `tool_choice: "none"`

C. `tool_choice: "auto"`

D. `tool_choice: {"type": "tool", "name": "x"}`

Q208. Customer Support · Easy. Minimal tool descriptions tend to cause which problem?

A. Automatic retries on every tool call that's made

B. Unreliable selection among similar tools

C. Faster responses with lower accuracy on schemas

D. Errors from the API about description length

Q209. Developer Productivity · Medium. Why state what a tool returns in its description?

A. So the agent can judge whether calling it helps with the current need
B. Because tools with no output described are rejected when they're registered
C. So that the model can skip calling the tool and predict its output instead
D. Because the API validates the output against the description at runtime

Q210. Customer Support · Hard. What should you measure to show that a description change has worked?

A. Whether the model describes the new tool favorably when you ask it
B. The number of developers who have approved the pull request for it
C. The word count of the new description, compared with the old one
D. Selection accuracy on a labelled set of realistic and ambiguous requests

Q211. Customer Support · Hard. How do the protocol and your own design divide this error metadata?

A. `isError` is in MCP, and the other two are fields you design
B. `isRetryable` is the only standard field, as the others are SDK-specific extensions
C. None are in MCP, so the flag must be communicated in free text by convention
D. All three are standard MCP fields, and clients validate them on every result

Q212. Customer Support · Medium. Which `lookup_order` description best supports reliable selection?

A. "Looks up things. Use it whenever you need to look something up for someone."
B. "A powerful, flexible, fast tool that handles anything to do with orders."
C. "Retrieves order information from the orders system of the company."
D. "Retrieves one order by ID (digits, e.g. 7781). For customer details, use get_customer."

Q213. Multi-Agent Research · Medium. A generic `fetch_url` tool lets agents load anything. How do you constrain it?

A. Keep it, and add a note saying only documents should ever be fetched

B. Replace it with `load_document`, which validates that URLs are document URLs

C. Let it load any URL, then ask the model to discard what it didn't need

D. Add a second `fetch_url` tool, which is meant for the non-document pages

Q214. Multi-Agent Research · Hard. When a subagent can't recover from an error, what should it send to the coordinator?

A. Only its partial results, leaving out any mention that a failure took place

B. A generic note that the search was unavailable, in order to keep messages short

C. The failure type, what was attempted, partial results, and possible alternatives

D. A full stack trace from the backend, including its database connection details

Q215. Customer Support · Medium. The orders database times out. Which structured response is best?

A. `isError: false`, with an empty result list, so that the workflow can continue

B. `isError: true`, category validation, `isRetryable: false`, and a stack trace

C. `isError: true`, category transient, `isRetryable: true`, and a short description

D. `isError: true`, with "Error" as the only content in the tool's response

Q216. Developer Productivity · Medium. A reviewer asks for all Kotlin test files in the repo. What's the right tool and pattern?

A. Glob with `**/*Test.kt`

B. Read with the directory path

C. Edit with `*Test.kt` as its target

D. Grep with the word "Kotlin"

Q217. Code Generation · Hard. Select the two practices for standard third-party integrations. **Select 2.**

A. Commit its token, so that onboarding is easier for everyone

B. Write a custom server first, then compare it to others

C. Scrape the product's web pages, rather than use its API

D. Prefer an existing community server when it meets the need

E. Keep its authentication token in an environment variable

Q218. Developer Productivity · Medium. A resource exposes issue summaries. What's the main benefit for the agent?

A. It no longer needs any tools at all during the session

B. It can skip reading tool descriptions for the current session

C. Its context window automatically grows to hold all the issues

D. It can see what's available, then fetch only the relevant details

Q219. Code Generation · Medium. Your team needs Jira access in Claude Code. What should you do first?

A. Start building a custom MCP server for Jira immediately

B. Copy Jira tickets into CLAUDE.md, and update them each week

C. Scrape the Jira web interface using a Bash script instead

D. Check for an existing community MCP server for Jira

Q220. Developer Productivity · Medium. Why is reading all files upfront a poor exploration strategy?

A. The Read tool is limited to one file per session

B. It prevents Grep from working later in that session

C. It fills the context with mostly irrelevant content

D. Files must be read only in alphabetical order

Q221. Multi-Agent Research · Hard. After a tool is forced on the first turn, what should happen on later turns?

A. The model must be restarted, since forced tools end the conversation
B. The same forced setting must stay, or the conversation becomes invalid
C. Later turns must use `tool_choice: "none"` so that the output is text only
D. Later turns can relax it, so the model chooses its next steps

Q222. Developer Productivity · Medium. The agent keeps using Grep on exported data, instead of your richer MCP search tool. What's the best first step?

A. Rename the MCP tool to `grep`, so that it replaces the built-in tool
B. Add a line to CLAUDE.md saying that MCP tools are generally better
C. Improve the MCP tool's description to explain its capabilities and outputs
D. Remove Grep from the system entirely, for all tasks in all projects

Q223. Developer Productivity · Medium. You need to understand an unfamiliar service. What's the best way to begin?

A. Grep for entry points, then Read and follow imports to trace the flows
B. Write a summary from your general knowledge of services like this
C. Read every source file first, so that you have complete understanding
D. Use Glob to list all files, and treat the names as the understanding

Q224. Developer Productivity · Medium. A team's `search` tool description reads "searches stuff." Agents often use it for tasks that the `grep_logs` tool does better. What's the proportionate fix?

A. Rewrite both descriptions to state what each searches, and when to prefer the other
B. Force `grep_logs` with `tool_choice` for all the requests that mention the word log
C. Delete `grep_logs`, since having two search tools will always cause confusion
D. Add few-shot examples for all possible searches to the system prompt immediately

Q225. Customer Support · Easy. What's the primary mechanism Claude uses to choose among available tools?

A. The order they're listed in

B. The length of their names

C. The tool descriptions

D. The most recently used tool

Domain 3: Claude Code Configuration and Workflows

Q226. Code Generation · Medium. You've found three bugs in a function, and fixing one changes how the others behave. How should you report them?

A. One at a time, with each in a new session
B. In three separate messages, with no connection between them
C. All together in one detailed message, as the fixes interact
D. Only the most serious, and hope the rest resolve

Q227. Code Generation · Easy. Which file holds instructions that apply to you alone, across all of your projects?

A. `CLAUDE.md` at the repository root
B. `~/.claude/CLAUDE.md`
C. A subdirectory's `CLAUDE.md`
D. `.claude/CLAUDE.md` in the project

Q228. Code Generation · Hard. A brainstorming skill generates many rejected ideas, and later work seems influenced by them. What should you change?

A. Set `argument-hint` to "final only", so that rejected ideas are never produced
B. Move the skill's whole text into CLAUDE.md, where it's loaded in all sessions
C. Add a closing instruction, asking the model to forget all the rejected ideas
D. Add `context: fork`, so the exploration doesn't remain in the main conversation

Q229. Code Generation · Medium. Your team uses plan mode for every one-line fix, and work has slowed down. What's the best advice?

A. Reserve plan mode for complex work, and execute clear small changes directly
B. Use the Explore subagent on all one-line fixes, as a lighter alternative for planning
C. Keep using it, because more planning always leads to better results for any task
D. Stop using plan mode altogether, since it's clearly slowing down all the team's work

Q230. Code Generation · Medium. You describe a log format in a long paragraph, and output varies across runs. What should you do?

A. Repeat the paragraph three times, for greater emphasis
B. Make the paragraph longer, to remove any possible ambiguity
C. Ask Claude to infer the format from similar projects
D. Show two or three sample inputs, with the exact expected outputs

Q231. CI/CD · Easy. What does `-p` stand for in the guide?

A. `--parallel`
B. `--print`
C. `--plan`
D. `--pipeline`

Q232. CI/CD · Medium. The pipeline must pass the PR diff to Claude. Which is correct in principle?

A. Add the diff to CLAUDE.md, and commit it to the repository before each run
B. Let Claude find it itself through memory from the previous job's session
C. Provide it as part of the `-p` prompt or piped context, in the job script
D. Use an interactive session, and type it in through a remote terminal

Q233. Code Generation · Medium. Where do topic-specific rule files live?

A. `.claude/skills/`
B. `.claude/commands/`
C. `.claude/rules/`
D. `.claude/hooks/`

Q234. Code Generation · Hard. You planned a library migration, and the approach is agreed. What's the sensible way to proceed?

A. Execute the planned approach directly, using the plan as the guide
B. Discard the plan and start the investigation again, using direct execution
C. Stay in plan mode permanently, as code must never be changed in that session
D. Ask for a further plan of the plan, to raise your confidence in it

Q235. Code Generation · Medium. Which belongs in a skill rather than in CLAUDE.md?

A. The rule that tests live beside the code they cover
B. The commit message format that all developers must use
C. The project's naming convention, for all source files
D. A release-notes workflow that's used before each release

Q236. Code Generation · Hard. You write a one-page checklist that's needed once a week. Why is a skill better than CLAUDE.md?

A. It loads only when invoked, saving context
B. Skills are always more strictly obeyed than CLAUDE.md instructions
C. A skill's content can't be read by other developers on the team
D. CLAUDE.md files can't contain any checklists, only prose rules

Q237. Code Generation · Easy. How do you make a rules file load only for Terraform files?

A. A first line in the file saying "only for Terraform"
B. YAML frontmatter with `paths: ["terraform/**/*"]`
C. A CLI flag on each run: `--rules terraform`
D. A filename such as `terraform-only.md`, with no frontmatter

Q238. Code Generation · Easy. Where do you put a command for your personal use across projects?

A. `.claude/rules/`
B. `.claude/commands/`
C. `.mcp.json`
D. `~/.claude/commands/`

Q239. Code Generation · Hard. You want to confirm that a colleague's session loads `.claude/rules/testing.md`. What's the direct method?

A. Inspect the Git history of the rules directory
B. Have them run `/memory` in that session
C. Have them run `--output-format json` on it
D. Ask Claude whether it generally knows about testing

Q240. Code Generation · Hard. During a large refactor, main context is nearly exhausted after exploration, and implementation hasn't started. What would've prevented this?

A. Running discovery in the Explore subagent, keeping only a summary
B. Reading each file twice, so that key facts are reinforced before implementing
C. Writing a longer system prompt, which helps the model to remember its findings
D. Using direct execution from the start, so that exploration isn't needed at all

Q241. CI/CD · Medium. Generated tests duplicate what's already covered. What helps most?

A. Ask for many more tests, then remove duplicates afterward
B. Generate them with `--output-format json`
C. Run generation on a nightly schedule instead
D. Provide existing test files in the context

Q242. CI/CD · Medium. Test generation produces many low-value tests. What's the most useful change?

A. Switch to the batch API, as batched generation produces better test quality
B. Document testing standards, valuable test criteria and available fixtures in CLAUDE.md
C. Lower the number of tests requested, and hope the best ones come out first
D. Ask for tests with a higher self-reported confidence, and discard all the others

Q243. Developer Productivity · Medium. A path rule mentions fixtures, and context seems smaller in sessions on unrelated files. Why?

A. The rule isn't loaded unless matching files are being worked on
B. Claude deletes unrelated rule files from the repository
C. Sessions automatically drop all rules after their first turn
D. Fixture rules are compressed by Claude Code before loading

Q244. Code Generation · Medium. What's the interview pattern's main value?

A. It lets Claude avoid implementing the feature at all
B. It makes each model response shorter and cheaper
C. It replaces the need for tests in the project
D. It surfaces considerations you hadn't anticipated

Q245. Code Generation · Easy. What file defines a skill?

A. `SKILL.md`
B. `package.json`
C. `CLAUDE.md`
D. `.mcp.json`

Q246. CI/CD · Easy. Which flag runs Claude Code non-interactively in a pipeline?

A. `--ci`
B. `--headless`
C. `-p`
D. `--batch`

Q247. CI/CD · Medium. A nightly job calling `claude "summarize today's changes"` keeps hitting the runner's timeout, as it's waiting at a prompt. What's the fix?

A. Redirect `/dev/null` into a normal interactive session
B. Add a CLAUDE.md line saying never to ask for input
C. Run with `-p`, passing the prompt on the command line
D. Set `CLAUDE_HEADLESS=true` in the job's environment

Q248. Code Generation · Medium. "Add a date validation conditional to `parseOrder()`." Which approach fits?

A. Plan mode with a design review
B. Forking two sessions for comparison
C. Direct execution
D. An Explore subagent, then a new plan

Q249. Code Generation · Medium. What's the main benefit of path-scoped rules?

A. They run faster, since they're compiled when each session has started
B. They load only when relevant, reducing irrelevant context and token use
C. They guarantee compliance, as Claude can't ignore a path-scoped rule
D. They override the user's CLAUDE.md in all of the repository's sessions

Q250. Code Generation · Medium. Test files live beside source files across the codebase, and all must follow one convention. What's best?

A. A `CLAUDE.md` file in each directory with tests
B. A root CLAUDE.md section, loaded for everything
C. A skill that developers must run before writing tests
D. A rule with `paths: ["**/*.test.tsx"]`

Q251. CI/CD · Medium. Security rules should apply only to `auth/` and `payments/`. What's a good configuration?

A. `scope: security` in the frontmatter
B. Two user-level files, one for each area
C. `paths: "everything except docs"`
D. `paths: ["auth/**/*", "payments/**/*"]`

Q252. CI/CD · Medium. After a new commit, a re-run of review posts many duplicate comments. What's the fix?

A. Use a different model on each run, so that comments come out differently
B. Delete all the earlier comments on the PR before each new review runs
C. Include prior findings, and ask only for new or unaddressed issues
D. Run the review once, and never re-run it, however many commits are pushed

Q253. Developer Productivity · Medium. Which indicates that you need plan mode?

A. A short description in the task's ticket
B. A desire to keep the session as short as possible
C. A change limited to one clear line of code
D. Architectural implications, or several valid approaches

Q254. Code Generation · Medium. Your `/analyze-codebase` skill outputs thousands of lines, crowding the session. What frontmatter helps?

A. `argument-hint: verbose`
B. `allowed-tools: all`
C. `context: fork`
D. `paths: ["**/*"]`

Q255. Code Generation · Easy. Which task best suits plan mode?

A. Fixing a typo in one error message
B. Migrating a library across 45 files
C. Renaming one local variable in a file
D. Adding a null check to one function

Q256. Code Generation · Medium. A monorepo's `payments/` package needs extra conventions that apply nowhere else. What's a good place for them?

A. The root file, with a note saying they're for payments
B. The user-level file of whoever maintains the package
C. A `CLAUDE.md` in the `payments/` directory
D. A skill that developers must remember to invoke

Q257. CI/CD · Hard. Select the two flags that give machine-parseable CI output. **Select 2.**

A. `--batch`

B. `--json-schema`

C. `--parse-comments`

D. `--output-format json`

E. `--ci-mode`

Q258. Code Generation · Medium. A file in `.claude/rules/` is committed, and teammates receive it. What's the scope?

A. Global: it applies to all company repos

B. Project: it's shared through version control

C. User: it's visible to its author only

D. Session: it's removed on exit

Q259. Code Generation · Medium. What's the main reason not to put team standards in a user-level file?

A. User-level files can only hold a very small number of lines

B. User-level files aren't read in interactive sessions at all

C. They aren't shared with teammates through version control

D. User-level files are deleted when Claude Code gets updated

Q260. Code Generation · Easy. Which task suits direct execution?

A. A single-file bug fix with a clear stack trace

B. Choosing between two integration designs

C. Splitting a monolith into services

D. Restructuring a repo's module boundaries

Q261. Code Generation · Hard. Fixing validation broke the error messages, and fixing those broke logging. Sequential messages keep regressing. What should you change?

A. Start each fix in a new session, so that it isn't influenced by the others
B. Describe all of the interacting issues together, and ask for one coherent fix
C. Disable the logging tests, so that regressions stop appearing in the output
D. Keep going one by one, as the regressions will eventually stop by themselves

Q262. CI/CD · Easy. Which of these is a real flag for structured CI output?

A. `--force-schema`
B. `--json-schema`
C. `CLAUDE_JSON=1`
D. `--batch-json`

Q263. Code Generation · Medium. You're building a cache in an unfamiliar domain, and unsure what you've missed. What technique fits?

A. Ask Claude to pick all the design parameters without any further discussion
B. Implement right away, and ask about any problems that come up afterward
C. Paste a long article about caching, and ask for an implementation from it
D. The interview pattern: have Claude ask questions before implementing

Q264. Code Generation · Easy. What does the Explore subagent return to the main conversation?

A. A new CLAUDE.md for the repo
B. A summary of its discovery
C. A complete transcript of it
D. Nothing, as it can only log

Q265. Code Generation · Medium. A migration script mishandles null values. What's the most effective way to communicate the fix?

A. A general request to handle edge cases more robustly
B. A specific test case, with example input and expected output
C. A note in CLAUDE.md that edge cases are important
D. A request to review the script again, more carefully

Q266. Code Generation · Hard. What does a forked skill give back to the main conversation?

A. A copy of the main session's context, overwritten by the fork
B. Nothing at all, as forked skills can't communicate results
C. A result from the isolated run, without its verbose working
D. Its full transcript, inserted line by line in the main session

Q267. Code Generation · Medium. You must choose between two caching approaches with different infrastructure needs. What should you do?

A. Execute directly with the first approach, and revert it if there's a problem later
B. Ask Claude to implement both approaches at once, in the same branch and session
C. Pick the option that needs fewer files, and skip any further investigation at all
D. Use plan mode to explore the code and weigh both approaches before changing anything

Q268. Developer Productivity · Hard. Which belongs in CLAUDE.md rather than a skill?

A. A 40-step procedure for preparing a quarterly release
B. "All services use structured logging with a correlation ID"
C. A verbose codebase analysis run only now and then
D. A one-off template for this week's migration report

Q269. CI/CD · Hard. Why run review in a separate Claude instance from code generation?

A. An independent instance has no prior reasoning to defend, so it catches more
B. The CLI refuses to review code that the same process has generated
C. Separate instances are billed at a lower rate for the review work
D. Separate instances automatically have access to more of the repository

Q270. CI/CD · Hard. Select the two practices for avoiding duplicates in CI. **Select 2.**

A. Reducing the job's timeout by half
B. Asking the model to be more conservative
C. Running every job with a different random seed
D. Providing existing tests when generating tests
E. Including prior review findings in re-runs

Q271. CI/CD · Hard. A team uses `claude -p` output as text, and a regex extracts findings. It breaks often. What's the best change?

A. Use a larger model, as a larger model's formatting will be more consistent
B. Use `--output-format json` and `--json-schema`, and consume the structured result
C. Improve the regex, adding more cases each time that a new format appears
D. Ask developers to read the output by hand, and to post comments themselves

Q272. Code Generation · Medium. A verbose discovery phase is filling the main context. What helps?

A. Running discovery twice, so that the model can remember it better
B. The Explore subagent, which isolates discovery and returns a summary
C. Raising `max_tokens`, so that there's more room for the discovery text
D. Writing all the output to CLAUDE.md, for reference in later sessions

Q273. Code Generation · Hard. Select the two statements that are true of the CLAUDE.md hierarchy. **Select 2.**

A. User-level files override everything for all teammates
B. User-level instructions aren't shared through version control
C. Only one CLAUDE.md file can be loaded in any session
D. Directory-level files apply to work in that subtree
E. Project-level files apply only to the developer who wrote them

Q274. CI/CD · Medium. Which feedback to Claude is most useful in test-driven iteration?

A. "Try harder, this one is important"
B. "Other developers would expect better"
C. The actual failing test output
D. "It still doesn't work properly"

Q275. Code Generation · Easy. Where do you put a `/changelog` command that the whole team should get on clone?

A. `~/.claude/commands/`
B. `.claude/rules/`
C. `~/.claude/CLAUDE.md`
D. `.claude/commands/`

Q276. Code Generation · Hard. A task looks simple, but while editing you discover that it touches an unclear shared abstraction used by 30 modules. What should you do?

A. Pause, and switch to plan mode to understand the impact and options
B. Restrict changes to the first module, then leave the remainder as they are
C. Ask for the changes to be made quickly, before context fills up with analysis
D. Continue with direct edits, since the task was described as being simple

Q277. CI/CD · Medium. A CI job needs a project's review procedure. Which helps most for consistency?

A. A verbal instruction at the team's weekly meeting

B. A checked-in skill or command that defines the procedure

C. A copy pasted into each job's log output

D. A personal command on one developer's laptop

Q278. Code Generation · Medium. Developers often invoke `/migrate` and forget the target version. What helps?

A. `context: fork` in the frontmatter

B. An `allowed-tools` entry for Read

C. A longer description in CLAUDE.md

D. `argument-hint` in the frontmatter

Q279. CI/CD · Medium. A reviewer session that also generated the code misses subtle bugs. Why?

A. Sessions lose the ability to read code after they've generated any of it

B. The model is prevented by policy from criticizing its own earlier output

C. It retains its reasoning, so it rarely questions its decisions

D. Generation uses up the session's allowed number of review comments

Q280. Code Generation · Medium. A new developer's Claude Code ignores the team's code style. Everyone else's follows it. What's the most likely cause?

A. The new developer's model is older, and doesn't support CLAUDE.md files

B. The rules are in other people's `~/.claude/CLAUDE.md`

C. CLAUDE.md files only load after they've been in the repo for 24 hours

D. The team's project needs a config flag, for loading any rules at all

Q281. Code Generation · Hard. Select the two tasks for plan mode. **Select 2.**

A. Restructuring a monolith into microservices
B. Updating one constant with a known new value
C. Choosing between integration approaches, with different infrastructure
D. Fixing a known typo in the README
E. Adding a single validation to one function

Q282. Code Generation · Medium. A `/generate-docs` skill should only be able to write files, and shouldn't run shell commands. What does the guide say to configure?

A. `argument-hint` in the skill's frontmatter
B. A warning sentence in the skill's body
C. `allowed-tools` in the skill's frontmatter
D. `context: fork` in the skill's frontmatter

Q283. Code Generation · Easy. Prose descriptions of a transformation keep giving inconsistent results. What's the most effective fix?

A. Lower the model's output token limit further
B. Rewrite the prose in much stronger language
C. Ask Claude to think more carefully this time
D. Provide 2-3 concrete input/output examples

Q284. Code Generation · Hard. A team has React, API and data conventions, and tests spread throughout. Which structure fits best?

A. Path-scoped rule files for each convention, including a `**/*.test.*` rule for tests
B. One root CLAUDE.md that contains everything, with headings to separate the sections
C. A skill per convention, invoked by each developer at the start of each task
D. A subdirectory CLAUDE.md for tests, copied to each directory that contains tests

Q285. Code Generation · Hard. Select the two frontmatter options the guide names for skills, besides `allowed-tools`. **Select 2.**

A. `force-tool`
B. `context: fork`
C. `priority: high`
D. `max-iterations`
E. `argument-hint`

Q286. Code Generation · Hard. Select the two correct reasons for choosing a glob rule over a subdirectory CLAUDE.md. **Select 2.**

A. One file avoids keeping many copies synchronized
B. The convention applies to file types across many directories
C. It guarantees that the model will follow the convention
D. It's the only way to scope anything to a directory
E. It loads even when working on non-matching files

Q287. Code Generation · Easy. Where do path-scoped rules live?

A. `.claude/rules/`
B. `.claude/paths/`
C. `~/.claude.json`
D. `.mcp.json`

Q288. Code Generation · Medium. You've found three unrelated bugs in three different modules. What's a reasonable approach?

A. Fix only one, as the others probably aren't important
B. Combine them in one message, as more context always helps
C. Fix them sequentially, since they're independent
D. Ask for a redesign of all three modules before fixing anything

Q289. Code Generation · Medium. You want your own version of the team's `/review`, without affecting anyone. What's the right approach?

A. Create a personal variant in `~/.claude/skills/`, with a different name
B. Copy it to `.claude/skills/` under a second name, and commit that
C. Edit the team's skill in the repo, and don't tell anyone about it
D. Change the team's skill locally, and exclude it from each commit

Q290. Code Generation · Hard. Which test suite gives the best guidance before implementation?

A. Tests written by Claude after it has finished with the implementation
B. Tests for expected behavior, edge cases, and performance requirements
C. One happy-path test, since additional tests may confuse the model
D. Tests that check only whether the code is able to compile successfully

Q291. Code Generation · Easy. Which pattern matches all TypeScript test files anywhere?

A. `**/test.ts`
B. `test/*.ts`
C. `**/*.test.ts`
D. `*.test.ts/**`

Q292. CI/CD · Medium. Which CLAUDE.md content helps CI test generation most?

A. The team's holiday calendar and on-call rota
B. Fixture locations, and criteria for good tests
C. A record of last quarter's deployment dates
D. A statement that all the tests should be good

Q293. Code Generation · Hard. A path rule isn't applied when editing `lib/foo.spec.ts`. Its frontmatter says `paths: ["**/*.test.ts"]`. What's the cause?

A. Globs aren't allowed in the `paths` field

B. `.spec.ts` needs a skill to be loaded

C. Rules only work in the `src/` tree

D. The glob doesn't match `.spec.ts`

Q294. CI/CD · Medium. You need machine-parseable findings for inline PR comments. What should you use?

A. `--verbose`, with a script that reads the log file

B. A request in the prompt to "please reply as JSON"

C. A regular expression over the normal text output

D. `--output-format json` with `--json-schema`

Q295. Code Generation · Medium. A rule has `paths: ["src/api/**/*"]`. When does it load?

A. Only when a developer invokes it with a slash command

B. Only when running in non-interactive mode with `-p`

C. When Claude works on files under `src/api/`

D. At the start of each session, for every developer

Q296. Code Generation · Hard. A team's destructive-operations worry leads them to limit a skill to file writes. On the exam, which option is the recommended approach?

A. Configure `allowed-tools` for the skill

B. Use `argument-hint` to confirm the operation

C. Use `context: fork`, so that it can't do damage

D. Rely on a skill description saying "be careful"

Q297. Code Generation · Medium. What does test-driven iteration with Claude involve?

A. Delete failing tests, so that the suite stays green
B. Run the tests once at the end of the entire project
C. Write the tests first, then iterate by sharing failures
D. Write the code first, then ask for tests that pass

Q298. Developer Productivity · Hard. Select the two reasons to use the Explore subagent. **Select 2.**

A. It isolates verbose discovery output
B. It enforces compliance rules on tool calls
C. It preserves the main conversation's context
D. It guarantees that the code it finds is correct
E. It removes the need for any planning at all

Q299. Code Generation · Medium. Which is a poor reason to enter plan mode?

A. There are architectural decisions to be made
B. You want to change one clear line with a known fix
C. The change will affect many files and modules
D. The task has several valid approaches to compare

Q300. Code Generation · Hard. Select the two techniques for inconsistent transformations. **Select 2.**

A. Stronger adjectives in the request
B. Larger maximum output token settings
C. Asking for confidence scores on results
D. Concrete input/output examples
E. Tests that show the expected results

Q301. Code Generation · Hard. Maintainers of each package know which standards apply to it. Which design lets them control that?

A. Each package's CLAUDE.md imports only the standards files its maintainers choose

B. A central script writes one combined file into each package's directory at build time

C. A user-level file on each maintainer's machine holds that package's standards

D. All standards load for all packages, and the model works out the relevant ones

Q302. Code Generation · Easy. How do you share instructions with the team through version control?

A. Put them in the model's output style

B. Put them in a project-level CLAUDE.md

C. Send them to each developer by email

D. Put them in `~/.claude/CLAUDE.md`

Q303. Code Generation · Hard. When would you not use the interview pattern?

A. For a small, well-specified change with no open questions

B. For a design with unclear invalidation requirements

C. For a feature with failure modes that aren't understood

D. For an unfamiliar domain with many design decisions

Q304. Code Generation · Easy. Which syntax references an external file from CLAUDE.md?

A. `#include`

B. `@import`

C. `require()`

D. `<link>`

Q305. CI/CD · Hard. A blocking pre-merge review must post inline comments quickly. Which combination fits?

A. Synchronous `-p` run, JSON schema output, and a script to post comments
B. Interactive session on a developer's laptop, copied into the PR by hand
C. Text output with a nightly job that reformats it for the next day
D. Batch API submission, with results collected on the following morning

Q306. Code Generation · Hard. Migration SQL files exist in `db/`, `services/*/migrations/` and `tools/`. Which design has the least maintenance?

A. A Git hook which comments on the rule when any SQL file has changed
B. One long section in the root CLAUDE.md that's loaded in every session
C. A CLAUDE.md in each of those directories, kept in sync by hand
D. One rule file with glob paths that match all migration files

Q307. Code Generation · Hard. Select the two cases for sending all issues in one message. **Select 2.**

A. Each issue has an isolated one-line fix
B. A change to one alters the others' behavior
C. Each can be verified independently
D. The issues are in unrelated services
E. The fixes depend on each other

Q308. Code Generation · Medium. Your root CLAUDE.md is 900 lines covering testing, APIs and deployment. What's a good restructuring?

A. Move it all to `~/.claude/CLAUDE.md`, so that it's no longer in the repository
B. Copy it into each subdirectory, so that each one has its own complete version
C. Split it into topic files in `.claude/rules/`, such as `testing.md`
D. Delete sections until it's short, and rely on developers to remember the rest

Q309. Code Generation · Hard. Select the two good uses for `.claude/rules/`. **Select 2.**

A. Storing the conversation history of past sessions

B. Storing API keys that the team needs to share

C. Conventions scoped to paths through frontmatter

D. Splitting a huge CLAUDE.md into topic files

E. Holding one developer's personal preferences

Q310. Code Generation · Medium. Different packages need different subsets of your standards. What's the best way to include them?

A. Paste all the standards into each package's file, and keep them in sync by hand

B. Create one skill per standard, and ask developers to run them before each edit

C. Include every standard at the root, and let Claude decide which ones to ignore

D. Use `@import` in each package's CLAUDE.md, for the standards relevant to it

Q311. CI/CD · Medium. In CI, Claude Code must follow project conventions. Where should those go?

A. In the model's hidden system prompt

B. In a Slack message to the team

C. In the CI runner's home-directory file

D. In the repository's CLAUDE.md

Q312. CI/CD · Hard. Your CI review keeps flagging issues that were already marked as accepted. What's the best design?

A. Lower the number of files reviewed, so that fewer issues get flagged

B. Run the review only on Fridays, so that repeated comments are less frequent

C. Tell the model that developers have stopped trusting it, and to be conservative

D. Pass prior findings and their status, and skip the accepted ones

Q313. Code Generation · Medium. A team must choose where to put its "always use parameterized SQL" rule. Which is right?

A. A personal command, so that each developer can decide when it applies
B. CLAUDE.md, as it's a universal standard that should always be loaded
C. A hook, since a CLAUDE.md can never contain any security guidance
D. A skill, as it's more convenient to invoke the rule when you remember

Q314. Code Generation · Medium. What's the downside of a very broad rule such as `paths: ["**/*"]`?

A. It loads for all files, losing the benefit of scoping
B. It applies to everybody except the file's author
C. It removes all other rule files from loading
D. It causes a parse error when the session is started

Q315. Code Generation · Medium. Claude seems to follow a rule in one session and not in another. Which command helps you diagnose this?

A. `/memory`
B. `/resume`
C. `/compact`
D. `/clear`

Q316. Code Generation · Hard. Select the two correct scope statements. **Select 2.**

A. `~/.claude/skills/` is personal to the developer
B. `.claude/commands/` is shared through version control
C. `.claude/commands/` works only for the repo's owner
D. `.claude/skills/` can't be committed to the repository
E. `~/.claude/commands/` reaches everybody on clone

Q317. Code Generation · Hard. A rule appears in root CLAUDE.md, yet it's not being followed in `services/api/`. `/memory` shows that `services/api/CLAUDE.md` is loaded and says something different. What's the best fix?

A. Resolve the conflict, so that the two files don't contradict each other
B. Delete `/memory` output from the session, so the conflict's no longer seen
C. Raise the priority of the root file using a numeric frontmatter field
D. Run `/compact`, as that merges all memory files into one consistent version

Q318. Code Generation · Easy. How many concrete examples does the guide suggest to clarify a transformation?

A. Exactly 1
B. 25–30
C. None
D. 2–3

Q319. CI/CD · Medium. Where does CI-invoked Claude Code get testing standards and fixture conventions?

A. The repository's CLAUDE.md
B. The developer's user-level file
C. An environment variable, per rule
D. The CI server's system log

Q320. Code Generation · Medium. Which option is a benefit of project-scoped commands?

A. They run faster than user-scoped commands on most machines
B. They're automatically loaded into every request's context
C. Everyone gets the same workflow after they pull the repo
D. They bypass all of Claude Code's permission checks

Q321. Code Generation · Medium. What's plan mode's main benefit?

A. Safe exploration and design before committing to changes, preventing costly rework
B. Faster generation of code, since nothing needs to be checked during a plan
C. Automatic merging of code to the main branch when a plan has been approved
D. Immunity from errors, since a planned change can't introduce any bugs at all

Q322. Code Generation · Hard. Claude's date parser passes your examples but fails on an unseen format. What's the best next step?

A. Start a fresh session with the same description
B. Tell it the code is wrong, and ask it to try again
C. Add a failing test covering that format, then iterate
D. Remove that format from what the product supports

Q323. Code Generation · Hard. "Upgrade our auth library; it has breaking changes, and it's used in about 60 places." What's the best workflow?

A. Plan mode for usages and approach, Explore for search, then execute
B. Direct execution, starting with the first file that happens to fail to compile
C. A skill with `context: fork` to make all of the changes without any review
D. A single prompt asking for all 60 changes at once, without any exploration

Q324. Code Generation · Medium. When is a subdirectory CLAUDE.md the better choice over a glob rule?

A. When conventions apply to test files spread across packages
B. When the rule has to load only for a certain file extension
C. When conventions apply to one directory's whole subtree
D. When the rule must apply to every `*.sql` file in the repo

Q325. Developer Productivity · Medium. A skill must be available in all of your projects, though not to your teammates. Where should it go?

A. `.claude/rules/`

B. `CLAUDE.md`

C. `.claude/skills/`

D. `~/.claude/skills/`

Domain 4: Prompt Engineering and Structured Output

Q326. Data Extraction · Hard. Select the two uses of few-shot examples named in the guide for extraction. **Select 2.**

A. Addressing empty or null extraction of required fields
B. Reducing the size of the input document
C. Handling varied document structures
D. Replacing the need for validation altogether
E. Guaranteeing that the JSON syntax is valid

Q327. CI/CD · Hard. A nightly batch job would need tool calls to read files during generation. How can it still use batch?

A. Include the file contents in each request, so no tools are needed
B. Have each batch request call back to your server by URL, for each needed file
C. Enable a flag allowing batch requests to pause while waiting for your tools to run
D. Use `tool_choice: "any"`, so that batch can execute tools on your behalf

Q328. Code Generation · Easy. Which technique is best for showing a required output format?

A. A bigger iteration limit
B. A paragraph on its importance
C. An example of that output
D. A second model reviewing it

Q329. Data Extraction · Easy. What's the main limit of retry loops?

A. They don't work with the use of JSON schemas at all
B. They can only ever be used with synchronous API calls
C. They can't supply information that's absent from the source
D. They always raise the model's confidence, not its accuracy

Q330. CI/CD · Medium. Your manager wants a pre-merge check moved to batch, to save money. What's the problem?

A. Batch costs more than the synchronous API for jobs of fewer than 100 items
B. Batch results come back in random order, which CI can't possibly handle
C. Batch can't process code, and only accepts plain natural-language documents
D. Batch has no latency guarantee, and can take up to 24 hours, blocking merges

Q331. Data Extraction · Medium. Extraction fails because the total appears only in an attachment that wasn't provided. Will retries help?

A. Yes, as long as the prompt makes the requirement more emphatic
B. No: the information is absent, so retries can't succeed
C. Only if `tool_choice` changes to forced selection on the retry
D. Yes: with enough retries, the model will eventually find it

Q332. Data Extraction · Medium. Which approach best matches two workflows: live chat replies, and a weekly audit?

A. Batch for chat, and synchronous for the audit
B. Synchronous for chat, and batch for the audit
C. Synchronous for both, as it's always more accurate
D. Batch for both, as that saves the most money

Q333. CI/CD · Medium. What does a per-file pass miss by design?

A. Style issues, which need a dedicated style-only agent
B. Cross-file data flow issues, which need an integration pass
C. Local bugs in the file, which need an integration pass
D. Any security bug, which can't be found in a single file

Q334. Data Extraction · Medium. What's a good response when a required value truly isn't in the source?

A. Return null, and flag it for follow-up
B. Copy a value from similar documents
C. Invent a plausible default value
D. Retry until a value is produced

Q335. Data Extraction · Medium. An extraction prompt says "extract the important dates," and results vary. What helps most?

A. Increase the output limit, so that more of the document's dates can be included
B. Name exactly which dates: invoice date, due date, service period start and end
C. Ask for all the dates that might be important, then have a script remove some
D. Tell the model that the dates are very important to the company's accounting

Q336. Code Generation · Hard. A developer generates code and asks the same session, "Any bugs?", and it says no. A new session finds two. What's the best team practice?

A. Ask the original session three times, as it may find more on later attempts
B. Require the generator to include a confidence score with each piece of code
C. Review generated code in a fresh session or an independent instance
D. Ban the use of AI for reviews, as the two sessions have different opinions

Q337. CI/CD · Hard. You want to see whether a prompt's criteria improved a review. What's the best evidence?

A. Whether developers say at the next meeting that it seems somewhat better
B. The total number of findings, since fewer findings must mean more accuracy
C. The model's own statement that it's more accurate with the new prompt
D. False-positive rate by category, on labelled examples, before and after

Q338. CI/CD · Easy. Which workload suits the batch API?

A. A developer waiting at a terminal
B. An overnight technical debt report
C. A blocking pre-merge check
D. A chatbot's live customer reply

Q339. Data Extraction · Medium. Where do you read the structured result from when using tool use for extraction?

A. The response's HTTP headers
B. The `tool_use` block's input
C. The `stop_reason` of the response
D. The final text block's content

Q340. Data Extraction · Hard. Select the two correct batch failure practices. **Select 2.**

A. Identify failures by `custom_id`
B. Change what caused failure, such as chunking long documents
C. Ignore the failures once 90% have succeeded
D. Resubmit everything, including all the successes
E. Retry unchanged, until each one finally passes

Q341. Data Extraction · Hard. How can few-shot examples reduce hallucination in extraction?

A. They make the model refuse any document containing informal measurements
B. They guarantee correct values because the model copies the example's numbers
C. They remove the need for any schema, since the examples define the format
D. They show how to handle informal or missing values, rather than inventing them

Q342. CI/CD · Hard. A prompt says, "Flag security problems." Results include many theoretical ones. Which revision is best?

A. "Flag security problems, but be really careful, and don't bother anyone with anything minor at all."
B. "Flag security problems with a confidence above 90%, and leave the others out of the final report."
C. "Flag only injection, auth bypass and secret exposure; skip theoretical hardening."
D. "Flag all security problems, and rank them with the most serious ones at the top of your output."

Q343. Data Extraction · Medium. Which failure is most likely fixable by retrying with error feedback?

A. A customer ID that exists only in a separate system of record
B. A signature that's on a page that wasn't included in the upload
C. A date returned as "March 4" when the format must be ISO 8601
D. A contract end date that isn't stated anywhere in the document

Q344. Data Extraction · Hard. Select the two schema designs that reduce hallucination. **Select 2.**

A. Nullable fields for possibly absent data
B. An "unclear" value for ambiguous data
C. A rule that empty values are never allowed
D. A single required free-text "notes" field
E. Required fields on all possible attributes

Q345. CI/CD · Hard. Select the two properties of effective multi-instance, multi-pass review. **Select 2.**

A. Reviewers read the generator's rationale first
B. All passes share a single session, for continuity
C. Local analysis and integration analysis are separate passes
D. The reviewer lacks the generator's reasoning context
E. One pass covers everything, for maximum consistency

Q346. Data Extraction · Hard. An SLA requires results in 26 hours, with batch processing taking up to 24. What's the longest submission interval that still guarantees it?

A. 2 hours
B. 26 hours
C. 6 hours
D. 12 hours

Q347. CI/CD · Medium. Which prompt pattern is a known failure for reducing false positives?

A. "Severity High means data loss, as shown below"
B. "Report null dereferences on reachable paths"
C. "Only flag issues if you're confident"
D. "Skip naming preferences unless they cause bugs"

Q348. Data Extraction · Easy. What's a consequence of "no guaranteed latency SLA"?

A. Results may be lost after one hour
B. Results always arrive in one minute
C. Results might take up to 24 hours
D. Results are charged at a higher rate

Q349. CI/CD · Hard. The generator session's summary is given to the reviewer instance "for context." The reviewer now misses the same bugs. Why?

A. The reviewer can't read any code if it has received a summary first
B. The summary carries the generator's reasoning, which undermines independence
C. The reviewer's model automatically defers to all of its previous messages
D. The summary used too few tokens to give the reviewer enough information

Q350. CI/CD · Hard. A team proposes removing false positives by showing only findings where the model's confidence exceeds 0.9. What's the concern?

A. Self-reported confidence isn't a reliable filter, so write explicit criteria first
B. Filtering after generation is prohibited, as all findings must reach the developer
C. A threshold of 0.9 is too low, and it would need to be set to at least 0.99
D. Confidence scores aren't available in structured output, so the plan can't be implemented

Q351. Code Generation · Medium. "Make this code better" gives uneven results. Which change helps most?

A. Ask for several versions of the improved code, and choose whichever looks the best
B. Repeat the phrase "make it better" at both the start and end of the request
C. Add "please take your time, and be very thorough about everything you do here"
D. State goals, such as "remove duplicated parsing, and keep the public API"

Q352. CI/CD · Easy. Which review instruction best improves precision?

A. "Be conservative, and report only the findings you're really sure about"
B. "Check that all of the comments in this code are accurate and helpful"
C. "Do a thorough and careful review, and take as much time as you need"
D. "Flag comments only when the claimed behavior contradicts the code"

Q353. Data Extraction · Hard. A source contract has two different start dates. What's the best extraction behavior?

A. Choose the later of the two dates, as later values usually supersede earlier ones
B. Set `conflict_detected`, include both values with locations, and route it for review
C. Retry until the model settles on one of the dates, and then accept whatever it says
D. Leave the field out, and don't mention that the document contained any conflict

Q354. CI/CD · Medium. Which is the best example of a false-positive-reducing few-shot?

A. Input: long file. Output: a list of all the variable names that appear within the file
B. Input: retry loop with a capped sleep. Output: no issue, because the cap prevents a hot loop
C. Input: any code at all. Output: looks fine to me, no need for any further review of it
D. Input: a security bug. Output: this is very bad, and you should really fix it right away

Q355. Data Extraction · Medium. Some batch documents failed for exceeding context limits. What modification helps?

A. Chunk those documents, then resubmit them
B. Resubmit them unchanged in a new batch
C. Use `tool_choice: "any"` when resubmitting
D. Raise the batch's `max_retries` to 100

Q356. CI/CD · Medium. Why do false positives matter beyond their own count?

A. They reduce the model's accuracy on later runs, since it learns from them
B. They increase the token count of the review, which then gets truncated
C. They trigger the API's safety system, and that blocks later reviews
D. They undermine developers' trust in the accurate findings as well

Q357. CI/CD · Easy. What does the guide say the Message Batches API offers?

A. 90% cost savings, 1-hour processing, and a guaranteed latency SLA
B. The same price, faster processing, and multi-turn tool calling inside
C. 25% cost savings, 4-hour processing, and streaming results as they finish
D. 50% cost savings, up to 24-hour processing, and no latency SLA

Q358. Data Extraction · Medium. Which schema choice best prevents fabricated values?

A. A rule in the prompt saying that values must be real
B. Required fields, each with a clear description
C. Nullable fields for information that may be absent
D. Replacing all string fields by numeric identifiers

Q359. CI/CD · Easy. Detailed instructions still give inconsistent output formats. What does the guide name as most effective?

A. More output tokens per call
B. Stronger capitalized rules
C. Few-shot examples
D. A longer list of don'ts

Q360. Data Extraction · Medium. A source document may not state a PO number, and the required field gets fabricated values. What's the schema fix?

A. Remove it, and all other optional data
B. Make the field optional or nullable
C. Keep it required, and add a pattern
D. Add "never fabricate" to a tool's name

Q361. CI/CD · Medium. Your review comments vary in structure. Which example content fixes this?

A. Examples of well-known bugs from famous open-source projects, without any format
B. Examples that show location, issue, severity and suggested fix in the required format
C. Examples of the team's coding style guide, pasted as one very long paragraph
D. A single example that contains all the kinds of bug that have ever been found

Q362. CI/CD · Medium. Developers dismiss many findings, and you want to know why. Which field helps?

A. `word_count`, recording how long each comment was when it got posted
B. `detected_pattern`, recording the code construct that triggered each finding
C. `is_important`, recording the model's own opinion of each finding's importance
D. `model_version`, recording which of the models generated each of the findings

Q363. CI/CD · Easy. Which is a good way to define review scope?

A. Mention that the team really dislikes noisy reviews, and hope that it adjusts
B. Ask for only those issues that a senior engineer would find important enough to fix
C. List what to report (bugs, security) and what to skip (minor style, local patterns)
D. Report anything that might be a problem, and let the developers filter findings later

Q364. CI/CD · Hard. Select the two steps that restore developer trust in a noisy reviewer. **Select 2.**

A. Temporarily disable the high false-positive categories
B. Hide the reviewer's comments behind a collapsed section
C. Add "be accurate" to the top of the system prompt
D. Rewrite those categories with explicit, example-based criteria
E. Post every finding, in order to prove thoroughness

Q365. CI/CD · Medium. One review category, style nits, has a high false-positive rate, and developers now ignore all findings. What's a good immediate step?

A. Require developers to respond to all of the comments that get posted
B. Lower the severity of all findings by one level across the board
C. Temporarily disable that category, while you improve its prompt
D. Add a reminder at the top of each comment, saying that it may be wrong

Q366. Data Extraction · Hard. Your validator catches a mismatch, and the retry fixes the sum by changing a line item that was correct. What does this suggest?

A. The validator should be turned off, since it's what led to the change in a correct value
B. Retry feedback should be removed, since the model can't use any specific information at all
C. Retries can force plausible but wrong fixes, so recheck against the source
D. A sum mismatch can only happen because of a schema problem, and not in the extraction

Q367. Data Extraction · Hard. Before batch-processing 500,000 documents, what should you do to minimize costly resubmissions?

A. Refine the prompt on a sample set, to maximize first-pass success
B. Skip validation for the first run, and do it during the second run only
C. Split it across many models, so that one of them is sure to do well
D. Submit it all at once, then improve the prompt using the failed results

Q368. Data Extraction · Hard. Dates arrive as "3/4/26", "March 4", and "2026-03-04". A strict schema still produces inconsistent values. What's the fix?

A. Use `tool_choice: "any"`, as this normalizes field formats automatically
B. Add format normalization rules to the prompt, alongside the schema
C. Make the date field required, so that the model will take more care
D. Change the type to `integer`, so that dates are stored as numbers

Q369. Customer Support · Hard. An agent's escalations are inconsistent for refund-policy edge cases. Which addition is most effective?

A. A few examples of borderline cases: resolve or escalate, and why
B. A long list of all the policy text that's been written in the last five years
C. A rule saying that the agent should escalate whenever it's unsure about something
D. A sentiment threshold, above which the agent should automatically escalate a case

Q370. CI/CD · Hard. What's the purpose of verification passes with self-reported confidence on each finding?

A. To remove any need for human reviewers on all findings
B. To enable calibrated routing of findings to review
C. To replace explicit criteria in the review's prompt
D. To make the model more accurate on the first attempt

Q371. Customer Support · Medium. Why do few-shot examples help with new situations that aren't in the examples?

A. They add the examples to the model's training data for later sessions
B. They force the model to match every new input to its nearest example
C. They demonstrate judgment that the model can generalize to novel patterns
D. They only help with cases identical to the examples that were provided

Q372. CI/CD · Easy. What's "attention dilution"?

A. Tokens fall as a session grows longer
B. Quality drops as one pass covers too much
C. Costs rise as more developers use a tool
D. Latency rises when two instances are used

Q373. Data Extraction · Hard. Select the two failures likely to succeed on a retry with feedback. **Select 2.**

A. A wrong date format
B. A value placed in the wrong field
C. A value found only in an unattached file
D. A page missing from the scanned source
E. A number that isn't present in the document

Q374. Data Extraction · Easy. What should a retry include to help the model correct itself?

A. The original document, the failed extraction, and the specific validation errors
B. A different document, so that the model isn't influenced by what it did before
C. Only the words "that was wrong, so please try once more" and nothing else
D. Only the schema again, with stronger instructions regarding its importance

Q375. CI/CD · Hard. Select the two fields that support feedback-loop analysis or self-checking. **Select 2.**

A. `tool_choice`
B. `conflict_detected`
C. `please_retry`
D. `max_tokens`
E. `detected_pattern`

Q376. CI/CD · Hard. Select the two workloads for batch. **Select 2.**

A. Overnight report generation
B. An IDE's inline suggestions
C. Live support replies
D. Weekly compliance audits
E. Pre-merge blocking checks

Q377. CI/CD · Hard. Tests suggested by Claude miss branch-level gaps. Which few-shot example helps?

A. One showing the coverage percentage for the project in the last quarterly report
B. One showing a covered line whose false branch is untested, and the test that closes it
C. One showing a very long test file from the project with no explanation at all
D. One showing a function, along with the statement "write good tests for this"

Q378. Data Extraction · Hard. You must deliver results within 30 hours of receipt, and batch processing can take 24. How often must you submit?

A. At least every 6 hours, leaving up to 24 for processing
B. Once every 48 hours, to get the largest possible batches
C. Once every 30 hours, to match the agreed SLA exactly
D. Once every 24 hours, as that's the length of the window

Q379. Developer Productivity · Medium. You need a tool call instead of a chatty reply, though any of three tools is fine. Which option does the guide name?

A. `max_tokens: 1`
B. `tool_choice: "auto"`
C. `stop_reason: "tool_use"`
D. `tool_choice: "any"`

Q380. CI/CD · Hard. Select the two instructions that are specific criteria. **Select 2.**

A. "Be conservative in what you flag"
B. "Prioritize quality over quantity"
C. "Only report what you're confident about"
D. "Report SQL built by string concatenation from user input"
E. "Skip whitespace and import-order issues"

Q381. Data Extraction · Hard. The document type's unknown, and you have one extraction tool for each type. What's the right `tool_choice`?

A. `"any"`
B. `"auto"`
C. `"none"`
D. A forced tool

Q382. CI/CD · Medium. What's the most effective way to review generated code?

A. The same session, asked to repeat its generation and compare the results
B. A second, independent Claude instance without the generator's reasoning
C. The same session, asked to be highly critical of what it has just done
D. The same session with extended thinking, asked to double-check its work

Q383. CI/CD · Medium. Reviewers disagree on what "high severity" means, and the model is inconsistent. What's the best fix?

A. Remove severity from the output, so that there's no longer any disagreement
B. Have the model report a numeric confidence, and use that as the severity
C. Ask the model to use its own best judgment on severity for each one
D. Define each severity level explicitly, with concrete code examples

Q384. Data Extraction · Medium. An extractor returns null for required fields when documents are formatted unusually. What's a good fix?

A. Add few-shot examples that show correct extraction from those varied formats
B. Ask the model in general terms to try harder at finding the missing values
C. Lower the retry count, so that fewer extra calls are spent on these documents
D. Make the fields required in the schema, so that a value has to be returned

Q385. CI/CD · Medium. Which input suits an independent reviewer?

A. The generator's whole conversation and its reasoning
B. Only the file names that were changed, without the diff
C. The generator's opinion about the risks of the code
D. The diff, the requirements and the review criteria

Q386. CI/CD · Easy. Why is self-review in the same session weak?

A. It keeps its reasoning, so it rarely questions its decisions
B. The same session can only produce positive feedback on any code at all
C. The model can't read code it has previously written in that session
D. The API charges twice as much for reviewing code in the same session

Q387. Data Extraction · Medium. Why doesn't tool use remove the need for validation?

A. It guarantees structure, and not whether values are correct
B. It returns plain text, which needs to be parsed and checked
C. It's disabled when documents are larger than a few pages
D. It stops the model from reading numbers in the document

Q388. Data Extraction · Hard. `extract_metadata` has to run before enrichment. Which configuration ensures that?

A. `tool_choice: "none"`, followed by a parse of the model's description of its plan
B. `tool_choice: "any"` on the first request, listing `extract_metadata` first
C. `tool_choice: {"type": "tool", "name": "extract_metadata"}` on the first request
D. `tool_choice: "auto"`, with a strong statement of the required order in the prompt

Q389. CI/CD · Easy. What's the main effect of vague instructions on review output?

A. A schema error on the structured output
B. A rejection of the request by the API
C. A shorter output, with higher precision
D. Inconsistent findings, with lower precision

Q390. Customer Support · Hard. Select the two properties of a good few-shot example for ambiguous requests. **Select 2.**

A. It's as long as the longest real transcript
B. It explains why one option was chosen
C. It repeats a case that's already handled well
D. It targets a case the model gets wrong
E. It contains no reasoning, in order to save tokens

Q391. Data Extraction · Medium. You have 15 examples, and they're all clear-cut invoices. Accuracy on odd invoices hasn't changed. Why?

A. Examples only ever affect format, and they can't affect accuracy at all

B. Invoices are the one document type that few-shot examples don't work on

C. Examples help most on ambiguous cases, and yours don't show them

D. 15 is over the API's limit, so the model has silently dropped them

Q392. Data Extraction · Medium. Which retry message is the most useful?

A. "Your answer failed our validation, so please read it again and be more accurate."

B. "This is very important for our finance team, so please try again and get it right."

C. "Please output JSON. Please output JSON. Please output JSON. Do not output anything else."

D. "`line_items` sum to 104.50, but `total` is 140.50. Re-check both against the document."

Q393. Data Extraction · Hard. Documents cite sources either inline or in a bibliography, and extraction misses one form. What's the best prompt change?

A. Remove citation fields from the schema, as those documents are too inconsistent

B. Add examples showing correct handling of both inline citations and bibliographies

C. Add an instruction that tells the model citations are important to the company

D. Add a rule stating that citations are always in the bibliography, for consistency

Q394. Data Extraction · Hard. Line items don't sum to the stated total in some outputs. Can a stricter schema alone fix it?

A. No: that's semantic, so add validation that compares calculated and stated totals

B. Yes: adding `required` to all of the fields makes sums come out correctly

C. No: nothing can fix this, so remove totals from the output altogether

D. Yes: changing number fields to strings removes all of the arithmetic issues

Q395. CI/CD · Medium. Which is the best report/skip criterion for comment accuracy?

A. Report when a comment states behavior the code doesn't have; skip outdated wording that doesn't mislead
B. Skip comments altogether, as they aren't executed, and so they can't cause any real bugs at all
C. Report any comment that could conceivably be misunderstood by someone reading it at some later time
D. Report all comments, since comments are generally less reliable than code and should be reviewed

Q396. CI/CD · Hard. Your review finds real bugs, but it also flags a local pattern the team has adopted deliberately. What's the best approach?

A. Disable the entire review category that the pattern happens to fall within
B. Ask the model to respect the team's patterns in general, wherever they appear
C. Require a higher confidence score on all findings that mention this pattern
D. Add the pattern to explicit skip criteria, with an example of acceptable code

Q397. Data Extraction · Medium. A field accepts a value in `currency`, and the model sometimes puts a number there. What does strict schema use guarantee?

A. That the model will have read the whole document first
B. That nothing is ever placed in the wrong field by the model
C. That its value is correct for that document in all cases
D. That its type and shape conform, and not that the value is right

Q398. CI/CD · Medium. A 30-file PR review gives contradictory findings across files. What's the best structure?

A. One pass per reviewer, then a vote among them across all findings in the PR
B. A separate review only for the three files that have the most changed lines
C. One large pass with a larger output limit, and a request for more consistency
D. Per-file local passes, plus a separate integration pass for cross-file issues

Q399. Data Extraction · Medium. A batch of 10,000 finishes with 300 failures. What's the best next step?

A. Process them all through one long synchronous request as one combined document

B. Discard the failures, since a 3% failure rate is within a normal tolerance

C. Resubmit the entire batch, so that everything has been processed in the same way

D. Resubmit only the failed documents by `custom_id`, with suitable changes

Q400. Data Extraction · Medium. You use strict schemas through tool use. Which errors remain possible?

A. Missing required keys defined in the tool's input schema

B. Semantic errors, such as line items not summing to the total

C. JSON syntax errors, such as unbalanced brackets or stray commas

D. Values having a type other than what the schema has declared

Q401. CI/CD · Medium. Reviewers flag safe patterns as issues. Which few-shot approach helps?

A. Examples that show high-confidence findings, with no mention of why they count

B. Only examples of real bugs, so that the model has plenty of positive cases

C. Examples distinguishing acceptable code from genuine issues, for similar-looking cases

D. Only examples of acceptable code, so that it reports far fewer issues overall

Q402. Data Extraction · Medium. Which is a semantic validation error?

A. Values don't sum to the stated total

B. An unclosed bracket in the JSON output

C. A trailing comma in the JSON output

D. A property name in the wrong quotes

Q403. CI/CD · Hard. Feedback data shows that a finding type is dismissed by one team, and accepted by another. What should you check?

A. Whether the first team should be told that they must accept all of its findings
B. Whether the review should be stopped for both teams until they've agreed on it
C. Whether the model should be told to be friendlier to the team that's dismissing
D. Whether criteria should differ by codebase, using `detected_pattern`

Q404. Data Extraction · Medium. Why extract both `calculated_total` and `stated_total`?

A. To flag discrepancies between what's stated and what the items add up to
B. To avoid validating the totals in code, since the model has done so already
C. To let the model choose whichever of the two it feels is the more accurate
D. To increase the chance that one of the two values is accepted by the schema

Q405. CI/CD · Medium. CI needs structured findings from Claude Code. What's the CLI equivalent of schema-enforced output?

A. `--batch` with `--output json`
B. `--tool-choice any` with `--strict`
C. `--output-format json` with `--json-schema`
D. `--print-json` with `--enforce`

Q406. CI/CD · Hard. Analysis shows that 70% of dismissed findings share one `detected_pattern`. What should you do?

A. Investigate that pattern, then revise the criteria or add examples for it
B. Raise the model's confidence threshold across all of the review's categories
C. Ignore it, since dismissal behavior says nothing about the review's quality
D. Retrain the model on those findings, so that it stops producing that pattern

Q407. Customer Support · Medium. What should a few-shot example for an ambiguous tool choice show?

A. Only the final tool name, so that the example stays as short as it can be

B. The reasoning for why one action was chosen over plausible alternatives

C. A customer's full transcript, including all of the irrelevant small talk

D. Every tool in the system, each one used once, for the sake of balance

Q408. Customer Support · Medium. How many targeted few-shot examples does the guide recommend for ambiguous scenarios?

A. As many as fit

B. 20–30

C. 2–4

D. 1 at most

Q409. Data Extraction · Hard. A retry loop runs up to five times on all failures. Costs are high, and absent-data cases always fail. What's the best improvement?

A. Reduce the limit to one retry for all failures, whatever their type

B. Increase the limit to ten retries, so that hard cases have a better chance

C. Send all failures to a larger model, without any explanation of the error

D. Classify failures first, and retry only those that retrying can fix

Q410. CI/CD · Medium. Why might a single pass report a pattern as bad in one file and fine in another?

A. The API randomizes the review's severity field

B. The second file wasn't sent to the model

C. The two files use different JSON schemas

D. Attention spreads unevenly across a large review

Q411. Data Extraction · Medium. With `tool_choice: "auto"`, what could happen on an extraction request?

A. The API automatically writes the JSON for the model
B. The tool is called twice, for the sake of redundancy
C. The request fails unless the document contains JSON
D. The model may return text and not call the tool

Q412. CI/CD · Medium. Extended thinking is enabled, and self-review still misses subtle issues. What should you try?

A. An independent instance for the review
B. A higher thinking budget in the same session
C. A second review in the same session
D. A stronger request to be self-critical

Q413. Data Extraction · Easy. A tool's JSON schema is passed as what?

A. A separate call to a validation endpoint
B. The tool's input parameters definition
C. The `stop_reason` value on the response
D. A file attachment to the model request

Q414. Data Extraction · Easy. What does the guide describe as the most reliable way to get schema-compliant structured output?

A. A prompt saying "reply in JSON"
B. A regex over the text response
C. Tool use with a JSON schema
D. A few-shot example by itself

Q415. Data Extraction · Hard. Select the two correct statements about a strict JSON schema used through tool use. **Select 2.**

A. Not eliminated: JSON syntax errors in tool input

B. Eliminated: a date that's in the wrong field

C. Not eliminated: a total that doesn't equal its line items

D. Eliminated: a value that the document doesn't contain

E. Eliminated: unbalanced JSON brackets

Q416. CI/CD · Hard. Your integration pass flags a bug that per-file passes didn't: a field renamed in a producer, but not in a consumer. What does this show?

A. Multi-pass review's value: that bug only appears across files

B. The integration pass should replace the per-file passes altogether

C. The bug could only have been caught by a human reviewer anyway

D. Per-file passes were broken, and should be thrown out entirely

Q417. Data Extraction · Medium. Can a batch request run a tool, receive its result, and continue in that same request?

A. No: batch doesn't support multi-turn tool calling inside a request

B. Yes: batch runs the agentic loop for you on Anthropic's servers

C. Yes, as long as tools are declared with a batch-specific flag

D. Only with `tool_choice: "any"` on each of the batch's requests

Q418. Customer Support · Hard. Your few-shot examples mention `get_customer` first in all cases. The agent now calls it even when it isn't needed. What's the fix?

A. Add a rule at the end of the prompt, saying to ignore the examples sometimes

B. Remove the tool's description, as the examples are now defining its use

C. Add more examples with the same pattern, in order to reinforce them further

D. Vary the examples to include cases where another tool is right, with the reasons

Q419. Data Extraction · Medium. Categories are mostly known, though new ones appear. What's the best design?

A. A free-text string, with no constraints of any kind on it
B. A required boolean for each category that's ever appeared
C. An enum with an "other" value, plus a detail string
D. An enum of today's values, and reject all other documents

Q420. CI/CD · Medium. Nightly test generation for all repositories is taking a costly synchronous run. What's the best option?

A. Run it only monthly, with no other changes to the process
B. Keep it synchronous, as test generation can't run in batch
C. Move all interactive code review to the batch API as well
D. Move it to the batch API, as nobody is waiting on it overnight

Q421. Data Extraction · Hard. Which is the best description of an "other" + detail pattern?

A. `category` is free text, and `other` is a list of all values that aren't currently in use
B. `other` is a required boolean, while `detail` holds the model's confidence in the category
C. `category` enum includes "other", and `category_detail` carries the specifics when it's chosen
D. `category` enum rejects anything unknown, while `detail` holds an error message for the retry

Q422. Data Extraction · Medium. Some source documents are unclear on a classification. What's the best schema addition?

A. A required confidence of 100
B. An "unclear" enum value
C. A retry until it's decided
D. A longer field description

Q423. CI/CD · Medium. A team adds "only report high-confidence findings" to its review prompt, and precision barely changes. Why?

A. The model can't produce any output when it's unsure about a finding
B. The phrase has to appear in the tool description to have any effect at all
C. General confidence instructions don't define which issues should count
D. The instruction is overridden by the JSON schema that's on the output

Q424. Data Extraction · Hard. A "conflict_detected" boolean is added to a schema. What's its purpose?

A. It validates the JSON syntax of the other fields in the output
B. It makes the model resolve conflicts by picking the most recent value
C. It tells the API to retry when the model isn't able to decide
D. It flags inconsistent source data, so downstream handling can route it

Q425. Data Extraction · Easy. How do you match batch results to their requests?

A. `custom_id`
B. `stop_reason`
C. Result order
D. Timestamps

Domain 5: Context Management and Reliability

Q426. Customer Support · Hard. A customer said "I need this by Friday" on turn 2, and by turn 30, the agent has forgotten it. Which design best prevents this?

A. Restart the conversation at turn 20, carrying over only the customer's name

B. Capture customer-stated expectations in the persistent facts block, as they're stated

C. Ask the customer at the end of each turn whether anything has changed for them

D. Increase the summary's length until it's as long as the conversation has become

Q427. Customer Support · Medium. Each `lookup_order` returns 40 fields, and only 5 are relevant. What should you do?

A. Call the tool less often, even though the agent needs more order information

B. Trim the results to the relevant fields before they accumulate in context

C. Ask the model to ignore the fields that aren't relevant to the current task

D. Keep them all, since a longer context always leads to better decisions

Q428. Developer Productivity · Hard. A multi-agent exploration crashed, and you must resume reliably. What's the guide's design?

A. Agents export state to a known location, and a manifest is loaded on resume

B. Ask the model to recollect from general knowledge what it was doing when the crash happened

C. Have every agent keep all of its state in memory, and hope that the crash doesn't recur

D. Restart everything from scratch, since state can't be recovered after any crash of any agent

Q429. Data Extraction · Medium. Reviewer capacity is limited. Which extractions should go to reviewers first?

A. Low-confidence fields, and documents with ambiguous or contradictory sources
B. The documents that have the largest file sizes, as they're the hardest to read
C. A uniform sample across all documents, with no reference to any risk indicators
D. The newest documents, regardless of confidence or any conflicts they contain

Q430. Customer Support · Medium. Which is best in a system prompt for escalation?

A. "Escalate whenever the customer seems unhappy, and try to avoid it whenever the customer seems content."
B. "Escalate only if your confidence falls below 70 out of 100 on the current customer's request."
C. "Escalate if the customer asks for a human, policy doesn't cover the request, or you can't progress. Examples: ..."
D. "Use your best judgment about when an escalation might be appropriate for the customer and the business."

Q431. Customer Support · Medium. After summarizing a long chat, the agent says "a partial refund was discussed," having lost "$47.50 on order 7781 by Friday." What's the best fix?

A. Ask the summarizer to be more careful about preserving numbers in its summaries
B. Keep transactional facts in a persistent "case facts" block, outside summarized history
C. Stop summarizing, and keep sending the whole transcript until the limit is reached
D. Have the customer repeat the key details once more every time they're needed

Q432. Multi-Agent Research · Medium. Why is returning "search unavailable" a problem?

A. It reveals sensitive internal information about the search provider being used
B. It hides what failed and what was tried, so the coordinator can't choose a recovery
C. It's too long for a coordinator that has only a small context budget available
D. It causes the coordinator to treat the search as having completed successfully

Q433. Multi-Agent Research · Hard. Why require subagents to include dates and source locations in structured output?

A. It makes JSON parsing faster, as dates are easier to validate in schemas
B. It's required by the API whenever a request contains a citation of any type
C. It lets the coordinator skip any evaluation of the subagent's reported findings
D. It supports accurate synthesis, including temporal interpretation and attribution

Q434. Multi-Agent Research · Medium. How do you mitigate position effects in a long aggregated input?

A. Put the most important material in the very middle of the input, where it's safest
B. Remove the headers, so that the model reads all the text with equal attention
C. Reverse the order of the sections on every other run, to average the effect
D. Put a key findings summary first, and organize details with explicit section headers

Q435. Customer Support · Hard. Select the two appropriate triggers. **Select 2.**

A. Negative sentiment detected in a message
B. Policy is ambiguous or silent on the request
C. The conversation has passed five messages
D. Self-reported confidence below a threshold
E. The customer explicitly asks for a human

Q436. Code Generation · Medium. You ask a question in a long session, and the answer ignores a key fact you found an hour ago. Why would a scratchpad have helped?

A. It would stop any further summarization from happening for that conversation
B. It would make the model's context window bigger for the rest of that session
C. It would keep the fact outside the conversation, to re-read later
D. It would send the fact directly to all of the agent's subsequent tool calls

Q437. Multi-Agent Research · Hard. A team terminates the whole research run whenever any subagent fails, "to protect quality." What's a better approach?

A. Retry the entire run from scratch until all subagents have succeeded at the same time
B. Keep terminating, because a partial report has less value than having no report at all
C. Continue with what's available, annotate gaps, and let the coordinator decide on recovery
D. Hide failures from the coordinator, so that the run is never interrupted by one

Q438. Multi-Agent Research · Medium. An access failure and a valid empty result look the same to the coordinator. What's the consequence?

A. It may report "nothing exists" when the search never ran, or retry pointlessly
B. Its synthesis will be faster, as it no longer has to process any results
C. It will always make the right decision, as both of them contain no results
D. The API detects the difference, and it raises an exception to the coordinator

Q439. Customer Support · Hard. The refund's within policy, the tool's available, and the customer is polite but says, "This seems complicated." What's the best action?

A. Escalate, as the customer described the case as complicated
B. Resolve it autonomously, as nothing triggers escalation
C. Offer to escalate, and wait for their reply first
D. Ask a human to check the refund before responding

Q440. Code Generation · Medium. What's the role of a scratchpad file in long exploration?

A. It increases the size of the context window for that specific session
B. It persists key findings across context boundaries, for later reference
C. It replaces the need for reading any source files during that exploration
D. It stores API credentials safely between the exploration's different phases

Q441. Multi-Agent Research · Hard. A synthesis agent merges findings and drops the excerpts "to save space." What's the cost?

A. Claims can no longer be verified against their sources
B. The merged findings will fail JSON schema validation
C. The report's numbers will automatically round to integers
D. The coordinator will refuse to accept the synthesized output

Q442. Customer Support · Medium. A billing subagent can't reach a payments service after retries. What should it return?

A. The failure, what it tried, any partial data, and what could be done instead
B. A note saying that everything's fine, along with a recommendation to wait
C. A success with an amount of zero, so that the conversation keeps moving
D. The full exception stack and connection string, so nothing has been hidden

Q443. Code Generation · Easy. What's a sign of context degradation in a long session?

A. The model starts to ask more questions about the requirements
B. A new tool gets discovered automatically at the next turn
C. Answers cite "typical patterns", not specific classes
D. The session's responses are always formatted in valid JSON

Q444. Multi-Agent Research · Medium. A 2019 figure and a 2024 figure look contradictory. What prevents this confusion?

A. Always preferring whichever figure is the larger of the two
B. Requiring publication or collection dates in structured outputs
C. Removing all of the older sources from every research result
D. Asking the synthesis agent to explain away all the differences

Q445. Multi-Agent Research · Hard. One of four subagents fails, with partial results. How should the final report handle it?

A. Leave out the failed area without mention, so that the report still reads completely
B. Fill the gap from the model's general knowledge, and present it the same way as the rest
C. Refuse to produce any report at all, until all four subagents have succeeded
D. Annotate coverage: what's well-supported, and where gaps exist

Q446. Multi-Agent Research · Easy. How is source attribution typically lost?

A. JSON schemas remove URLs when validating any of the fields
B. The model is forbidden from repeating any source's exact name
C. Summarization compresses findings without keeping claim-source mappings
D. Subagents aren't able to read URLs that come from search results

Q447. Developer Productivity · Medium. Before spawning subagents for phase two, what should the main agent do?

A. Nothing, since subagents inherit all of the main agent's phase one findings
B. Copy the full phase one transcript, including every tool result that was returned
C. Summarize the phase one findings, and inject them into their initial context
D. Delete the phase one findings, in order to give the next phase a clean start

Q448. Multi-Agent Research · Medium. Financial data, news and technical findings are all in one report. How should they be rendered?

A. All as tables, as tables are always more accurate
B. All as prose, for a uniform reading experience
C. Each appropriately: tables, prose, and structured lists
D. All as bullet points, since these are easiest to scan

Q449. Developer Productivity · Hard. What should a state manifest contain?

A. What each agent completed, where its output is, and what remains
B. The agents' API keys, so that they can reconnect themselves
C. A one-sentence statement that the work was progressing
D. All of the raw tool outputs from the whole run, inline

Q450. Customer Support · Hard. Why is sentiment-based escalation a poor proxy?

A. Sentiment doesn't measure complexity: angry customers may have simple issues, and calm ones complex ones
B. Sentiment analysis models are unavailable for use with most kinds of customer support conversations
C. It's only a poor proxy for written messages, though it's reliable with any voice transcripts
D. Sentiment-based routing increases costs, and that's the sole reason it isn't recommended here

Q451. Developer Productivity · Medium. You need "all test files" and "the refund flow's dependencies" while keeping the main session focused. What's best?

A. Copy all the repository's files into the main agent's prompt, before starting
B. Ask each question in a new unrelated session, and don't record the results
C. Run them both in the main session, as subagents can't read the repository
D. Spawn subagents for those, and keep coordination in the main agent

Q452. Data Extraction · Easy. Why can't you rely on "97% accuracy overall" before automating?

A. Accuracy can't be measured on structured extraction tasks
B. 97% is always too low for automation, whatever the task
C. It only measures latency, and not the quality of the work
D. It may hide poor performance on document types or fields

Q453. Multi-Agent Research · Hard. Select the two practices for handling conflicting statistics. **Select 2.**

A. Rewrite them as one blended approximate value
B. Include both, with their sources
C. Include their dates and methods, if known
D. Remove both, to avoid any disagreement
E. Select the value that's most recent, silently

Q454. Code Generation · Hard. Claude now gives contradictory answers about a class it examined earlier. What's the most effective response?

A. Use scratchpad notes or re-check it, and compact or delegate further exploration
B. Tell Claude in strong words that it needs to be more consistent from now on
C. Keep going, since inconsistency is an unavoidable and harmless side effect
D. Increase `max_tokens`, as more output room would fix any problem with memory

Q455. Multi-Agent Research · Easy. What does a coverage annotation tell the reader?

A. Which findings are well-supported, and where gaps exist
B. How many tokens each of the sections of the report took
C. Which developer has last modified the coordinator's prompt
D. Which model version was used for each of the findings

Q456. Multi-Agent Research · Medium. A long aggregated input has findings at its start, middle and end. The output omits those in the middle. What's this called?

A. Attention dilution across agents
B. The stale tool result problem
C. Progressive schema degradation
D. The "lost in the middle" effect

Q457. Data Extraction · Medium. How should field-level confidence thresholds be set?

A. Set each at 90%, as that's a typical threshold
B. Set them so that review volume is exactly zero
C. Use the model's scores as probabilities directly
D. Calibrate them using labelled validation sets

Q458. Multi-Agent Research · Medium. Two credible sources give different market sizes. What should the report do?

A. Omit the statistic, so that the report has no contradictions
B. Average the two values, and report only that single number
C. Pick the one from the source that has the better reputation
D. Present both with attribution, annotating the conflict

Q459. Customer Support · Easy. What's the best tool for improving when-to-escalate judgment?

A. A sentiment score on each new message
B. Few-shot examples with explicit criteria
C. A higher iteration limit for each case
D. A shorter system prompt for the agent

Q460. Multi-Agent Research · Medium. What must subagents output to preserve provenance through synthesis?

A. A confidence score on each claim, with no reference to any source
B. A bibliography that isn't connected to the individual claims in it
C. Structured claim-source mappings, with URLs, document names and excerpts
D. Long narrative summaries that mention sources in a general way

Q461. Data Extraction · Hard. Select the two steps before reducing human review. **Select 2.**

A. Analyze accuracy by document type and field
B. Check only the overall accuracy figure
C. Trust the model's scores as stated
D. Calibrate thresholds on a labelled validation set
E. Remove the audit sampling to save budget

Q462. Customer Support · Medium. A customer is frustrated about a late delivery, a case the agent can handle, and hasn't asked for a human. What's the best response?

A. Ignore their frustration, and proceed straight on to calling the appropriate order tools
B. Acknowledge it and offer to resolve, escalating only if they repeat the request
C. Ask them to rate how frustrated they are, and then escalate above a certain threshold
D. Escalate immediately, because frustration is one of the reliable signs of a complex case

Q463. Customer Support · Hard. An agent hands off routine warranty swaps, yet it attempts policy exceptions by itself. What's the best way to correct this?

A. Add explicit escalation criteria with few-shot examples of when to escalate or resolve
B. Remove the escalation tool, so that the agent is forced to resolve all of its cases
C. Ask the model for a confidence score, and escalate anything that scores below 80
D. Add sentiment analysis, and have it send all the unhappy customers to a human agent

Q464. Customer Support · Hard. Select the two things that belong in a persistent facts block. **Select 2.**

A. All 40 fields of each order record
B. The model's full reasoning from each turn
C. The customer's stated deadline
D. The refund amount and the order number
E. Every greeting from the customer

Q465. Multi-Agent Research · Medium. Which pair are both named as anti-patterns?

A. Returning partial results, and including the query that was attempted in the error
B. Reporting failure types, and suggesting some alternative approaches to the coordinator
C. Silently returning empty results, and terminating the whole workflow on one failure
D. Retrying a transient failure locally, and reporting unrecoverable errors upward

Q466. Customer Support · Medium. Policy covers own-site price adjustments, and the customer asks for a competitor price match. What should the agent do?

A. Apply the own-site policy by analogy
B. Escalate, as policy doesn't cover it
C. Offer half of the requested adjustment
D. Refuse, as it isn't listed in the policy

Q467. Customer Support · Medium. `get_customer` returns three "J. Smith" matches. What should the agent do?

A. Use all three, and merge the results
B. Pick the most recently active
C. Pick the one with the most orders
D. Ask for additional identifiers

Q468. Developer Productivity · Medium. Over a long run, tool results dominate context. Which is the best first step?

A. Reduce the number of tools that are defined overall
B. Ask the model to write shorter responses at each turn
C. Trim verbose results, keeping only the relevant fields
D. Move to a bigger model with a larger context window

Q469. Customer Support · Easy. What does progressive summarization risk losing?

A. Amounts, dates, and customer-stated expectations
B. Prior tool definitions and their input schemas
C. The API version and the model's identifier
D. The system prompt and the list of available tools

Q470. Multi-Agent Research · Hard. A coordinator receives "timeout on source A after 2 retries; 40% of results collected; source B may cover this." What can it now do?

A. Only retry source A, as coordinators can only retry failed operations
B. Nothing different from what it would do with a generic failure message
C. Decide whether to try source B, accept the partial results, or adjust the scope
D. Only terminate the run, since a failure message always ends the workflow

Q471. Data Extraction · Medium. A model reports 95% confidence on a field, and it's right 70% of the time in that band. What's the issue?

A. The band's sample must be flawed, as stated confidence can't be wrong
B. The model is working as intended, as confidence and accuracy are unrelated
C. The confidence is poorly calibrated, so thresholds need labelled data
D. The schema is at fault, since it should enforce a minimum confidence

Q472. Customer Support · Medium. What's wrong with escalating whenever the agent "feels unsure"?

A. The model can't express uncertainty in any of its text responses at all
B. Escalations cost more than resolved cases, and that alone is the sole issue
C. The API doesn't allow the escalation tool to be called with an unsure model
D. Self-reported confidence is an unreliable proxy for actual case complexity

Q473. Code Generation · Medium. Context fills with verbose discovery output during a long session. Which command helps?

A. `/compact`
B. `/resume`
C. `/memory`
D. `/init`

Q474. Customer Support · Medium. Which is an appropriate escalation trigger?

A. The customer's message uses negative sentiment
B. The model reports a confidence below 70%
C. The conversation has reached a tenth message
D. Policy is silent on the customer's specific request

Q475. Developer Productivity · Hard. Select the two techniques for extended codebase exploration. **Select 2.**

A. Scratchpad files recording key findings
B. Treating general patterns as equivalent to code
C. Avoiding any record of the findings so far
D. Reading every file before starting any work
E. Subagents for specific verbose questions

Q476. Data Extraction · Medium. What should you verify before removing human review for high-confidence extractions?

A. Accuracy by document type and field, across all segments
B. The model's stated confidence, averaged across all documents
C. That customer complaints have remained low in recent weeks
D. That the total volume of documents is above a set minimum

Q477. Data Extraction · Medium. How do you keep measuring error rates for automated, high-confidence extractions?

A. Take a stratified random sample for human review
B. Review only the low-confidence extractions
C. Wait until someone reports a specific error
D. Sample only the most common document type

Q478. Developer Productivity · Easy. What's the benefit of delegating verbose exploration to a subagent?

A. The subagent's result is always correct
B. The main agent gets all the raw output
C. The main agent's context stays focused
D. The API's price per token is reduced

Q479. Multi-Agent Research · Hard. Select the two anti-patterns. **Select 2.**

A. Returning empty results as success after a failure
B. Terminating the workflow on a single subagent failure
C. Annotating gaps in the final report
D. Separating access failures from valid empties
E. Local retries of transient failures

Q480. Code Generation · Hard. Select the two indicators that context management is needed. **Select 2.**

A. A short and focused new conversation
B. Consistent citations of specific classes
C. A clear stack trace for a one-file bug
D. Inconsistent answers about previously examined code
E. References to "typical patterns" rather than specifics

Q481. Multi-Agent Research · Easy. What's the best error report from a search subagent to its coordinator?

A. Failure type, attempted query, partial results and alternatives
B. "Search unavailable", to keep messages small and simple
C. A fatal error which stops the entire research workflow
D. An empty list marked as success, for workflow continuity

Q482. Multi-Agent Research · Medium. A search subagent's backend times out once. What should it do first?

A. Return an empty success, and let the synthesis agent proceed with it
B. Tell the coordinator at once, so that it can decide about a retry
C. Retry locally, propagating only if it can't resolve the failure
D. Stop the whole workflow, so that the report isn't left incomplete

Q483. Customer Support · Hard. Where should the case facts block go in each request?

A. Only in the first message, as the model will remember it for the whole conversation
B. In each prompt, outside the summarized history, so that it's always present and exact
C. In a tool's description, so that it's only loaded whenever that tool gets called
D. Inside the rolling summary, where it can be condensed along with everything else

Q484. Customer Support · Easy. Why send the complete conversation history in each API request?

A. To reset the rate limit counter that applies to the account
B. To enable automatic prompt caching on all of the old messages
C. To maintain coherence, as the model has only what's sent
D. To let the API remove the older messages on your behalf

Q485. Multi-Agent Research · Hard. Select the two fields that help a coordinator recover from a subagent failure. **Select 2.**

A. The subagent's total token usage
B. The current date on the server
C. The attempted query
D. The subagent's prompt word count
E. Partial results gathered before the failure

Q486. Data Extraction · Hard. Accuracy's 98% overall, 99.5% on invoices, and 60% on handwritten forms, which are 2% of volume. What's the right conclusion?

A. Automate everything, as the overall figure is above the 97% target
B. Don't automate handwritten forms yet, and route them to reviewers
C. Stop automating everything, as a 60% segment makes all the results invalid
D. Report only the invoices' figure in the team's published quality metrics

Q487. Data Extraction · Hard. A new vendor's invoice format appears. Audit samples show a new error pattern. What does this demonstrate?

A. The format is unsupported, and that vendor must change the way it invoices
B. The confidence calibration must have been correct, so no action is needed
C. Ongoing sampling of high-confidence output detects novel error patterns
D. High-confidence extractions never have to be audited after the launch

Q488. Customer Support · Hard. Two accounts match, and the customer gives a ZIP code that matches both. What's next?

A. Pick the account with the most recent activity on it
B. Proceed on both accounts, then let the customer sort it out
C. Apologize, and tell them that nothing can be done
D. Ask for another identifier, such as an order number or email

Q489. Customer Support · Medium. A multi-issue session mixes up two orders' details. What's the best structure?

A. A rule telling the agent to be careful not to mix the two orders' details up
B. A separate context layer with structured data per issue: order ID, amount, status
C. One combined summary paragraph, updated after every turn with what was discussed
D. A fresh session per message, so that one order's details can't touch another's

Q490. Customer Support · Hard. Select the two correct behaviors when tool results give multiple matching customers. **Select 2.**

A. Hold any account action until it's resolved
B. Choose the account with the highest value
C. Choose the most plausible by heuristic
D. Proceed using the first result returned
E. Ask for additional identifiers

Q491. Data Extraction · Hard. Select the two things that should go to human review. **Select 2.**

A. Every document, regardless of capacity
B. Low-confidence extractions
C. Documents with contradictory sources
D. Documents that have the fewest pages
E. All high-confidence invoices with no audit reason

Q492. Multi-Agent Research · Hard. Select the two mitigations for losing findings in a long aggregated input. **Select 2.**

A. A key findings summary placed at the start
B. Placing the most critical findings near the middle
C. Removing all headings and structure from the input
D. Explicit section headers organizing the details
E. Lengthening each section's supporting reasoning

Q493. Customer Support · Medium. Which escalation trigger shows "inability to make meaningful progress"?

A. The customer used a long sentence to explain the issue
B. The conversation took more than three minutes overall
C. The agent used a tool that it hadn't used earlier
D. Two approaches have failed, and no other tool applies

Q494. Data Extraction · Hard. Why stratify rather than sample uniformly?

A. Stratification removes the need for any human review of the samples
B. Uniform sampling can't be used on high-confidence extractions at all
C. Stratified samples are always cheaper, whatever the sample size is
D. Rare, risky segments get enough samples to measure their error rates

Q495. Multi-Agent Research · Hard. How should a document analysis agent handle conflicting values?

A. Choose the more plausible value by itself, and leave out the other one altogether
B. Stop, and return an error, as analyses can't continue when values are conflicting
C. Average the two conflicting values, and then record the source of only one of them
D. Complete the analysis with both values annotated, and let the coordinator decide

Q496. Customer Support · Easy. A customer writes, "Get me a human." What should the agent do?

A. Ask them to explain why they want a human
B. Escalate immediately, without first investigating
C. Offer a discount, in order to retain them
D. Investigate first, as the issue may be easy

Q497. Multi-Agent Research · Medium. A synthesis agent has a small context budget, and search agents return long reasoning. What's the best change?

A. Give the synthesis agent a larger context by way of a longer system prompt
B. Pass only the final sentence of each search agent's reasoning to the synthesis
C. Ask synthesis to skim through long inputs and skip what looks unimportant
D. Have upstream agents return structured facts, citations and relevance scores

Q498. Multi-Agent Research · Medium. A query returned zero matches, with no failure. How should it be reported?

A. As a valid empty result
B. As a permission error
C. As a workflow failure
D. As a transient error

Q499. Multi-Agent Research · Medium. How should the report's structure treat findings of differing certainty?

A. Write them all with equal confidence, for the sake of a consistent reading experience
B. Separate well-established findings from contested ones, keeping source characterizations
C. Mark everything as contested, in order to avoid making any claim that could be wrong
D. Include only the established ones, and delete anything that has been contested at all

Q500. Customer Support · Hard. A customer asks for a human, then adds, "but first, what's my order status?" What's the best behavior?

A. Refuse to answer anything, because the customer has already asked to speak to a person
B. Honor the human request promptly, and answer the quick factual question if it doesn't delay that
C. Treat the question as cancelling the request, and continue handling the case autonomously
D. Ask the customer to choose between the status answer and speaking with a human agent

Answer Key

Use this for quick marking. Explanations follow.

Q	Ans	Q	Ans	Q	Ans	Q	Ans	Q	Ans
1	C	2	A	3	B	4	B	5	C
6	B	7	B	8	C	9	C	10	A
11	C	12	C, E	13	A	14	B	15	D
16	D, E	17	D	18	C	19	B, E	20	D
21	D	22	A	23	D	24	A	25	A
26	A, E	27	A	28	A	29	B	30	D
31	C, D	32	A	33	A	34	B	35	C
36	D	37	D	38	B	39	A	40	A
41	B	42	B	43	C	44	C	45	B
46	D	47	A	48	D	49	D	50	D
51	B, D	52	D	53	A, E	54	A	55	B
56	C	57	B, C	58	B	59	B	60	B
61	B, D	62	B	63	B	64	B	65	D
66	D	67	C	68	B	69	C	70	A
71	A	72	C	73	D	74	D	75	B
76	B	77	A	78	C	79	C, E	80	B
81	C	82	A	83	D	84	A	85	A

Q	Ans	Q	Ans	Q	Ans	Q	Ans	Q	Ans
86	A	87	D	88	B	89	C	90	A
91	C	92	D	93	D	94	A, D	95	B, D
96	C	97	B	98	D	99	A	100	B
101	C	102	D	103	A	104	C	105	A
106	D	107	B	108	B	109	D	110	A
111	A	112	A	113	C	114	C, D	115	D
116	C	117	B	118	D	119	C	120	B
121	A, C	122	B	123	C	124	A	125	C
126	C	127	D	128	C	129	C	130	B
131	C	132	D	133	D	134	C	135	A
136	C	137	D	138	B	139	A	140	A, E
141	B	142	C	143	C	144	A	145	A
146	C	147	D	148	B, C	149	D	150	D
151	A	152	A	153	A	154	B, C	155	D
156	D	157	C, E	158	B	159	B	160	D
161	C	162	D	163	A	164	B, E	165	A
166	A, C	167	B	168	D	169	B	170	C

Q	Ans	Q	Ans	Q	Ans	Q	Ans	Q	Ans
171	A, C	172	A	173	D	174	D	175	A
176	B	177	C	178	D	179	C	180	B
181	D	182	A	183	B	184	C	185	B
186	C	187	A	188	B	189	B	190	C
191	B	192	D	193	B	194	C	195	C, D
196	B	197	C	198	B	199	C	200	A
201	C	202	A	203	B	204	D	205	B, C
206	B	207	A	208	B	209	A	210	D
211	A	212	D	213	B	214	C	215	C
216	A	217	D, E	218	D	219	D	220	C
221	D	222	C	223	A	224	A	225	C
226	C	227	B	228	D	229	A	230	D
231	B	232	C	233	C	234	A	235	D
236	A	237	B	238	D	239	B	240	A
241	D	242	B	243	A	244	D	245	A
246	C	247	C	248	C	249	B	250	D
251	D	252	C	253	D	254	C	255	B

Q	Ans	Q	Ans	Q	Ans	Q	Ans	Q	Ans
256	C	257	B, D	258	B	259	C	260	A
261	B	262	B	263	D	264	B	265	B
266	C	267	D	268	B	269	A	270	D, E
271	B	272	B	273	B, D	274	C	275	D
276	A	277	B	278	D	279	C	280	B
281	A, C	282	C	283	D	284	A	285	B, E
286	A, B	287	A	288	C	289	A	290	B
291	C	292	B	293	D	294	D	295	C
296	A	297	C	298	A, C	299	B	300	D, E
301	A	302	B	303	A	304	B	305	A
306	D	307	B, E	308	C	309	C, D	310	D
311	D	312	D	313	B	314	A	315	A
316	A, B	317	A	318	D	319	A	320	C
321	A	322	C	323	A	324	C	325	D
326	A, C	327	A	328	C	329	C	330	D
331	B	332	B	333	B	334	A	335	B
336	C	337	D	338	B	339	B	340	A, B

Q	Ans	Q	Ans	Q	Ans	Q	Ans	Q	Ans
341	D	342	C	343	C	344	A, B	345	C, D
346	A	347	C	348	C	349	B	350	A
351	D	352	D	353	B	354	B	355	A
356	D	357	D	358	C	359	C	360	B
361	B	362	B	363	C	364	A, D	365	C
366	C	367	A	368	B	369	A	370	B
371	C	372	B	373	A, B	374	A	375	B, E
376	A, D	377	B	378	A	379	D	380	D, E
381	A	382	B	383	D	384	A	385	D
386	A	387	A	388	C	389	D	390	B, D
391	C	392	D	393	B	394	A	395	A
396	D	397	D	398	D	399	D	400	B
401	C	402	A	403	D	404	A	405	C
406	A	407	B	408	C	409	D	410	D
411	D	412	A	413	B	414	C	415	C, E
416	A	417	A	418	D	419	C	420	D
421	C	422	B	423	C	424	D	425	A

Q	Ans	Q	Ans	Q	Ans	Q	Ans	Q	Ans
426	B	427	B	428	A	429	A	430	C
431	B	432	B	433	D	434	D	435	B, E
436	C	437	C	438	A	439	B	440	B
441	A	442	A	443	C	444	B	445	D
446	C	447	C	448	C	449	A	450	A
451	D	452	D	453	B, C	454	A	455	A
456	D	457	D	458	D	459	B	460	C
461	A, D	462	B	463	A	464	C, D	465	C
466	B	467	D	468	C	469	A	470	C
471	C	472	D	473	A	474	D	475	A, E
476	A	477	A	478	C	479	A, B	480	D, E
481	A	482	C	483	B	484	C	485	C, E
486	B	487	C	488	D	489	B	490	A, E
491	B, C	492	A, D	493	D	494	D	495	D
496	B	497	D	498	A	499	B	500	B

Answers: Domain 1

1. C (TS 1.7). Session context doesn't track the repo by itself. The others aren't behaviors.

2. A (TS 1.2). That's an iterative refinement loop: evaluate gaps, target more research, and resynthesize until coverage is sufficient. Accepting leaves the gap, general knowledge loses citations, and a full restart repeats the same narrow work.

3. B (TS 1.7). Targeted re-analysis avoids exhausting context. The others don't address it.

4. B (TS 1.2). Subagents run with isolated context. Whatever they need has to go in their prompt.

5. C (TS 1.2). Distinct assignments minimize duplication. The others invite overlap.

6. B (TS 1.4). Only a code-level gate guarantees the sequence. The others depend on behavior.

7. B (TS 1.5). A guaranteed rule needs interception. Examples and descriptions are probabilistic, and an audit is after the fact.

8. C (TS 1.2). Narrow subtasks miss parts of a broad question. The other options aren't consequences of decomposition width.

9. C (TS 1.6). Separate passes avoid attention dilution while still catching integration issues. A bigger model doesn't fix attention, and sampling files misses bugs.

10. A (TS 1.4). A proper summary prevents repetition. The other three are required parts of a good handoff.

11. C (TS 1.4). A clear message lets it take the right next step. Silence stalls, fake success misleads, and a generic error prevents recovery.

12. C and E (TS 1.6). Dependence on findings and unknown scope call for adaptivity. Schemas and repeated steps don't, and tight latency argues against it.

13. A (TS 1.3). Task is the spawning mechanism, and it must be allowed for the coordinator. `stop_reason` isn't configured, and the others aren't prerequisites. Build today: Claude Code docs name this tool Agent, with `Task` kept as an alias. The exam guide uses Task.

14. B (TS 1.6). Choose the simplest pattern that fits. Adaptivity adds cost and variance with no benefit for fixed steps.

15. D (TS 1.4). The gate relies on actual tool results, in code. Asking the model, reading customer text (which anyone can type), or waiting doesn't prove verification.

16. D and E (TS 1.7). Inform and target. Don't assume detection or re-explore everything.

17. D (TS 1.7). This combines the guide's tools according to how valid the context is. The extremes lose reliability or continuity.

18. C (TS 1.1). Tool results reach the model only through history. If they aren't appended, it can't incorporate them. There's no automatic data sync, no `tool_error` stop reason in this flow, and more iterations wouldn't restore lost data.

19. B and E (TS 1.4). Those two run in code. The other three influence the model, which is probabilistic.

20. D (TS 1.5). Interception is enforcement. The others are guidance. Build today: Claude Code's docs also advise OS-level sandboxing, since text matching can be bypassed by other forms of a command.

21. D (TS 1.2). Refinement continues until coverage is sufficient. Stopping at the first draft skips evaluation, a fixed count ignores quality, and a subagent's preference isn't the criterion.

22. A (TS 1.3). Claim-level metadata preserves attribution. Prose and bibliographies don't map claims to sources, and self-reported confidence doesn't help trace them.

23. D (TS 1.5). Normalizing in code removes ambiguity before reasoning begins. A prompt table is less reliable, and escalation is disproportionate.

24. A (TS 1.4). High-stakes requirements can't tolerate any failures. Tone and suggestions can.

25. A (TS 1.3). Tool restrictions in `AgentDefinition` prevent misuse. Prompt-only limits are probabilistic, and the other two ignore the cause.

26. A and E (TS 1.5). Interception acts on outgoing calls to enforce rules. Tone and ambiguity are judgment, and normalizing results is `PostToolUse`.

27. A (TS 1.1). A response can contain text and `tool_use` blocks together, such as "Let me check that file." Checking for text is a named anti-pattern. Only `stop_reason` reliably signals completion. The other options describe behavior that doesn't exist.

28. A (TS 1.4). Ordering that must always hold belongs in code. Better models and longer prompts lower but don't remove risk, and a nightly reversal comes after the money has moved.

29. B (TS 1.3). Explicit context passing is the rule. There's no inheritance or standard mid-run channel for this, and prompts can carry whatever's needed.

30. D (TS 1.1). Returning the failure as a tool result lets the model adapt. Stopping wastes the session, omitting a result breaks the `tool_use` / `tool_result` pairing, and infinite retries hang the system.

31. C and D (TS 1.1). Text can accompany tool calls, and wording isn't fixed. Pricing, streaming and encryption claims here are false.

32. A (TS 1.1). An arbitrary cap as the primary stop cuts off legitimate work. A cap is fine as a safeguard against runaway loops, but the model's `stop_reason` should end normal runs. A bigger cap keeps the same flaw, and removing history breaks reasoning.

33. A (TS 1.1). Ambiguity suits model-driven reasoning, and a hard rule suits code. Trees can't cover open-ended requests, a prompt alone has a non-zero failure rate, and fixed per-type agents lose flexibility.

34. B (TS 1.5). `PostToolUse` runs after a tool returns. The others aren't hook types named by the guide.

35. C (TS 1.5). Deterministic normalization before reasoning is the pattern. The others are unreliable or wasteful.

36. D (TS 1.7). Stale results lead to wrong reasoning, without any error. The others don't occur.

37. D (TS 1.2). Central routing provides observability. Hidden notes and transcripts in the report aren't part of the pattern, and a subagent retains no memory between invocations.

38. B (TS 1.5). It reduces accumulated tokens from verbose results (TS 5.1). It doesn't change call rates, it does change what the model sees, and it acts on outputs rather than inputs.

39. A (TS 1.7). Old reads remain in history unless they're refreshed. Tell it what changed, or start fresh with a summary.

40. A (TS 1.6). It's an open-ended task, so map first, prioritize by impact, and adapt as dependencies appear. The others ignore importance.

41. B (TS 1.5). The redirection still needs a useful handoff (TS 1.4). The others leave the human without enough to act on.

42. B (TS 1.6). Open-ended tasks need adaptive plans. The other options don't address that.

43. C (TS 1.6). Use each pattern where it fits. Full dynamism adds cost for common cases, and the extremes either waste steps or lose automation.

44. C (TS 1.4). Multi-concern requests are decomposed, investigated (in parallel where possible), and synthesized. The other options leave issues unresolved or escalate unnecessarily.

45. B (TS 1.3). Forking gives independent branches from one baseline. A single session mixes them, blank sessions lose the analysis, and a shared writable file couples them.

46. D (TS 1.7). When prior context is mostly valid, resume and inform. Fresh starts discard useful work here.

47. A (TS 1.1). The anti-pattern is an arbitrary cap as the primary stop. A high limit that guards against runaway behavior is sensible. Lowering it would cut off real work, and the API doesn't impose this limit.

48. D (TS 1.2). Parallelism only adds coverage when work is partitioned. Sharing history wouldn't fix the assignments, dedup comes too late, and more identical agents make more duplicates.

49. D (TS 1.6). Integration issues appear only across files. The others are false or wasteful.

50. D (TS 1.1). Parsing natural language to decide termination is an anti-pattern, since wording isn't guaranteed. String checks don't cost tokens, don't affect history, and have nothing to do with streaming.

51. B and D (TS 1.2). If the assignments omit a topic and the work assigned is done well, the fault lies in the decomposition. Formatting and tool blips don't explain a missing subtopic.

52. D (TS 1.7). Validity of context drives the choice between resuming and starting fresh with a summary.

53. A and E (TS 1.6). Fixed, predictable sequences fit chaining. The other three are open-ended.

54. A (TS 1.1). That's the lifecycle: request, check `stop_reason`, run requested tools, add results, repeat. The model requests tools, so your code doesn't choose them, and no webhook is involved in the standard loop.

55. B (TS 1.3). The subagent has no shared history, so references to earlier work are empty. Findings must be included. The other reasons aren't real requirements.

56. C (TS 1.5). A hook is code, so test it, and make unexpected input visible. The others are slow, or remove the safeguard.

57. B and C (TS 1.3). Spawning needs Task allowed, and the subagent needs explicit context. The rest aren't requirements for spawning.

58. B (TS 1.6). Adaptive plans incorporate discoveries. Sticking blindly or restarting wastes the finding.

59. B (TS 1.1). `tool_use` means the model is waiting for results. They're appended to the conversation so it can reason about its next step. Ending there drops the work, a fresh conversation loses context, and asking for confirmation adds a pointless round trip.

60. B (TS 1.6). Consistency comes from explicit criteria, and reconciliation from an integration pass. Merging back reintroduces dilution, and a vague request doesn't help.

61. B and D (TS 1.7). Staleness favors a fresh start. Mostly valid context favors resuming.

62. B (TS 1.7). With mostly stale results, a fresh session with injected facts is more reliable. Compaction doesn't refresh data, and forks inherit the same stale results.

63. B (TS 1.7). A summary carries the conclusions without stale data. A raw paste brings stale data back, one sentence carries nothing, and a name doesn't transfer context to a fresh session.

64. B (TS 1.3). Keep what downstream needs: facts, citations, metadata. Remove reasoning chains and noise. The extremes either lose data or waste context.

65. D (TS 1.6). Fixed sequences suit predictable work. Open-ended work suits dynamic decomposition, and the other options aren't the criterion.

66. D (TS 1.5). Judgment on free text suits the model. The other three are clear rules in code.

67. C (TS 1.5). That's the deciding criterion. The others are false.

68. B (TS 1.2). Coordinators own error handling and routing. Peer signaling breaks the pattern, silent handling hides failures, and bypassing exposes raw errors.

69. C (TS 1.4). Shared facts prevent re-asking and contradictions. They don't merge tool calls, replace verification, or remove an issue.

70. A (TS 1.1). A tool-only turn has no text but isn't finished. Checking for text in either direction is an anti-pattern. Empty content isn't automatically an exception or a refusal.

71. A (TS 1.5). Central, deterministic normalization scales with tools. The others are unreliable or disproportionate.

72. C (TS 1.3). Multiple Task calls in a single response run in parallel. Separate turns run them sequentially, that flag doesn't exist, and multiple coordinators lose central control.

73. D (TS 1.3). Complete prior findings go in the prompt. Subagents can't query another's history, a one-liner loses details, and memory isn't shared.

74. D (TS 1.2). New capabilities plug in through the hub. Hidden spawning and subscriptions bypass controlled flow, and swapping the hub creates confusion.

75. B (TS 1.3). Structured formats keep content and metadata like source URLs, document names and page numbers distinct. Prompts accept text too, the model still reads it, and structure doesn't verify truth.

76. B (TS 1.3). Structured facts with metadata support accurate synthesis. Narratives dilute context, pointers don't work across isolated contexts, and interpretation alone loses the originals.

77. A (TS 1.1). Each request must carry the full conversation for continuity. Without earlier turns, the model has no record of what it did. There's no server-side conversation memory here, and no special restart mode.

78. C (TS 1.6). It describes one pass spreading itself too thin. The rest are different issues.

79. C and E (TS 1.5). `PostToolUse` transforms results after execution. Blocking before execution is interception, and the others aren't its role.

80. B (TS 1.2). Coordinators should analyze requirements and pick agents dynamically. A fixed order isn't dynamic, and peer or self-selection removes central control.

81. C (TS 1.5). This is the central distinction. The others are false.

82. A (TS 1.4). Recipients often lack the conversation. A summary gives them what they need to act.

83. D (TS 1.4). Hard rules go in code. The others are guidance, with a non-zero failure rate.

84. A (TS 1.4). Handle what's possible, and hand off the rest clearly. Escalating everything wastes capacity, pressing on risks harm, and closing abandons the third.

85. A (TS 1.2). Assigning distinct subtopics or source types prevents overlap at the source, which is the coordinator's job. Direct messaging breaks hub-and-spoke, a dedupe service treats a symptom, and one agent loses parallel coverage.

86. A (TS 1.7). Forking reuses the baseline and keeps branches independent. Alternating mixes them, and repeating analysis wastes work and may diverge.

87. D (TS 1.4). Specific and actionable, with amount, order and reason. The others force the human to reinvestigate.

88. B (TS 1.6). Incremental exploration builds understanding. Reading everything exhausts context, general knowledge isn't this code, and a fixed list can't adapt.

89. C (TS 1.5). Normalization in code is deterministic and applies to every result. The model doing conversions is error-prone, and a subagent over-engineers it.

90. A (TS 1.3). Definitions include descriptions, system prompts and tool restrictions.

91. C (TS 1.4). That's the core distinction in TS 1.4. Prompts aren't a guarantee, and removing automation goes further than the requirement needs.

92. D (TS 1.5). The prompt reduces wasted attempts and helps communication. Only the hook is a guarantee. Nothing automatically informs the model, and both together cause no failure.

93. D (TS 1.4). When compliance must be guaranteed, probabilistic instructions fall short. No API threshold exists, and repetition still leaves a failure rate.

94. A and D (TS 1.3). Forks are for divergent exploration from a shared baseline. They aren't a context-passing, capacity or determinism mechanism.

95. B and D (TS 1.1). Those two are the core of the loop. Text isn't a completion signal, a fixed count is an anti-pattern as the primary stop, and discarding turns removes what the model needs for reasoning.

96. C (TS 1.1). `stop_reason` is the API's explicit signal: continue on `tool_use`, stop on `end_turn`. Parsing words is unreliable, text can accompany tool calls, and a fixed count ignores what the task needs.

97. B (TS 1.4). Gates cover specific tools, so new money-moving tools need them too. The others don't enforce anything.

98. D (TS 1.1). Every `tool_use` needs a corresponding `tool_result`, matched by ID, and they go back together. Dropping one breaks the exchange, separate model calls add turns, and results don't belong in the system prompt.

99. A (TS 1.4). Judgment-style behavior suits prompts, and guaranteed rules suit code.

100. B (TS 1.7). Forks are independent conversation branches. Note: their conversations are independent, but a shared working directory's files are not, so isolate files too if needed.

101. C (TS 1.3). It states a goal and quality criteria for the output. Procedures are brittle, and a vague brief gives no criteria.

102. D (TS 1.7). Names let you continue a particular conversation. They don't do the others.

103. A (TS 1.3). Goals and criteria enable adaptation, while procedures break when reality differs. More steps or forced tools make it more rigid.

104. C (TS 1.6). The steps are predictable and fixed, so a chain is simple and appropriate.

105. A (TS 1.2). All inter-subagent communication goes through the coordinator, for observability, consistent error handling and controlled flow. Subagents have isolated contexts, so neither shared memory nor inherited history applies by default.

106. D (TS 1.4). Customer text can influence a prompt-based rule, and a gate can't be talked around. The others lower probability or react later.

107. B (TS 1.6). The plan doesn't adapt to findings. The others aren't causes.

108. B (TS 1.2). The subagents did what they were asked. Overly narrow decomposition is the coordinator's failure. Fixing downstream agents treats a symptom, and synthesis shouldn't invent unresearched content.

109. D (TS 1.1). Parsing natural language is the anti-pattern. The other three describe correct loop behavior.

110. A (TS 1.4). Programmatic enforcement is deterministic. Prompts and examples have non-zero failure rates, and a report acts after the fact.

111. A (TS 1.1). Fixed pipelines suit predictable work, while this task varies. Scripts can call tools, `end_turn` isn't related, and sequences don't inherently exceed limits.

112. A (TS 1.3). Those are the configured parts. History isn't inherited, billing isn't part of it, and there's no such shared memory.

113. C (TS 1.2). Hub-and-spoke buys observability and control, at a real latency cost. Addressing a high-frequency need with scoped tools is the proportionate response. The other claims are false.

114. C and D (TS 1.4). Those are actionable. Reasoning dumps, token counts and tool lists don't help a human resolve the case.

115. D (TS 1.3). Claim-source mappings survive only if they're carried and merged explicitly. A general summary or log link doesn't map claims to sources.

116. C (TS 1.3). The guide names the Task tool. The others aren't the spawning tool it describes.

117. B (TS 1.1). The agentic loop lets Claude reason about what to call next using new information. Decision trees pre-define paths, which helps with fixed rules but handles ambiguity poorly. Good descriptions don't make these the same, and token use doesn't settle accuracy.

118. D (TS 1.7). Forking serves divergent exploration. The others are single-path tasks.

119. C (TS 1.7). The guide names `--resume <session-name>`. The others aren't its options.

120. B (TS 1.1). Tool results are returned as `tool_result` blocks in the conversation history, tied to the `tool_use` block. The system prompt isn't for this, no registration call exists, and results aren't written into the assistant's turn.

121. A and C (TS 1.2). Isolation protects the main context and keeps focus. It doesn't give shared memory, it increases the need for explicit context, and it doesn't make outputs deterministic.

122. B (TS 1.6). Adaptive plans generate subtasks from findings. Fixed, fully specified steps fit chaining.

123. C (TS 1.3). There's no automatic shared memory. Context has to be passed explicitly.

124. A (TS 1.2). Information should flow through the coordinator, explicitly. Merging histories breaks isolation, whole transcripts waste context, and that shared-memory flag isn't a feature.

125. C (TS 1.3). Subagents receive only what's explicitly passed. Complete findings with their sources must be in the prompt. Nothing here depends on prompt length, self-found sources or expiring shared memory.

126. C (TS 1.2). Those are the stated reasons. It doesn't remove prompts or tools, and it's an architectural choice rather than an API restriction.

127. D (TS 1.1). `end_turn` is the normal completion signal. Token exhaustion has its own stop reason, tool failures come back as tool results, and history size isn't reported this way.

128. C (TS 1.2). Coordinators should select subagents dynamically according to complexity. The other options leave the wasteful pipeline in place.

129. C (TS 1.5). Blocking with a clear redirection lets the agent continue properly. Fake success misleads, crashing loses the session, and changing amounts silently breaks policy.

130. B (TS 1.2). Coordination covers decomposition, delegation, aggregation, and deciding which subagents to invoke. Doing all searches itself defeats delegation, and memory isn't shared automatically.

131. C (TS 1.3). Parallel spawning requires multiple calls in a single response. Separate turns wait for each result.

132. D (TS 1.5). Hooks run in your application, outside the model, which is what makes them deterministic.

133. D (TS 1.4). A structured summary gives the essentials to someone who can't see the transcript. A raw dump is slow to read, a vague note forces repeated questions, and a score doesn't help them act.

134. C (TS 1.7). Informing it of specific changes allows targeted re-analysis. Don't assume detection, full re-exploration wastes context, and deletion loses valid work.

135. A (TS 1.7). Forking creates divergent branches for exploration. It isn't merging, sharing or compaction.

Answers: Domain 2

136. C (TS 2.2). Timeouts are transient. The others won't succeed if retried unchanged.

137. D (TS 2.1). It gives a specific name, scope, outputs, and boundary. The others are too general, or point to documents the model can't see.

138. B (TS 2.1). Rescoping tackles the overlap itself. The others don't change it.

139. A (TS 2.4). Environment variable expansion keeps secrets out of version control. Private repos and encoding still commit a secret, and CLAUDE.md is committed text.

140. A and E (TS 2.1). Prompt keywords and overlapping tools are known causes. The others aren't related.

141. B (TS 2.1). Overlap is the root cause, so differentiate them, for example `extract_web_results` with a web-specific description. A classifier over-engineers it, merging creates a vague tool, and "think harder" doesn't remove ambiguity.

142. C (TS 2.4). Tools from all configured servers are discovered at connection and available together.

143. C (TS 2.3). Limited cross-role tools serve high-frequency needs. Over-granting or merging loses scoping, and a strict round trip adds cost for a common case.

144. A (TS 2.2). A policy violation is a business error. The others are transient, validation, and permission or authentication failures.

145. A (TS 2.5). Focused searches limit context. Glob doesn't search contents, and the others don't reduce what's consumed.

146. C (TS 2.4). Build custom servers for team-specific workflows that nothing else covers. Standard integrations should reuse existing servers, and small naming differences or learning goals don't justify one.

147. D (TS 2.3). Agents misuse tools outside their specialization, so scoping fixes this. Prompt rules are weaker, more tools make it worse, and forcing every turn removes flexibility.

148. B and C (TS 2.2). Hiding failures and uniform errors are the anti-patterns. The other three are recommended.

149. D (TS 2.4). Tools from all servers are available at once, and more tools increase decision complexity (TS 2.3). The others aren't facts.

150. D (TS 2.1). The keyword-sensitive instruction is the cause. The others build around it and widen the wrong tool's scope.

151. A (TS 2.3). `any` guarantees structured output, and lets the model pick the matching schema. `auto` may return text, forcing one type mislabels the others, and parsing a word adds fragility.

152. A (TS 2.2). It's an access failure, and retrying doesn't help. Which details to expose should be considered, but the agent has to know the call didn't succeed.

153. A (TS 2.4). User scope keeps it personal. The others affect the team, or don't configure servers.

154. B and C (TS 2.1). Clear formats and boundaries are the proportionate fix. A classifier over-engineers it, while shared names and shorter text add ambiguity.

155. D (TS 2.1). Purpose-specific tools with defined contracts are clearer to select and validate. A free-form mode keeps the ambiguity, more words on a generic tool don't fix its scope, and a subagent adds layers.

156. D (TS 2.4). All servers' tools are available together, so selection depends on descriptions (TS 2.1). The agent can use several servers at once, and scope doesn't set a priority.

157. C and E (TS 2.2). Category and retryability guide recovery. The others don't.

158. B (TS 2.4). Project scope is shared. Try it at user scope first. The others are false.

159. B (TS 2.3). Forced selection guarantees the first call. `any` doesn't guarantee which tool, a description is only guidance, and the last reverses the order. Build today: forced selection is rejected on some current models, so check the docs.

160. D (TS 2.3). A read-only exploration agent has only what its role requires. Giving every tool, with or without a warning, isn't scoping.

161. C (TS 2.2). Partial results are valuable if the gap is visible. Discarding them wastes work, silence hides the gap, and a full rerun is disproportionate.

162. D (TS 2.4). Resources expose content catalogs. Actions are tools, and credentials shouldn't be exposed.

163. A (TS 2.3). `auto` leaves it open. `any` requires some tool, and forced selection names one. Build today: forced modes return 400 on Opus 5.5 and Sonnet 5.5. See the docs.

164. B and E (TS 2.4). Team goes in project scope and personal in user scope. Secret values aren't committed.

165. A (TS 2.3). Constraints enforce role boundaries. The others are false.

166. A and C (TS 2.3). Those limit what's possible. The others widen access, or don't relate.

167. B (TS 2.4). Project-scoped `.mcp.json` is shared through version control. User config is personal, and the others aren't server configuration.

168. D (TS 2.5). Search content, then read, then follow. The others waste context, or guess.

169. B (TS 2.5). Grep searches file contents, so it finds calls. Glob matches paths, while Write and Edit change files.

170. C (TS 2.1). Specific formats reduce wrong calls. Identical names and removed detail add ambiguity, and retrying wrong IDs wastes calls.

171. A and C (TS 2.3). `any` and forced selection guarantee a call. `auto` doesn't, and text is only guidance. Build today: both forced forms are rejected on some current models.

172. A (TS 2.2). Policy violations are business errors. Retrying can't help. Changing the date would be wrong, and a false success misleads.

173. D (TS 2.5). Write replaces full contents. Edit makes targeted changes.

174. D (TS 2.5). Read + Write is the reliable fallback. Retrying can't make the text unique, and Grep and Glob don't modify files. Adding surrounding context to make the match unique also works in practice.

175. A (TS 2.2). `isError` is the MCP pattern. Empty text looks like success, that stop reason doesn't exist, and omitting results breaks the exchange.

176. B (TS 2.1). Few-shot examples handle remaining ambiguity (TS 4.2). Removing descriptions loses the main signal, a checker model is over-engineering at this stage, and renaming again doesn't address this class of queries.

177. C (TS 2.5). Tracing across wrappers needs each alias. Searching only the original name misses callers, Glob doesn't read content, and wrappers don't contain all callers.

178. D (TS 2.5). Glob matches path patterns. Grep searches inside files.

179. C (TS 2.1). These help selection and correct use. Internal details and generic reminders don't.

180. B (TS 2.2). Distinguishing them drives different recovery. Uniform treatment prevents it.

181. D (TS 2.2). Local recovery for transient errors reduces noise, and unresolved errors go up with context. Always propagating adds load, swallowing hides problems, and terminating over-reacts.

182. A (TS 2.2). A valid empty result is information. Marking it an error causes pointless retries or escalation.

183. B (TS 2.2). Catch errors and return safe, categorized messages. The others aren't what happens. This is a real risk found in framework labs in this course.

184. C (TS 2.4). The value comes from the developer's environment. Committing is what you're avoiding, and the others aren't how it works.

185. B (TS 2.3). Tool scoping is enforcement. A tool that isn't available can't be called.

186. C (TS 2.2). Specific errors let it correct the input. The others prolong the loop.

187. A (TS 2.3). Sets of about 4–5 role-relevant tools keep selection reliable. Fifteen-plus degrades it, one tool is too few for most roles, and an entire catalogue is worse still.

188. B (TS 2.2). It gives the agent accurate words to use. The API doesn't display it, it isn't required by validation, and politeness doesn't change policy.

189. B (TS 2.2). It's metadata that guides recovery. It doesn't restrict other behavior.

190. C (TS 2.4). It explains capabilities, outputs, and when to prefer it. The others make claims or give irrelevant details.

191. B (TS 2.5). Edit targets unique text. Write replaces the whole file, and the others don't change files.

192. D (TS 2.1). Boundaries with similar tools are essential. The others don't help in selecting it.

193. B (TS 2.1). Distinct contracts justify a split. Too many tools can hurt (TS 2.3), and the other claims are false.

194. C (TS 2.3). Too many tools degrade selection. The others aren't real limits.

195. C and D (TS 2.5). That's the guide's fallback. The others don't perform the change reliably.

196. B (TS 2.2). Uniform errors prevent appropriate recovery. They aren't rejected, don't force escalation, and don't change `stop_reason`.

197. C (TS 2.4). Resources give visibility into what's available, with no exploratory calls.

198. B (TS 2.1). It explains limits, what empty means, and a boundary. The others mislead or give no information.

199. C (TS 2.1). Prompt wording can create unintended tool associations. Reviewing the prompt comes before changing the tools. The other options aren't indicated by the evidence.

200. A (TS 2.3). Out-of-role tools invite misuse and worsen selection. The others aren't how tools work.

201. C (TS 2.2). Invalid input is a validation error, which the agent can fix and retry. Nothing here is a timeout, an access problem or a policy rule.

202. A (TS 2.4). It's the typical diagnosis of a scoping mistake: personal config isn't shared. The others are false.

203. B (TS 2.2). Hiding an access failure as empty results misleads the agent, and it may state wrong conclusions. There's no automatic detection or conversion.

204. D (TS 2.3). A limited cross-role tool handles the high-frequency need while keeping complex cases central. Full search invites misuse, keeping everything adds latency, and batching delays verification.

205. B and C (TS 2.5). Contents go to Grep, paths to Glob, and modifications to Edit or Write.

206. B (TS 2.5). Edit needs a unique match, and more context provides one. A shorter string makes the problem worse. The guide's named fallback remains Read + Write.

207. A (TS 2.3). `any` requires a tool while leaving the choice open. Build today: `any` returns 400 on Opus 5.5, Sonnet 5.5, Fable 5.1 and Mythos 5.1. Use `auto` with instructions and strict tools, or structured outputs.

208. B (TS 2.1). With little to distinguish them, the model chooses unreliably. The others aren't consequences.

209. A (TS 2.1). Knowing the outputs helps the model choose. The API doesn't validate against prose, and predicting results is what you don't want.

210. D (TS 2.1). Test selection on ambiguous requests, as the guide's preparation section advises. Word count and opinions don't measure behavior.

211. A (TS 2.2). This is the current spec's position, per the MCP tools docs. The guide teaches the metadata pattern. On the exam, follow the guide's usage of these fields.

212. D (TS 2.1). It gives format, example, purpose, and a boundary against a similar tool. The others are vague.

213. B (TS 2.3). Constrained tools enforce scope in code. A note is guidance only, and the other two don't reduce what can be fetched.

214. C (TS 2.2). Structured context enables recovery. A generic note hides details, silent partials mislead, and raw traces leak internals with little value.

215. C (TS 2.2). It's transient, and worth retrying. Empty success hides the failure, and bare "Error" gives nothing to reason with, while the validation category misclassifies it, and a stack trace may leak internals.

216. A (TS 2.5). It's a path pattern. Grep looks in contents, and the others aren't for finding files.

217. D and E (TS 2.4). Reuse servers and protect secrets. The rest add work or risk.

218. D (TS 2.4). Catalog visibility reduces exploratory calls. The others are false.

219. D (TS 2.4). Use existing servers for standard integrations, and reserve custom ones for team-specific workflows. The others are fragile, or waste effort.

220. C (TS 2.5). Incremental exploration keeps context useful. The others are false.

221. D (TS 2.3). Forcing is for the specific step, with later steps processed in follow-up turns. The others aren't requirements.

222. C (TS 2.4). Better descriptions correct the preference. Removing Grep hurts other tasks, a generic line is weak, and a name clash causes confusion.

223. A (TS 2.5). Build understanding incrementally. Reading everything exhausts context, general knowledge isn't this code, and file names alone aren't enough.

224. A (TS 2.1). Improve the descriptions first. Deleting loses capability, examples for every case aren't proportionate, and keyword-based forcing is brittle.

225. C (TS 2.1). Descriptions are the main selection signal. Order, name length and recency aren't how selection works, though clear names do help.

Answers: Domain 3

226. C (TS 3.5). Interacting problems are best handled together. Independent ones can go sequentially.

227. B (TS 3.1). User-level memory is personal and isn't in version control. The others are project or directory level.

228. D (TS 3.2). Isolation stops exploratory content affecting later work. "Forget" isn't reliable, the others don't isolate, and CLAUDE.md makes it worse.

229. A (TS 3.4). Match the mode to complexity. The extremes waste time, or lose planning when it's needed.

230. D (TS 3.5). Concrete examples remove interpretation, which prose leaves open. Longer or repeated paragraphs keep the same ambiguity, and similar projects may use different formats.

231. B (TS 3.6). `-p` is short for `--print`, the non-interactive mode for pipelines. It has nothing to do with planning, parallelism or a pipeline option.

232. C (TS 3.6). Non-interactive input fits. The others are manual or pollute the repo.

233. C (TS 3.1). Rules live in `.claude/rules/`. Skills and commands are invoked workflows.

234. A (TS 3.4). Combining them is recommended: plan to investigate, then execute.

235. D (TS 3.2). Task-specific, on-demand workflows suit skills. The others always apply.

236. A (TS 3.2). On-demand loading saves context. The others are false.

237. B (TS 3.3). The `paths` field with globs activates rules conditionally. The others don't scope loading.

238. D (TS 3.2). `~/.claude/commands/` is user scope, so it's available in your projects and isn't shared. `.claude/commands/` is shared with the team, and the other two don't define commands.

239. B (TS 3.1). `/memory` verifies what's loaded. Asking the model isn't reliable, and Git shows only that the file exists.

240. A (TS 3.4). Isolating discovery prevents exhaustion in multi-phase tasks. The others don't.

241. D (TS 3.6). Showing the existing tests tells it what's covered. Generating more and filtering wastes effort, and neither JSON output nor a schedule shows existing coverage.

242. B (TS 3.6). Clear standards improve quality. The others don't define "valuable."

243. A (TS 3.3). Path-scoped rules load only when matching files are in play, so unrelated sessions don't carry them. Rule files aren't deleted, compressed, or dropped after a turn.

244. D (TS 3.5). Having Claude ask questions first surfaces considerations you may have missed. It still implements the feature, doesn't aim to shorten responses, and doesn't replace tests.

245. A (TS 3.2). A skill lives in `.claude/skills/<name>/SKILL.md`. CLAUDE.md is memory, `.mcp.json` configures servers, and `package.json` is unrelated.

246. C (TS 3.6). `-p` (or `--print`) is correct. The others aren't options in the guide. Work guide sample 10 as well.

247. C (TS 3.6). `-p` is the supported non-interactive mode. `CLAUDE_HEADLESS` doesn't exist, and the others are workarounds that don't configure the mode.

248. C (TS 3.4). It's a small, clear change. The others are disproportionate.

249. B (TS 3.3). Conditional loading reduces context. They're still guidance, and not enforcement.

250. D (TS 3.3). Glob rules apply by file type, regardless of location. Copies drift, the root loads everywhere, and skills rely on memory. Work guide sample 6 as well.

251. D (TS 3.3). Multiple globs in `paths` work. The others aren't valid or shared.

252. C (TS 3.6). Prior findings in context prevent duplicates. Deleting loses history, and the others don't address it.

253. D (TS 3.4). Architectural implications and several valid approaches are reasons to explore first. A short ticket, a wish for speed, or a one-line change don't call for it.

254. C (TS 3.2). It runs the skill in an isolated sub-agent context, so its output doesn't pollute the main conversation. The others don't isolate.

255. B (TS 3.4). Large-scale changes with many files benefit from planning. The others are small and clear.

256. C (TS 3.1). Directory-level files scope to that subtree. User-level isn't shared, root loads everywhere, and a skill is on-demand.

257. B and D (TS 3.6). `--output-format json` and `--json-schema` are the guide's flags. The other three aren't real options in the guide, and `--batch` is a common distractor.

258. B (TS 3.1). A file in the repository reaches everyone who pulls it, so it's project-scoped. It isn't personal, it persists past sessions, and it doesn't reach other repositories.

259. C (TS 3.1). That's what distinguishes user level. The others are false.

260. A (TS 3.4). Well-scoped fixes don't need a plan phase. The others involve architectural decisions.

261. B (TS 3.5). These problems interact, so describe them together and ask for one coherent fix. Continuing singly keeps regressing, fresh sessions lose the connection, and disabling tests hides the problem.

262. B (TS 3.6). `--json-schema` is named in the guide. The other three look plausible, and they aren't real flags in it.

263. D (TS 3.5). Interviewing surfaces considerations such as invalidation and failure modes.

264. B (TS 3.4). It returns a summary, which keeps verbose discovery out of the main context. A full transcript would defeat the purpose, and it doesn't write CLAUDE.md.

265. B (TS 3.5). A specific input and expected output pins down the behavior. General requests to be robust or careful don't say what's wrong, and a CLAUDE.md note is too general for this bug.

266. C (TS 3.2). The point of isolation is that the working stays out, while the outcome returns.

267. D (TS 3.4). Multiple valid approaches call for planning. The others risk costly rework.

268. B (TS 3.1). Always-applicable standards go in CLAUDE.md. Occasional, task-specific workflows go in skills.

269. A (TS 3.6). An independent instance has no earlier reasoning to defend, so it catches more (TS 4.6). It's not about pricing, CLI restrictions or repository access.

270. D and E (TS 3.6). Prior findings and existing tests show what's already been said or covered. Being conservative, random seeds and shorter timeouts don't provide that information.

271. B (TS 3.6). Schema-enforced JSON removes fragile parsing. More regex cases keep breaking, manual posting loses the automation, and a larger model doesn't guarantee a format.

272. B (TS 3.4). Explore isolates discovery and returns a summary. Running it twice uses more, `max_tokens` limits output rather than context, and CLAUDE.md shouldn't hold discovery output.

273. B and D (TS 3.1). Those are true. Multiple files can load, project files are shared, and user files affect only that user.

274. C (TS 3.5). Real failure output shows exactly what's wrong. Encouragement or "it still doesn't work" gives nothing to act on.

275. D (TS 3.2). Project commands are shared through version control. Build today: commands are merged into skills, and `.claude/skills/changelog/SKILL.md` also creates `/changelog`.

276. A (TS 3.4). Revealed complexity changes the right mode. The others ignore the new information.

277. B (TS 3.2). A checked-in definition is versioned and available to CI. A laptop's command, a verbal note or a log entry can't be relied on by a pipeline.

278. D (TS 3.2). `argument-hint` prompts for required parameters. `context: fork` isolates context, a CLAUDE.md description doesn't prompt, and `allowed-tools` deals with tool access.

279. C (TS 3.6). Session context biases self-review. An independent instance does better.

280. B (TS 3.1). User-level instructions aren't shared. Moving them to project level fixes it. The others are false.

281. A and C (TS 3.4). Those are architectural. The rest are simple, well-scoped changes.

282. C (TS 3.2). The guide teaches `allowed-tools` to restrict tools. Build today: current docs describe it as tools usable without asking permission. Use deny rules to actually block tools.

283. D (TS 3.5). Examples communicate transformations most effectively.

284. A (TS 3.3). Scoped rules load what's relevant, regardless of location. The others load everything, rely on memory, or drift.

285. B and E (TS 3.2). Those are named by the guide. The rest aren't guide options.

286. A and B (TS 3.3). Spread-out conventions and single maintenance are the reasons. Rules aren't guarantees, and scoping to a directory can use CLAUDE.md too.

287. A (TS 3.3). Path-specific rules are files in `.claude/rules/` with `paths` frontmatter.

288. C (TS 3.5). Independent issues can be fixed one at a time, with less risk. Combining them adds no benefit, and ignoring two or redesigning everything isn't proportionate.

289. A (TS 3.2). Personal variants with different names avoid affecting teammates. The others change shared behavior, or are fragile.

290. B (TS 3.5). Complete requirements guide iterations. Late tests can simply mirror the code.

291. C (TS 3.3). `**/` matches any directory, and `*.test.ts` the file name.

292. B (TS 3.6). Fixtures and criteria for valuable tests directly shape what gets generated. Calendars and deployment dates aren't relevant, and "be good" gives no criteria.

293. D (TS 3.3). The pattern doesn't match that file name. Add `**/*.spec.ts`.

294. D (TS 3.6). These flags enforce structure. Text parsing and prompt-only requests are fragile.

295. C (TS 3.3). It's conditional upon matching paths. It isn't always loaded, or manually invoked.

296. A (TS 3.2). The guide's example is exactly this. A fork isolates context, not capabilities. Build today: enforce with deny rules or hooks, as above.

297. C (TS 3.5). Tests define behavior, and failures guide improvements.

298. A and C (TS 3.4). Explore keeps verbose discovery out of the main conversation, returning a summary. It doesn't enforce rules, guarantee correctness, or replace planning.

299. B (TS 3.4). Planning a one-line known fix adds time with no benefit. The other three, several approaches, many files and architecture, are the reasons plan mode exists.

300. D and E (TS 3.5). Examples and tests state exactly what's expected. Stronger adjectives, larger token limits and confidence scores don't clarify the transformation.

301. A (TS 3.1). Selective `@import` by maintainers fits. The others add machinery, load everything, or aren't shared.

302. B (TS 3.1). Project-level files are checked in, so teammates get them. User-level files aren't shared, and emails or output styles aren't how instructions load.

303. A (TS 3.5). It helps with uncertainty. Well-specified work doesn't need it.

304. B (TS 3.1). The guide names `@import`. Current docs write this as `@path/to/file`.

305. A (TS 3.6). It's blocking, so it must be synchronous, and it has to be parseable. Batch has no latency guarantee.

306. D (TS 3.3). One glob rule covers them all. Multiple files drift, the root wastes context, and a hook doesn't provide conventions during authoring.

307. B and E (TS 3.5). When fixes interact, they need to be considered together. Issues that are unrelated, isolated or independently verifiable can be handled one at a time.

308. C (TS 3.1). `.claude/rules/` organizes topics. Deleting loses standards, user-level isn't shared, and copies drift.

309. C and D (TS 3.1). Topic organization and path scoping are its uses. Secrets don't belong there, and personal preferences belong at user level.

310. D (TS 3.1). `@import` keeps it modular and selective. Copies drift, the root loads everything, and manual skills are unreliable.

311. D (TS 3.1). Project-level CLAUDE.md is how CI-invoked Claude Code gets project context. A runner's home file isn't version-controlled with the project.

312. D (TS 3.6). Prior findings with status avoid repeated noise. The others don't address it.

313. B (TS 3.2). Universal standards belong in always-loaded memory. Skills are on-demand.

314. A (TS 3.3). A rule matching everything always loads, so you lose the context savings that scoping provides. It isn't a parse error, it applies to everyone, and it doesn't stop other rules loading.

315. A (TS 3.1). `/memory` shows which memory files are loaded. The others manage context or sessions.

316. A and B (TS 3.2). Project scope is shared, and user scope is personal.

317. A (TS 3.1). Conflicting instructions lead to inconsistent behavior, so remove the conflict. The others aren't features.

318. D (TS 3.5). 2-3 input/output examples usually clarify requirements.

319. A (TS 3.6). The repository's CLAUDE.md gives CI its project context. Logs aren't instructions, user-level files aren't in the repo, and one variable per rule isn't how this is done.

320. C (TS 3.2). Checked-in commands give everyone the same workflow. They aren't faster, aren't loaded into every request, and don't bypass permissions.

321. A (TS 3.4). It lets you explore and design before changing code, which prevents costly rework. It isn't faster generation, it doesn't merge anything, and planned changes can still contain bugs.

322. C (TS 3.5). Add it to the concrete evidence. Vague feedback doesn't help, and the others avoid the issue.

323. A (TS 3.4). This combines planning, discovery isolation and execution. The others skip understanding.

324. C (TS 3.3). Directory files fit one location. Spread-out file types fit globs.

325. D (TS 3.2). User-scoped skills are personal and apply across your projects.

Answers: Domain 4

326. A and C (TS 4.2). Those are named. Schemas handle syntax, and validation remains necessary.

327. A (TS 4.5). Pre-fetch context. Batch can't execute tools mid-request.

328. C (TS 4.2). An example shows the format exactly, which prose often doesn't. A paragraph on importance adds no format detail, and an iteration limit or a second reviewing model doesn't teach the format either.

329. C (TS 4.4). Retries can correct format and structure errors, but they can't create information that the source doesn't contain. They work with schemas and aren't limited to synchronous calls.

330. D (TS 4.5). Blocking workflows need the synchronous API. Results are correlated by `custom_id`, so order isn't the issue.

331. B (TS 4.4). Retries fix format and structure, and they don't create missing data. Fetch the attachment, or return null.

332. B (TS 4.5). Match API to latency needs. Accuracy isn't the difference.

333. B (TS 4.6). Local passes see one file. Integration covers what spans files.

334. A (TS 4.4). Absent data can't be retried into existence, and fabricating is the worst outcome. Null plus a flag is honest, and values from other documents aren't evidence.

335. B (TS 4.1). Naming the specific dates removes the ambiguity. Saying they're important doesn't define them, and over-extracting or raising limits leaves the decision open.

336. C (TS 4.6). A fresh instance lacks the reasoning that made the first session confident. Asking again in that session keeps the bias, a confidence score doesn't remove it, and banning AI review isn't warranted.

337. D (TS 4.1). Measure by category on labelled data. Fewer findings can mean missed bugs.

338. B (TS 4.5). An overnight report can wait up to the batch window. The other three have someone waiting, and batch has no latency guarantee.

339. B (TS 4.3). The model's arguments are the structured data. You don't need to run anything to get them.

340. A and B (TS 4.5). Use `custom_id` to find what failed, then fix the cause before resubmitting. Resubmitting successes wastes money, unchanged retries repeat the failure, and ignoring failures loses data.

341. D (TS 4.2). Examples teach handling, such as informal measurements and nulls. They don't guarantee correctness, and copied values would be a failure.

342. C (TS 4.1). It defines categories and exclusions. Confidence filtering and "be careful" aren't criteria, and ranking keeps the noise.

343. C (TS 4.4). Format errors are correctable. The others are absent information.

344. A and B (TS 4.3). Both give the model honest alternatives to inventing.

345. C and D (TS 4.6). A reviewer without the generator's reasoning catches more, and separate passes avoid dilution. Shared sessions, reading the rationale first, or a single pass bring back those problems.

346. A (TS 4.5). 26 − 24 = 2 hours. Longer intervals could leave an item waiting too long.

347. C (TS 4.1). "Only flag if confident" gives no categories, so it barely improves precision. The other three are specific criteria, and they're what the guide recommends.

348. C (TS 4.5). Results can take up to the 24-hour window, so plan for that. One-minute results aren't promised, batch is cheaper rather than dearer, and results aren't lost after an hour.

349. B (TS 4.6). Give requirements and code, and don't pass the generator's rationale.

350. A (TS 4.1). The guide warns against confidence-based filtering in place of criteria. Calibrated confidence can route review (TS 4.6, 5.5), yet it isn't a substitute.

351. D (TS 4.1). Stated goals tell Claude what "better" means. "Be thorough", repeating the phrase, or generating several versions leaves the criteria undefined.

352. D (TS 4.1). Explicit criteria beat general caution. "Be conservative" and "be thorough" don't define what counts.

353. B (TS 4.4). Surface conflicts rather than resolving them arbitrarily.

354. B (TS 4.2). It shows acceptable code and explains why it's fine, which the model can generalize. "Looks fine" has no reasoning, a bug example doesn't reduce false alarms, and a variable list is irrelevant.

355. A (TS 4.5). Chunking fixes the cause. Resubmitting unchanged fails again, and `tool_choice` or more retries don't make a document smaller.

356. D (TS 4.1). Trust erosion is the stated risk. The model doesn't learn across runs.

357. D (TS 4.5). The guide states 50% savings, a processing window of up to 24 hours, and no latency SLA. It doesn't support multi-turn tool calling within a request, and the other figures are wrong.

358. C (TS 4.3). A nullable field lets the model say a value is absent. Required fields pressure it to invent one, and a prompt rule alone is weaker, while numeric IDs don't address it.

359. C (TS 4.2). Few-shot examples achieve consistently formatted, actionable output.

360. B (TS 4.3). Required fields pressure the model to invent. Nullable fields allow honest absence.

361. B (TS 4.2). Examples in the required format show exactly what to produce. Famous bugs with no format, a pasted style guide, or one overloaded example don't demonstrate the structure.

362. B (TS 4.4). `detected_pattern` lets you group dismissals by the code construct that triggered them. Model version, word count and the model's opinion don't show which patterns cause false alarms.

363. C (TS 4.1). Explicit report and skip lists are concrete. "Senior engineer" and "noisy" are undefined, and reporting everything pushes the work onto developers.

364. A and D (TS 4.1). Disable, improve, then re-enable. The others keep or hide the noise.

365. C (TS 4.1). High false-positive categories undermine trust in the accurate ones. Disabling restores trust while you improve. The others keep the noise.

366. C (TS 4.4). Feedback can produce forced fixes. Keep source evidence and send doubts to review.

367. A (TS 4.5). Refining on a sample finds problems cheaply, before they multiply. Submitting everything first makes every prompt flaw expensive, and skipping validation or spreading across models doesn't address it.

368. B (TS 4.3). Schemas enforce shape, and normalization rules guide values. The guide pairs them.

369. A (TS 4.2). Targeted examples calibrate judgment (TS 5.2). Unsure-based and sentiment-based rules are unreliable.

370. B (TS 4.6). The guide uses it for calibrated review routing. The score still needs calibrating (TS 5.5).

371. C (TS 4.2). Generalization is the benefit. They don't retrain anything, or force matching.

372. B (TS 4.6). One pass covering too much gives shallow, uneven analysis, which is why large reviews get split. Cost, latency and token count are separate concerns.

373. A and B (TS 4.4). Format and placement are fixable. Absent information isn't.

374. A (TS 4.4). It needs the document to re-check, its failed answer, and the specific errors. A bare "try again" gives no direction, the schema alone doesn't show the error, and another document is irrelevant.

375. B and E (TS 4.4). Those two serve analysis and self-checking. The others aren't output fields for this.

376. A and D (TS 4.5). Audits and overnight reports can wait, so they suit batch's window. Pre-merge checks, live replies and IDE suggestions have someone waiting, and batch has no latency SLA.

377. B (TS 4.2). Few-shot examples can demonstrate branch-level coverage reasoning.

378. A (TS 4.5). An item could wait up to the full interval before submission, then up to 24 hours. So the interval can be at most 6 hours. The guide's example of 4-hour windows leaves margin.

379. D (TS 4.3). `any` requires a tool. `stop_reason` is an output, not a setting.

380. D and E (TS 4.1). They name exact categories to report and skip. "Confident", "conservative" and "quality over quantity" leave the decision undefined.

381. A (TS 4.3). `any` guarantees structure and lets the model pick. Build today: `any` returns 400 on Opus 5.5 and Sonnet 5.5. Use `auto` with strict tools and an instruction.

382. B (TS 4.6). Independent instances beat self-review instructions or extended thinking, per the guide.

383. D (TS 4.1). Explicit criteria with examples produce consistency. The others avoid the problem, or substitute an unreliable signal.

384. A (TS 4.2). Examples of varied structures address empty extractions. Making them required invites fabrication.

385. D (TS 4.6). The diff, requirements and criteria are enough to review. The generator's reasoning or opinions remove independence, and file names alone don't contain the code.

386. A (TS 4.6). Prior reasoning biases it. Independence is the fix.

387. A (TS 4.4). A schema guarantees shape, while totals and field placement can still be wrong. Tool use returns structured data and handles long documents, so the other claims are false.

388. C (TS 4.3). Forced selection names the tool. `any` doesn't guarantee which one. Build today: forced selection is rejected on some current models.

389. D (TS 4.1). With no defined criteria, findings vary and precision falls. Vague wording doesn't cause schema errors or API rejections, and it doesn't raise precision.

390. B and D (TS 4.2). Target the cases it gets wrong, and show why one choice beats another. Long transcripts, already-solved cases and bare answers teach little.

391. C (TS 4.2). Target ambiguous cases. More easy cases add little.

392. D (TS 4.4). It gives the specific mismatch to check against the document. Generic requests, stated importance, or repeating a format instruction don't tell the model what's wrong.

393. B (TS 4.2). Few-shot examples handle varied structures. A false rule breaks the other form.

394. A (TS 4.3). Schemas can't check meaning. Validate in code (TS 4.4).

395. A (TS 4.1). It defines what to report and what to skip, in terms of behavior. "Could be misunderstood" flags nearly everything, reviewing all comments isn't a criterion, and comments can mislead.

396. D (TS 4.1). Name the exception. General statements don't define it, and disabling loses real findings.

397. D (TS 4.3). Values in wrong fields are semantic errors, which schemas don't prevent.

398. D (TS 4.6). It avoids attention dilution and contradictions. Work guide sample 12 as well.

399. D (TS 4.5). Targeted resubmission saves cost, and fixes causes such as oversized documents through chunking.

400. B (TS 4.3). Schemas remove syntax errors, and not meaning errors. So you still validate semantics.

401. C (TS 4.2). Contrasting examples reduce false positives while allowing generalization.

402. A (TS 4.4). The others are syntax errors, which tool use eliminates.

403. D (TS 4.4). Teams may legitimately differ, so segment by team and `detected_pattern`, then adapt the criteria. Forcing acceptance, stopping reviews or changing tone doesn't explain the difference.

404. A (TS 4.4). It's a self-correction validation flow, and the discrepancy gets flagged.

405. C (TS 4.3). Those are the guide's flags (TS 3.6). The others aren't.

406. A (TS 4.4). That's what the field is for. Training is out of scope, and a global threshold is imprecise.

407. B (TS 4.2). Showing reasoning enables generalization. The others don't teach the judgment.

408. C (TS 4.2). The guide says 2-4 targeted examples, showing reasoning on ambiguous cases. One is rarely enough, and dozens cost context without improving coverage of hard cases.

409. D (TS 4.4). Distinguish fixable errors from absent information.

410. D (TS 4.6). It's the dilution problem. Focused passes and explicit criteria reduce it.

411. D (TS 4.3). `auto` leaves it optional, so use `any` or forced selection to guarantee it (per the guide).

412. A (TS 4.6). The issue is retained context, and more thinking in that context doesn't remove it.

413. B (TS 4.3). Extraction tools define the schema as their input.

414. C (TS 4.3). It eliminates JSON syntax errors. Build today: `output_config.format` also exists. The exam guide's answer is tool use.

415. C and E (TS 4.3). Schema-enforced tool input removes syntax errors, while a wrong total is a semantic error that it can't detect. Wrong-field and invented values are semantic as well, so they aren't eliminated.

416. A (TS 4.6). A rename across producer and consumer appears only across files, while per-file passes are better at local depth. Keep both, and neither pass is broken.

417. A (TS 4.5). You can't execute tools mid-request. A single-turn structured extraction is fine.

418. D (TS 4.2). Biased examples teach the wrong pattern. Include the distinctions.

419. C (TS 4.3). An enum keeps most values clean, while "other" plus a detail captures new ones. Free text loses structure, rejecting documents loses data, and a boolean per category doesn't scale.

420. D (TS 4.5). Nobody is waiting overnight, so batch gives half the cost. Test generation can run in batch if context is provided, monthly runs lose value, and interactive review must stay synchronous.

421. C (TS 4.3). It keeps the enum while capturing new values in a detail field. Free text loses the enum, rejecting unknowns loses data, and a boolean with a confidence doesn't record the category.

422. B (TS 4.3). It gives an honest option for ambiguity, rather than a forced guess.

423. C (TS 4.1). Specific categorical criteria improve precision where general instructions don't. The others are false.

424. D (TS 4.3). It surfaces a disagreement so that code or people can handle it (TS 4.4, 5.5). It doesn't resolve conflicts, validate syntax, or trigger API retries.

425. A (TS 4.5). `custom_id` correlates them. Order isn't guaranteed.

Answers: Domain 5

426. B (TS 5.1). Stated expectations are a named casualty of summarization.

427. B (TS 5.1). Tool results accumulate disproportionately, so trim in code (for example, with a `PostToolUse` hook).

428. A (TS 5.4). Structured state with a manifest gives crash recovery.

429. A (TS 5.5). Limited reviewers should go where risk is highest: low confidence, and ambiguous or contradictory sources. Newness, file size and uniform sampling ignore risk.

430. C (TS 5.2). Explicit triggers with examples give consistent judgment. "Best judgment" gives no criteria, and sentiment and self-reported confidence are unreliable proxies.

431. B (TS 5.1). A case facts block keeps amounts, dates and IDs exact. Summaries compress them, and the others waste context or annoy the customer.

432. B (TS 5.3). A generic status hides what failed and what was tried, so the coordinator can't choose well. It isn't too long, it doesn't reveal secrets, and it isn't read as success.

433. D (TS 5.1). Metadata lets synthesis attribute claims and interpret them in time (TS 5.6). It's not about parsing speed or an API rule, and the coordinator still has to evaluate findings.

434. D (TS 5.1). Leading with key findings and using explicit headers counter the tendency to miss the middle. Removing headers or placing key items mid-input makes it worse, and reversing order doesn't solve it.

435. B and E (TS 5.2). Those are named. The others are unreliable proxies, or unrelated.

436. C (TS 5.4). A scratchpad holds facts outside the conversation, so they survive as context fades. It doesn't enlarge context, stop summarization, or send anything to tools automatically.

437. C (TS 5.3). Continue with what's available, and annotate gaps. Terminating discards good work, hiding failures misleads, and rerunning everything until all succeed is wasteful.

438. A (TS 5.3). The difference drives the decision: retry or find alternatives, or accept that nothing exists. Treating them alike leads to false conclusions or pointless retries, and the API doesn't detect it.

439. B (TS 5.2). The customer didn't ask for a human, and it's within capability.

440. B (TS 5.4). Findings written to a file survive context limits and can be re-read. It doesn't grow the context window, it isn't for credentials, and it doesn't replace reading source files.

441. A (TS 5.6). Without excerpts and sources, a reader can't check a claim, so mappings must survive merging. Dropping them doesn't cause schema failures, rejections or rounding.

442. A (TS 5.3). The structured context is useful, and a raw dump risks leaking secrets.

443. C (TS 5.4). Vague references to "typical patterns", in place of specifics found earlier, suggest that details have faded. Valid JSON, new tools and more questions don't indicate it.

444. B (TS 5.6). With dates attached, a difference in time isn't mistaken for a contradiction. Preferring the larger, deleting older sources, or explaining differences away loses evidence.

445. D (TS 5.3). Coverage annotations keep the report useful and honest. Leaving out the gap misleads, general knowledge has no sources, and refusing throws away three good results.

446. C (TS 5.6). When findings are compressed, the claim-to-source links vanish unless they're kept explicitly. Schemas don't strip URLs, and neither of the other two is a real limitation.

447. C (TS 5.4). Summaries carry knowledge forward without verbosity.

448. C (TS 5.6). Match form to content: tables for figures, prose for narrative, and lists for technical findings. Forcing one format makes some content harder to read and loses structure.

449. A (TS 5.4). A concise record of what's complete, where outputs are, and what remains lets the coordinator resume. Inline raw outputs are bloated, a sentence is too vague, and keys must never go there.

450. A (TS 5.2). Sentiment measures mood, and not complexity: an angry customer can have an easy problem, and a calm one a policy exception. Neither availability nor cost is the main objection.

451. D (TS 5.4). Subagents isolate verbose exploration, so the main agent keeps its overview. They can read repositories, pasting everything exhausts context, and unrecorded sessions lose the results.

452. D (TS 5.5). A high average can hide a failing document type or field, so validate by segment first. No fixed number is always too low, accuracy can be measured here, and the figure isn't about latency.

453. B and C (TS 5.6). Keep both values, with sources, dates and methods, so that readers can judge. Silently picking one, deleting both, or blending them hides the disagreement.

454. A (TS 5.4). It's degradation. Persist findings and manage context.

455. A (TS 5.3). It shows which findings are well-supported and where sources were unavailable, so readers don't mistake gaps for absence. Model versions, tokens and authorship don't address that.

456. D (TS 5.1). Models process the start and end reliably, and may omit the middle.

457. D (TS 5.5). Raw scores aren't probabilities until they're checked against labelled data. A blanket 90% is arbitrary, and zero review volume isn't a goal that shows quality.

458. D (TS 5.6). Show both with sources, and note the conflict. Choosing one by reputation, averaging, or omitting hides the disagreement from the reader.

459. B (TS 5.2). Explicit criteria with few-shot examples teach the judgment. Iteration limits don't, sentiment is an unreliable signal, and a shorter prompt removes guidance.

460. C (TS 5.6). Downstream agents preserve and merge the mappings.

461. A and D (TS 5.5). Segment analysis shows whether all types and fields perform well, and calibration makes thresholds meaningful. Stated scores and overall accuracy can mislead, and audit sampling should continue.

462. B (TS 5.2). Frustration alone isn't a trigger, unless they ask.

463. A (TS 5.2). Explicit criteria with examples calibrate judgment. Work guide sample 3 as well.

464. C and D (TS 5.1). Transactional facts and expectations go in. The others bloat it.

465. C (TS 5.3). Hiding failures and terminating on one failure are both named. The other pairs describe recommended practices such as partial results and local retries.

466. B (TS 5.2). Policy gaps warrant escalation, and the agent shouldn't invent policy.

467. D (TS 5.2). Multiple matches require clarification, and not heuristics.

468. C (TS 5.1). Fix what's consuming it. Bigger capacity doesn't fix attention.

469. A (TS 5.1). Specifics such as amounts, dates and stated expectations get condensed into vague language. System prompts, tool definitions and API settings aren't part of what's summarized.

470. C (TS 5.3). Knowing the failure, what's collected, and an alternative lets it choose among options. A generic message wouldn't allow that, and coordinators aren't limited to retrying or terminating.

471. C (TS 5.5). Stated and actual accuracy differ here, so thresholds need calibrating on labelled data. Confidence and accuracy should relate, a stated score can be wrong, and a schema can't fix this.

472. D (TS 5.2). Self-reported confidence doesn't track real complexity, so it escalates the wrong cases. Models can express uncertainty and the API allows the tool, so the other reasons are false.

473. A (TS 5.4). `/compact` reduces context usage. `/memory` inspects loaded files.

474. D (TS 5.2). Policy gaps are named triggers. Sentiment and self-reported confidence are unreliable.

475. A and E (TS 5.4). Subagents isolate verbose work, and scratchpads keep findings outside the conversation. Reading everything exhausts context, and keeping no record invites degradation.

476. A (TS 5.5). Consistent performance across segments comes first.

477. A (TS 5.5). A stratified sample of automated output measures errors and reveals new patterns. Low-confidence review misses errors in what's automated, and complaints or one type give partial coverage.

478. C (TS 5.4). Isolation keeps the main context on coordination. Subagents can still be wrong, per-token prices don't change, and returning all the raw output would defeat the purpose.

479. A and B (TS 5.3). Hiding a failure as an empty success misleads, and stopping everything for one failure discards good work. The other three are the recommended behaviors.

480. D and E (TS 5.4). Vagueness and inconsistency about code it already examined signal degradation. Short sessions, simple bugs and specific citations suggest that context is healthy.

481. A (TS 5.3). Failure type, query, partial results and alternatives let the coordinator choose a recovery. A generic status hides details, an empty success misleads, and a fatal stop wastes other work.

482. C (TS 5.3). Transient failures can often be resolved locally, so retry first, and report only what can't be fixed. Immediate escalation adds noise, an empty success hides it, and stopping over-reacts.

483. B (TS 5.1). Include it in each prompt, outside the summarized history, so it stays exact and present. The model has no memory beyond each request, summaries condense it, and tool descriptions are the wrong place.

484. C (TS 5.1). Requests are stateless, so continuity requires the history.

485. C and E (TS 5.3). The attempted query and partial results show what was done and what exists. Token usage, the date and prompt length don't inform recovery.

486. B (TS 5.5). Keep automating strong segments, and route the weak one to people. The overall figure hides that segment, one weak segment doesn't invalidate the rest, and hiding it isn't honest reporting.

487. C (TS 5.5). That's the value of sampling, so recalibrate and add the segment.

488. D (TS 5.2). It's still ambiguous, so ask for another identifier. Recent activity is a guess, giving up isn't necessary, and acting on both risks touching the wrong account.

489. B (TS 5.1). Structured per-issue data keeps each order's facts separate. A combined paragraph blurs them, a reminder is only guidance, and a session per message loses continuity.

490. A and E (TS 5.2). Ask for more identifiers, and take no account action until it's clear. Heuristics such as value, plausibility or result order can act on the wrong customer.

491. B and C (TS 5.5). Low confidence and contradictory sources carry the most risk. Page count isn't a risk measure, and sending everything ignores limited capacity.

492. A and D (TS 5.1). Models handle the start and end of long inputs best, so lead with key findings and use headers to organize details. Removing structure or burying key items in the middle makes it worse.

493. D (TS 5.2). It's a real lack of progress. The others are unrelated.

494. D (TS 5.5). A uniform sample gives small, risky segments very few items, so their error rates can't be measured. Stratifying isn't always cheaper, and it doesn't remove human review.

495. D (TS 5.6). Conflicts pass upward annotated, before synthesis.

496. B (TS 5.2). An explicit request is honored at once. Investigating first, asking why, or offering a discount all resist what the customer asked for.

497. D (TS 5.1). Modify upstream output for limited downstream budgets.

498. A (TS 5.3). Valid empties are successes. Mislabeling triggers needless recovery.

499. B (TS 5.6). Separate sections keep established and contested findings distinct. Equal confidence overstates, deleting contested material loses information, and marking everything contested says nothing.

500. B (TS 5.2). Honor the explicit request. A quick answer is acceptable, but don't make it a condition.